

FUSION2 User Manual

Command-line lidar processing tools for forest monitoring

Jacob Strunk

Contents

List of Figures	9
List of Tables	11
Preface	13
Which tool do I need?	13
A typical workflow	13
How this manual is organized	14
Where the examples come from	14
Building this manual	14

I

Getting started

1	Installing FUSION2	17
1.1	Download a release	17
1.2	Put the tools on your PATH	17
1.3	Check the installation	17
1.4	Build from source	18
2	Quick start	19
2.1	Build the ground surface	19
2.2	Build the canopy height model	19
2.3	Compute gridded forest metrics	19
2.4	Find tree tops	19
2.5	Next steps	19
3	Conventions every tool shares	21
3.1	Command-line syntax	21
3.2	Inputs: one file or a whole folder	21
3.3	Outputs	21
3.4	Point filtering: <code>/class</code> and <code>/return</code>	22
3.5	Units	22
3.6	Heights above ground	22
3.7	Missing values: <code>/nodata</code> and <code>/noheight</code>	23
3.8	Coordinate systems	23

II

Worked examples, tool by tool

4	The example tile	27
4.1	The tile	27
4.2	Other inputs the examples use	27
4.3	How the examples were produced	27
4.4	How to read a tool chapter	28
5	catalog: summarize point cloud files	29
5.1	What it does	29
5.2	Example commands	29
5.3	Options used	29
5.4	Outputs	29
5.5	Results	29
5.6	Checks	30
6	filterdata: keep points by return, class, or elevation	33
6.1	What it does	33
6.2	Example commands	33
6.3	Options used	33
6.4	Outputs	34
6.5	Results	34
6.6	Checks	35
7	thindata: keep at most one point per cell	37
7.1	What it does	37
7.2	Example commands	37
7.3	Options used	37
7.4	Outputs	37
7.5	Results	37
7.6	Checks	38
8	groundfilter: build the ground surface	39
8.1	What it does	39
8.2	Example commands	39
8.3	Options used	39
8.4	Outputs	39
8.5	Results	40
8.6	Checks	42
9	clipdata: cut out boxes, plots, and height ranges	45
9.1	What it does	45
9.2	Example commands	45
9.3	Options used	45
9.4	Outputs	46

9.5	Results	46
9.6	Checks	48
10	canopymodel: canopy height surface	49
10.1	What it does	49
10.2	Example commands	49
10.3	Options used	49
10.4	Outputs	50
10.5	Results	50
10.6	Checks	51
11	returndensity: point density and first-return share	53
11.1	What it does	53
11.2	Example commands	53
11.3	Options used	53
11.4	Outputs	53
11.5	Results	53
11.6	Checks	54
12	densitymetrics: returns by height layer	55
12.1	What it does	55
12.2	Example commands	55
12.3	Options used	55
12.4	Outputs	55
12.5	Results	56
12.6	Checks	56
13	gridmetrics: the full metric grid	59
13.1	What it does	59
13.2	Example commands	59
13.3	Options used	59
13.4	Outputs	60
13.5	Results	60
13.6	Checks	61
14	cloudmetrics: one summary for a file or for each plot	63
14.1	What it does	63
14.2	Example commands	63
14.3	Options used	63
14.4	Outputs	64
14.5	Results	64
14.6	Checks	64

15	topometrics: slope and aspect	67
15.1	What it does	67
15.2	Example commands	67
15.3	Options used	67
15.4	Outputs	67
15.5	Results	67
15.6	Checks	68
16	gridsurfacestats: surface roughness and volume	71
16.1	What it does	71
16.2	Example commands	71
16.3	Options used	71
16.4	Outputs	71
16.5	Results	71
16.6	Checks	72
17	canopymaxima: find tree tops	73
17.1	What it does	73
17.2	Example commands	73
17.3	Options used	73
17.4	Outputs	73
17.5	Results	73
17.6	Checks	74
18	treeseq: outline tree crowns	77
18.1	What it does	77
18.2	Example commands	77
18.3	Options used	77
18.4	Outputs	77
18.5	Results	77
18.6	Checks	79
19	pipeline: run tools tile by tile	81
19.1	What it does	81
19.2	Example commands	81
19.3	Options used	81
19.4	Outputs	82
19.5	Results	82
19.6	Checks	82
20	Processing many tiles	85
20.1	When you only need gridmetrics: its own batch/tiled mode	85
20.2	Single-stage runs of any other tool: <code>pipeline.exe /tool:<name></code>	85
20.3	The canonical pipeline: ground, then canopy height, then tree tops	85

20.4	Crown segmentation instead of tree tops	85
20.5	Terrain metrics only	86
20.6	Point-cloud-only chain (no raster stage at all)	86
20.7	The <code>_processing/</code> subfolder and the state tracking file	86
20.8	Resuming and retrying	86
20.9	Valid chaining	87

III	Reference
------------	------------------

21	Command-line options, tool by tool	91
21.1	<code>gridmetrics</code>	91
21.2	<code>cloudmetrics</code>	93
21.3	<code>canopymodel</code>	94
21.4	<code>canopymaxima</code>	94
21.5	<code>treeseq</code>	94
21.6	<code>groundfilter</code>	94
21.7	<code>clipdata</code>	95
21.8	<code>filterdata</code>	96
21.9	<code>thindata</code>	96
21.10	<code>returndensity</code>	96
21.11	<code>topometrics</code>	96
21.12	<code>catalog</code>	97
21.13	<code>pipeline</code>	97
21.14	<code>gridsurfacestats</code>	98
21.15	<code>densitymetrics</code>	98
22	Built-in help for every tool	99
22.1	<code>catalog</code>	99
22.2	<code>filterdata</code>	99
22.3	<code>thindata</code>	99
22.4	<code>groundfilter</code>	100
22.5	<code>clipdata</code>	100
22.6	<code>canopymodel</code>	101
22.7	<code>returndensity</code>	101
22.8	<code>densitymetrics</code>	102
22.9	<code>gridmetrics</code>	102
22.10	<code>cloudmetrics</code>	103
22.11	<code>topometrics</code>	104
22.12	<code>gridsurfacestats</code>	104
22.13	<code>canopymaxima</code>	105
22.14	<code>treeseq</code>	105
22.15	<code>pipeline</code>	105

23	Metric definitions	107
23.1	NA vs. 0 – the /nodata and /noheight sentinel convention	107
23.2	Elevation Statistics & Percentiles	107
23.3	Canopy Cover & Structure Metrics	108
23.4	Pulse Density & Return Type Ratios	109
23.5	Surface Area Ratio, Roughness & Cut/Fill Volume	110
23.6	Vertical Density Stratum Bands & Per-Stratum Statistic Bundles	110
23.7	Novel 2D & 3D Experimental Spatial Metrics	112
24	C++ library interface (libfusion_core)	115
24.1	Point Cloud I/O (fusion::lidar)	115
24.2	Raster I/O & GDAL Engine (fusion::raster)	117
24.3	Parallel Batch Engine (fusion::batch)	118
24.4	Status Logging (fusion::batch)	118
24.5	Experimental Lidar Metrics (fusion::metrics)	119
24.6	CLI Argument Parsing (fusion::cli)	119
	Appendices	121
A	Unit tests	123
A.1	What each test group covers	123
A.2	Results for this release	124
B	Checks on the example tile	125
B.1	Summary	125
B.2	Every check	126
C	Legacy FUSION programs not in FUSION2	135
C.1	Summary of Current FUSION2 Toolset	135
C.2	Legacy FUSION functionality missing from FUSION2	135
C.3	Option and Feature Gaps Within Ported Tools	138
C.4	Architectural Summary	139
D	Changelog	141
D.1	[Unreleased]	141
D.2	[0.0.2] - 2026-09-26	141
D.3	[0.0.1] - 2026-09-09	142

List of Figures

Figure 5.1	catalog: points per 10 ft cell from the /density raster, with the CSV report's bounding box outlined when the raster sits on the tile.	30
Figure 6.1	filterdata: points kept by each filter (blue) over all points in the side-view strip (grey). ...	34
Figure 7.1	thindata: points kept (orange) and original points (grey) in a 20 x 20 ft window, with the 2 ft thinning grid.	38
Figure 8.1	groundfilter: ground elevation with contour lines.	40
Figure 8.2	groundfilter: FUSION2 and lidR reference ground surfaces along the side-view strip, with the points written as ground.	41
Figure 8.3	groundfilter: FUSION2 ground against the lidR reference, cell by cell (left) and as a difference map (right).	42
Figure 9.1	clipdata: box and per-plot clips over the tile (left), and everything outside the plots (right).	47
Figure 9.2	clipdata: the height-limited clip through the box, in height above ground.	47
Figure 10.1	canopymodel: plain, smoothed, and slope-adjusted canopy height models on one height scale.	50
Figure 10.2	canopymodel: plain and smoothed canopy surfaces over the points in the side-view strip.	51
Figure 11.1	returndensity: point density (left) and first-return percent (right).	54
Figure 12.1	densitymetrics: returns per height layer for the whole tile (top) and each cell's share of returns by layer (bottom).	56
Figure 13.1	gridmetrics: six representative metric bands, each on its own scale.	60
Figure 14.1	cloudmetrics: height percentiles for the whole tile and each plot.	64
Figure 15.1	topometrics: slope with downhill arrows and contours, from the FUSION2 ground surface (left) and the USGS 3DEP DTM of the same ground (right). The dashed square marks the tile.	68
Figure 16.1	gridsurfacestats: surface area ratio, roughness, and volume difference between the canopy and its smoothed version.	72
Figure 17.1	canopymaxima: tree tops over the canopy height model.	74
Figure 18.1	treeseq: crown outlines (orange) and canopymaxima tree tops (aqua) over the canopy height model.	78
Figure 18.2	treeseq: crown area against crown maximum height.	79
Figure 19.1	pipeline: stitched canopy with tree tops (left), and differences from the single-run canopy (middle) and ground (right). Dashed lines mark tile boundaries.	82

List of Tables

Table 0.1	FUSION2 tools by task.	13
Table 1.1	Contents of a FUSION2 release zip.	17
Table 3.1	The three accepted option spellings.	21
Table 3.2	<code>/class</code> settings.	22
Table 3.3	<code>/return</code> settings.	22
Table 4.1	The example tile.	27
Table 4.2	The example run the manual's worked examples come from.	28
Table 5.1	Options used in the <code>catalog</code> example.	29
Table 5.2	Automated checks on the example run of <code>catalog</code>	30
Table 6.1	Options used in the <code>filterdata</code> examples.	33
Table 6.2	Automated checks on the example run of <code>filterdata</code>	35
Table 7.1	Options used in the <code>thindata</code> example.	37
Table 7.2	Automated checks on the example run of <code>thindata</code>	38
Table 8.1	Options used in the <code>groundfilter</code> example.	39
Table 8.2	Automated checks on the example run of <code>groundfilter</code>	42
Table 9.1	Options used in the <code>clipdata</code> examples.	45
Table 9.2	Automated checks on the example run of <code>clipdata</code>	48
Table 10.1	Options used in the <code>canopymodel</code> examples.	49
Table 10.2	Automated checks on the example run of <code>canopymodel</code>	51
Table 11.1	Options used in the <code>returndensity</code> example.	53
Table 11.2	Automated checks on the example run of <code>returndensity</code>	54
Table 12.1	Options used in the <code>densitymetrics</code> example.	55
Table 12.2	Automated checks on the example run of <code>densitymetrics</code>	56
Table 13.1	Options used in the <code>gridmetrics</code> example.	59
Table 13.2	Automated checks on the example run of <code>gridmetrics</code>	61
Table 14.1	Options used in the <code>cloudmetrics</code> examples.	63
Table 14.2	Automated checks on the example run of <code>cloudmetrics</code>	64
Table 15.1	Options used in the <code>topometrics</code> examples.	67
Table 15.2	Automated checks on the example run of <code>topometrics</code>	68
Table 16.1	Options used in the <code>gridsurfacestats</code> example.	71
Table 16.2	Automated checks on the example run of <code>gridsurfacestats</code>	72
Table 17.1	Options used in the <code>canopymaxima</code> example.	73
Table 17.2	Automated checks on the example run of <code>canopymaxima</code>	74
Table 18.1	Options used in the <code>treeseg</code> example.	77
Table 18.2	Automated checks on the example run of <code>treeseg</code>	79
Table 19.1	Options used in the <code>pipeline</code> example.	81
Table 19.2	Automated checks on the example run of <code>pipeline</code>	83
Table A.1	Unit test groups and what each protects.	123
Table B.1	Automated check results by tool on the example run.	125
Table B.2	Every automated check on the example run, in the order it ran.	126

Preface

FUSION2 is a set of 15 command-line tools for turning airborne lidar point clouds into the products forest inventory and monitoring work depends on: ground surfaces, canopy height models, gridded height and density metrics, plot-level summaries, and individual tree tops and crowns. Each tool is a single Windows executable that reads LAS or LAZ point clouds (or GeoTIFF rasters) and writes GeoTIFF rasters, LAS/LAZ point clouds, or CSV and SQLite tables that open directly in GIS software and in R or Python.

FUSION2 is derived from the stand-alone command-line programs in legacy FUSION (version 4.40), written by Robert J. McGaughey at the USDA Forest Service Pacific Northwest Research Station. FUSION2 keeps legacy FUSION's tool names and option style (`/option:value`), so existing batch scripts port with few changes. It replaces legacy FUSION's own `.dtm` raster format with GeoTIFF, reads and writes LAS 1.0-1.4 and LAZ directly, and adds batch processing across many tiles. Appendix C. lists the legacy FUSION programs FUSION2 does not include.

Which tool do I need?

Table 0.1: FUSION2 tools by task.

Task	FUSION2 tool	Chapter
Summarize point counts, extents, and density of a set of files	<code>catalog</code>	Chapter 5.
Keep points by return number, classification, or elevation	<code>filterdata</code>	Chapter 6.
Reduce point density to at most one point per cell	<code>thindata</code>	Chapter 7.
Classify ground points and build a ground surface (DEM)	<code>groundfilter</code>	Chapter 8.
Cut out boxes, plots, or height ranges of points	<code>clipdata</code>	Chapter 9.
Build a canopy height model	<code>canopymodel</code>	Chapter 10.
Map point density and the share of first returns	<code>returndensity</code>	Chapter 11.
Map returns by height layer	<code>densitymetrics</code>	Chapter 12.
Compute gridded height, cover, and intensity metrics	<code>gridmetrics</code>	Chapter 13.
Compute the same metrics for a whole file or for each plot	<code>cloudmetrics</code>	Chapter 14.
Compute slope and aspect from a ground surface	<code>topometrics</code>	Chapter 15.
Measure surface roughness and volume between two surfaces	<code>gridsurfacestats</code>	Chapter 16.
Find individual tree tops	<code>canopymaxima</code>	Chapter 17.
Outline individual tree crowns	<code>treeseg</code>	Chapter 18.
Run several tools tile by tile over a large area	<code>pipeline</code>	Chapter 19., Chapter 20.

A typical workflow

Most projects run the tools in the same order, because later tools need the ground surface that an earlier tool builds:

1. **Check the data** with `catalog`: point counts, extents, and density per file.
2. **Build the ground surface** with `groundfilter`. Every tool that reports height above ground reads this surface.
3. **Build the canopy height model** with `canopymodel`, using that ground surface.
4. **Compute area-based metrics** with `gridmetrics` (for wall-to-wall maps) and `cloudmetrics` (for field plots).
5. **Detect trees** with `canopymaxima` and `treeseq`, working from the canopy height model.
6. **Scale up** with `pipeline`, which runs the same chain tile by tile across a whole project area and stitches the results.

How this manual is organized

- **Getting started** covers installing the tools, a four-command quick start, and the conventions every tool shares: option syntax, point filtering, units, missing values, and coordinate systems.
- **Worked examples, tool by tool** gives each tool its own chapter. Each chapter explains what the tool does, shows the exact commands used on one example lidar tile, explains every option in those commands, shows figures drawn from the tool's actual output, and lists the automated checks that output passed.
- **Reference** lists every option of every tool, each tool's built-in help text, the definitions of every metric, and the C++ library interface for developers.
- **Appendices** report how FUSION2 is tested: the unit tests that run on every build, and the full set of automated checks on the example tile. They also list the legacy FUSION programs not included and the changelog.

Where the examples come from

Every command, figure, and check in the worked-example chapters comes from one scripted run of all 15 tools on a single example tile, kept in the companion repository [FUSION2-examples](#). That run checks each tool's output against an independent calculation made in R with the `lidR`, `terra`, and `sf` packages. The commands shown are exactly the commands that run executed. Chapter 4. describes the tile and how to reproduce the run.

Building this manual

The manual is a Quarto book in the FUSION2 repository's `docs/` folder. `build.ps1` renders it to an HTML site (`docs/output/`) and a PDF (`docs/pdf/FUSION2_Manual.pdf`), and bundles the PDF with each release. To render it on its own:

```
quarto render docs
```



Getting started

1	Installing FUSION2	17
1.1	Download a release	17
1.2	Put the tools on your PATH	17
1.3	Check the installation	17
1.4	Build from source	18
2	Quick start	19
2.1	Build the ground surface	19
2.2	Build the canopy height model	19
2.3	Compute gridded forest metrics	19
2.4	Find tree tops	19
2.5	Next steps	19
3	Conventions every tool shares	21
3.1	Command-line syntax	21
3.2	Inputs: one file or a whole folder	21
3.3	Outputs	21
3.4	Point filtering: <code>/class</code> and <code>/return</code>	22
3.5	Units	22
3.6	Heights above ground	22
3.7	Missing values: <code>/nodata</code> and <code>/noheight</code>	23
3.8	Coordinate systems	23

1. Installing FUSION2

FUSION2 runs on 64-bit Windows. Each tool is a single self-contained executable with no separate libraries to install.

1.1 Download a release

1. Open the [FUSION2 Releases](#) page and download the zip file from the latest release (FUSION2_tools_v<version>-<date>.zip).
2. Unzip it into a folder of your choice, for example C:\FUSION2.

The folder then holds:

Table 1.1: Contents of a FUSION2 release zip.

Item	What it is
15 .exe files	The tools: <code>catalog</code> , <code>filterdata</code> , <code>thindata</code> , <code>groundfilter</code> , <code>clipdata</code> , <code>canopymodel</code> , <code>returndensity</code> , <code>densitymetrics</code> , <code>gridmetrics</code> , <code>cloudmetrics</code> , <code>topometrics</code> , <code>gridsurfacestats</code> , <code>canopymaxima</code> , <code>treeseq</code> , and <code>pipeline</code>
<code>proj_data\proj.db</code>	The coordinate-system database the tools use to read and write map projections
<code>FUSION2_Manual.pdf</code>	This manual
<code>CHANGELOG.md</code>	What changed in each release

Keep `proj_data` in the same folder as the executables. The tools look for it next to themselves. Without it, they still run, but they may not recognize some coordinate systems and will print a warning about `proj.db`.

1.2 Put the tools on your PATH

Adding the folder to your `PATH` lets you run a tool by name from any folder. In PowerShell, for the current user:

```
$dir = "C:\FUSION2"
$sold = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$sold;$dir", "User")
```

Open a new terminal afterwards so the change takes effect. Without this step, run each tool by its full path (C:\FUSION2\gridmetrics.exe).

1.3 Check the installation

Every tool prints its options when run with `/?`:

```
gridmetrics /?
```

If the options list appears, the tools are ready. Chapter 22. collects this help text for every tool.

1.4 Build from source

The FUSION2 repository's [README.md](#) explains how to build the tools with CMake and the `build.ps1` script, including the minimal (GeoTIFF-only) and full-format GDAL build profiles. `build.ps1` also runs the unit tests described in Appendix A. and renders this manual.

2. Quick start

This chapter takes one lidar tile, `my_tile.laz`, from raw points to a ground surface, a canopy height model, a grid of forest metrics, and a list of tree tops, in four commands. The commands are written for the Windows Command Prompt and assume the tools are on your `PATH` (Chapter 1.). Cell sizes are in the point cloud's own horizontal units, feet for the tile used in this manual.

Create the output folder first. Not every tool creates a missing output folder, and some report success without writing anything when the folder does not exist.

```
mkdir out
```

2.1 Build the ground surface

```
groundfilter my_tile.laz /output-raster:out/ground.tif /cellsize:5
```

`groundfilter` finds the points that hit the ground and turns them into a ground elevation raster (a digital elevation model, or DEM) with 5 ft cells. The tools that follow use it to turn each point's elevation into a height above ground (Chapter 8.).

2.2 Build the canopy height model

```
canopymodel my_tile.laz /ground:out/ground.tif /cellsize:3 /output:out/chm.tif
```

`canopymodel` keeps the highest point in each 3 ft cell and subtracts the ground elevation beneath it, giving a canopy height model (CHM): a raster of vegetation height (Chapter 10.).

2.3 Compute gridded forest metrics

```
gridmetrics my_tile.laz out/ground.tif /cellsize:10 /outdir:out
```

`gridmetrics` computes height percentiles, canopy cover, height variability, intensity statistics, and more for every 10 ft cell, and writes them as bands of one GeoTIFF, `out/my_tile_gridmetrics.tif` (Chapter 13.). The ground surface can be given as the second positional argument, as here, or with `/ground:.`

2.4 Find tree tops

```
canopymaxima out/chm.tif /output:out/tree_tops.csv /minht:2
```

`canopymaxima` finds local height peaks on the canopy height model and writes one row per tree top, with its location, height, and an estimated crown diameter (Chapter 17.). `/minht:2` ignores peaks lower than 2 ft.

2.5 Next steps

- Chapter 3. explains the option syntax and the behavior every tool shares, such as which points are filtered out by default and how missing values are written.
- The worked-example chapters, starting with Chapter 4., cover each tool in depth with figures of its output.
- Chapter 20. shows how to run the same steps over hundreds of tiles with `pipeline` or `gridmetrics'` batch mode.

3. Conventions every tool shares

The FUSION2 tools share one option parser, one point filter, and one set of rules for units, missing values, coordinate systems, and table output. This chapter covers those shared rules once, so the tool chapters can focus on what each tool does.

3.1 Command-line syntax

Every tool takes its main input as a **positional** argument, placed on the command line with no option name in front of it. Everything else is an option (`/name:value`) or a flag (`/name`):

```
clipdata my_tile.laz /output:out/box.las /extent:1023780,598900,1023880,599000
```

Three spellings are accepted for every option and flag, and they can be mixed on one command line:

Table 3.1: The three accepted option spellings.

Style	Option	Flag
Windows (used in this manual)	<code>/cellsize:2</code>	<code>/first</code>
Single dash	<code>-cellsize 2</code>	<code>-first</code>
Double dash	<code>--cellsize=2</code>	<code>--first</code>

Option names are not case-sensitive. A tool run with `/?`, `-h`, or `--help` prints its options and exits; Chapter 22. lists that help for every tool. A misspelled option name is not reported as an error; the tool ignores it and uses the option's default, so check a new command's console output for the settings it actually used.

In the Windows Command Prompt, a long command can continue onto the next line with `^` at the end of a line. The worked examples use this layout, one option per line. In PowerShell, use a backtick (```) instead, or put the whole command on one line.

3.2 Inputs: one file or a whole folder

Every tool that reads points accepts either a single `.las/.laz` file or a folder. Given a folder, the tool reads every LAS/LAZ file in it as one combined point cloud. LAS versions 1.0 through 1.4 are read, compressed (LAZ) or not. A COPC file (a LAZ file with a built-in spatial index) is read like any other LAZ file; `gridmetrics`' batch mode also uses its index to read only the parts each tile needs (Chapter 20.).

Tools that need a ground surface (`/ground:`) accept a single raster (GeoTIFF, ERDAS Imagine, or ENVI) or a folder of raster tiles. A folder is joined on the fly into one virtual raster, so a tiled DEM delivery can be used as is.

3.3 Outputs

Create output folders before running a tool. `gridmetrics` creates a missing `/outdir` folder, but other tools do not, and some of them exit without an error message and without writing their output when the folder is missing.

Rasters are written as GeoTIFF, with each band named (for example `elev_p95` or `canopy_cover`) so GIS software and R show the band names. Point clouds are written as LAS or LAZ, chosen by the output file's extension. They keep the input's point format, scale, offsets, GPS-time type, and coordinate system.

Every raster-producing tool (`gridmetrics`, `densitymetrics`, `canopymodel`, `groundfilter`, `returndensity`, `topometrics`, `gridsurfacestats`) can also write its cell values as a table:

- `/output-table:<path>` writes one row per cell. The file extension picks the format: `.csv` for a comma-separated file, `.sqlite` for a SQLite database table.
- `/noraster` skips the GeoTIFF and writes only the table. It is only valid together with `/output-table`, since a run must produce at least one output.

3.4 Point filtering: `/class` and `/return`

Every tool that reads points (`gridmetrics`, `cloudmetrics`, `canopymodel`, `groundfilter`, `clipdata`, `filterdata`, `thindata`, `returndensity`, `densitymetrics`, and point-reading stages run by `pipeline`) filters them the same way before doing anything else.

`/class:<spec>` chooses which ASPRS point classes to keep:

Table 3.2: `/class` settings.

Setting	Points kept
omitted (default)	every class except the noise classes 7 (low point) and 18 (high noise)
<code>/class:all</code> or <code>/class:*</code>	every class, including 7 and 18
<code>/class:2,3,4,5</code> or <code>/class:1-5</code>	only the listed classes (lists, ranges, or both)
<code>/class:~7,9,18</code>	every class except the listed ones (replaces the default 7 and 18)

`/return:<spec>` chooses which returns of each laser pulse to keep:

Table 3.3: `/return` settings.

Setting	Returns kept
omitted (default)	every return
<code>/return:1</code> or <code>/return:1,2</code> or <code>/return:1-2</code>	the listed return numbers
<code>/return:first</code>	the first return of each pulse
<code>/return:last</code>	the last return of each pulse
<code>/return:only</code>	returns of pulses that produced a single return
<code>/return:intermediate</code>	returns that are neither first nor last

Points flagged as withheld in the LAS file are always dropped, whatever `/class` says.

3.5 Units

The tools do no unit conversion. Every distance, height, and cell size is in the input's own units: feet for a point cloud in a feet-based projection, meters for one in a meter-based projection. The example tile in this manual is in US survey feet, so `/cellsize:10` means 10 ft cells and `/minht:2` means 2 ft above ground. On a meter-based tile the same commands would mean 10 m cells and 2 m.

Density outputs are points per square unit of the input: points per square foot for the example tile.

3.6 Heights above ground

Tools that report heights (`gridmetrics`, `cloudmetrics`, `canopymodel`, `densitymetrics`, `clipdata` with `/minz//maxz`) subtract the ground elevation beneath each point, read from the `/ground` raster by bilinear

interpolation between the four nearest cell centers. A point up to one cell outside the ground raster uses the raster's edge value. A point further outside gets no height and is left out.

`/minht` sets the height below which points are left out of the height statistics (default 2). `/heightcut` sets the height that separates canopy from non-canopy returns for canopy cover (it defaults to `/minht`).

3.7 Missing values: `/nodata` and `/noheight`

`gridmetrics` and `cloudmetrics` distinguish two kinds of empty:

- **No points at all** (`/nodata`, default `NA`): nothing landed in the cell or plot. Every value is written as `NA`, including point counts.
- **Points, but none above `/minht`** (`/noheight`, default `0`): the height and intensity statistics are written as `0`, which is a real answer ("no canopy here"). Canopy cover and point counts are computed normally.

Both options take `NA` or a number such as `0` or `-9999`. In GeoTIFF output, `NA` is stored as NaN and registered as the file's NoData value, so R (`terra`), Python (`rasterio`), and GIS software read those cells as missing without extra settings. Chapter 23. gives the full rule.

3.8 Coordinate systems

Outputs carry the input's coordinate system. Rasters get it as GeoTIFF georeferencing. Point clouds keep it in the same form the input used, either a WKT record or GeoTIFF keys. Rasters derived from other rasters (`topometrics`, `gridsurfacestats`, `treeseg`) inherit their input raster's coordinate system. Tables of tree tops and crowns (`canopymaxima`, `treeseg`) give coordinates in that same system. The worked examples check that every raster and point cloud output has one.

II

14.1	What it does	63
14.2	Example commands	63
14.3	Options used	63
14.4	Outputs	64
14.5	Results	64
14.6	Checks	64
15	topometrics: slope and aspect	67
15.1	What it does	67
15.2	Example commands	67
15.3	Options used	67
15.4	Outputs	67
15.5	Results	67
15.6	Checks	68
16	gridsurfacestats: surface roughness and volume	71
16.1	What it does	71
16.2	Example commands	71
16.3	Options used	71
16.4	Outputs	71
16.5	Results	71
16.6	Checks	72
17	canopymaxima: find tree tops	73
17.1	What it does	73
17.2	Example commands	73
17.3	Options used	73
17.4	Outputs	73
17.5	Results	73
17.6	Checks	74
18	treeseq: outline tree crowns	77
18.1	What it does	77
18.2	Example commands	77
18.3	Options used	77
18.4	Outputs	77
18.5	Results	77
18.6	Checks	79
19	pipeline: run tools tile by tile	81
19.1	What it does	81
19.2	Example commands	81
19.3	Options used	81
19.4	Outputs	82
19.5	Results	82
19.6	Checks	82
20	Processing many tiles	85
20.1	When you only need gridmetrics: its own batch/tilled mode	85
20.2	Single-stage runs of any other tool: pipeline.exe /tool:<name>	85
20.3	The canonical pipeline: ground, then canopy height, then tree tops	85
20.4	Crown segmentation instead of tree tops	85
20.5	Terrain metrics only	86
20.6	Point-cloud-only chain (no raster stage at all)	86
20.7	The _processing/ subfolder and the state tracking file	86
20.8	Resuming and retrying	86
20.9	Valid chaining	87

4. The example tile

Every worked example in this part of the manual runs on one lidar tile, so the chapters can be read in order and the outputs of early tools (the ground surface, the canopy height model) feed the tools that follow, just as they would in a real project.

4.1 The tile

Table 4.1: The example tile.

Property	Value
File	<code>data/raw/lidar/TC_1372_forest200ft.laz</code> in FUSION2-examples
Source	A 200 x 200 ft window of lidar tile TC_1372 (NV5 Geospatial, 2022), western Washington
Forest	Closed-canopy conifer forest: about 92% canopy cover, 95th-percentile height about 142 ft
Format	LAS 1.4, point data format 6, LAZ-compressed
Points	70,532 (246 flagged as withheld, which every tool drops)
Classes	66,614 unclassified (1), 3,672 ground (2), 235 low noise (7), 11 high noise (18)
Coordinate system	NAD83(HARN) / Washington South, US survey feet (EPSG:2927), NAVD88 heights in feet
Elevations	Ground about 175-181 ft; highest return 356 ft

Because the tile is in feet, every cell size, height, and distance in the example commands is in feet (Chapter 3).

A closed-canopy tile is a demanding test. Only about 5% of the returns are classified as ground, the canopy is tall, and gaps are narrow, so ground filtering, canopy height, and tree detection all have to work under the conditions where they most often fail.

4.2 Other inputs the examples use

- **Two circular plots.** `clipdata` and `cloudmetrics` use a shapefile of two 30 ft radius plots, `plotA` and `plotB`, placed inside the tile (`out/clipdata/example_plots.shp`). The example script writes this shapefile before running those tools.
- **A clip box.** A 100 x 100 ft box starting 25 ft in from the tile's lower-left corner, given to `clipdata` with `/extent`.
- **An independent ground model.** A USGS 3DEP bare-earth DTM covering the same ground, plus 50 ft on every side (`data/raw/lidar_dtm_examples/site_tc_1372_forest/usgs_3dep_dtm.tif`), used to test `topometrics` on a raster the tools did not produce.

4.3 How the examples were produced

The commands in each tool chapter are the exact commands run by the script `R/analysis/005_example_all_tools.qmd` in FUSION2-examples. That script runs all 15 tools in the order the chapters follow, then checks every output against an independent calculation in R. The figures are drawn from the output files by `005_example_all_tools_REPORT.qmd`, and

`007_export_fusion2_docs_assets.qmd` copies the commands, figures, and check results into this manual. Table 4.2 identifies the run shown.

Table 4.2: The example run the manual’s worked examples come from.

Item	Value
run folder	run_VS013_20260928200605
command log	all_tools_commands_VS001_20260928200702.csv
check log	all_tools_check_log_VS012_20260928200702.csv
tools tested (modified)	2026-09-28 20:04
commands	24
checks	161
checks passed	161
checks warned	0

In the commands, paths are relative to the FUSION2-examples project folder, and `out/` stands for the run’s output folder. To repeat a command, run it from the FUSION2-examples folder after creating `out/` and the subfolders the command writes into (Chapter 3.).

To reproduce the whole run:

1. Clone [FUSION2-examples](#).
2. Put the FUSION2 tools on your `PATH`, or set the environment variable `FUSION2_BIN_DIR` to their folder.
3. Run `R/analysis/005_example_all_tools.qmd` chunk by chunk in RStudio or Positron. It needs the R packages `data.table`, `terra`, `sf`, `lidR`, `ggplot2`, and `RForInvT`.

4.4 How to read a tool chapter

Each tool chapter has the same sections:

1. **What it does:** the tool’s purpose and when to use it.
2. **Example commands:** the commands run on the example tile.
3. **Options used:** what each option in those commands does. The full list of options is in Chapter 21..
4. **Outputs:** the files the tool writes.
5. **Results:** figures drawn from those files, with what to look for.
6. **Checks:** the automated checks run on the outputs. A result of `pass` means the check held. `fail` means the output is wrong or missing. `warn` means the output works but something about it deserves a look, such as missing coordinate-system metadata. Long values are shortened with `...`; Appendix B. lists every check, and the full log is `docs/examples/tables/all_tools_check_log.csv` in the FUSION2 repository.

5. catalog: summarize point cloud files

5.1 What it does

`catalog` is the first tool to run on a new delivery of lidar data. It reads every LAS/LAZ file given to it and reports each file's point count, bounding box, elevation range, LAS point format, and whether it is compressed. With `/density`, it also writes a raster counting the points in each cell, which shows at a glance whether coverage is even or has gaps and stripes.

`catalog` needs no ground surface and changes no data.

5.2 Example commands

```
REM catalog
catalog data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/catalog/catalog.csv ^
/density:10
```

5.3 Options used

Table 5.1: Options used in the `catalog` example.

Option	Value	What it does
(input)	TC_1372_forest200ft.laz	The file to summarize. A folder summarizes every LAS/LAZ file in it, one row per file.
/output	out/catalog/catalog.csv	The CSV report.
/density	10	Also write a point-count raster with 10 ft cells, named after the report (<code>catalog.tif</code>).

5.4 Outputs

- **catalog.csv**: one row per input file, with the columns `FileName`, `PointCount`, `MinX`, `MaxX`, `MinY`, `MaxY`, `MinZ`, `MaxZ`, `Format` (LAS point data format), and `IsCompressed`.
- **catalog.tif**: one band, `point_density`, holding the number of points in each cell. The grid starts at the data's lower-left corner and carries the input's coordinate system.

5.5 Results

Figure 5.1 maps the density raster. The black outline is the bounding box from the CSV report; it sits on the edge of the grid, which confirms the raster is placed on the tile's real coordinates. Counts are fairly even across the tile. Stripes along one direction would mark overlapping flight lines, and empty cells inside the tile would point to missing data.

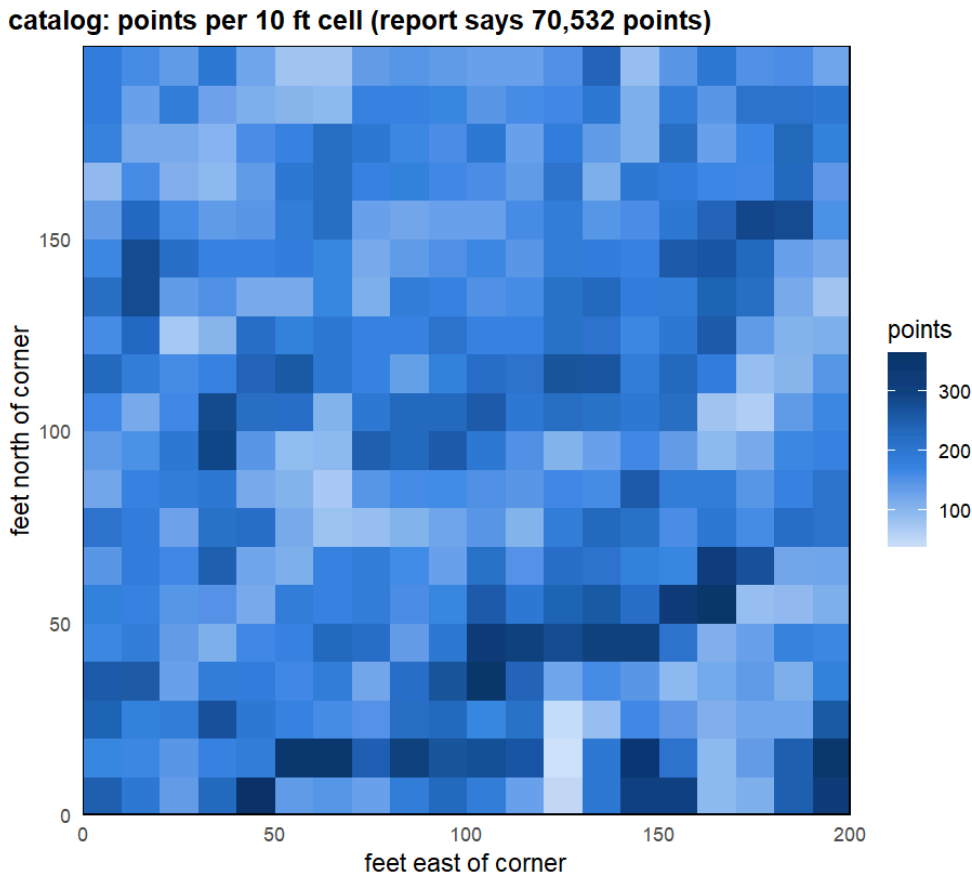


Figure 5.1: `catalog`: points per 10 ft cell from the `/density` raster, with the CSV report’s bounding box outlined when the raster sits on the tile.

5.6 Checks

Table 5.2 lists the checks on this run. They confirm that the reported point count and bounding box match the file, and that the density raster sits on the tile, carries a coordinate system, and counts every point exactly once.

Table 5.2: Automated checks on the example run of `catalog`.

Check	Result	Observed	Expected
catalog: exits with status 0	pass	0	0
CSV report written	pass	catalog.csv	catalog.csv
point count matches the tile	pass	70532	70532
XY bounds match the tile within 0.01 ft	pass	1023755.00, 1023954.98, 598875.00, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
Z range matches the tile within 0.01 ft	pass	175.38, 355.99	175.38, 355.99
/density raster written	pass	1	>= 1 .tif
density raster: raster sits on the tile’s real coordinates	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...

Check	Result	Observed	Expected
density raster: coordinate system attached	pass	present	present
density raster sums to the point count	pass	70532	70532

6. filterdata: keep points by return, class, or elevation

6.1 What it does

`filterdata` writes a new point cloud holding only the points that pass a filter: chosen return numbers, chosen classes, an elevation range, or any combination. Typical uses are pulling out first returns for canopy analysis, ground-classed points for checking a vendor's ground classification, or points in an elevation band to inspect a suspected artifact.

The return and class filters use the syntax shared by every tool (Section 3.4). The elevation limits `/minz` and `/maxz` apply to raw elevation, not height above ground; to clip by height above ground, use `clipdata` with `/ground` (Chapter 9).

6.2 Example commands

The example runs three separate filters, each writing its own file. The elevation band is a 20 ft window centered on the tile's median point elevation.

```
REM filterdata first_returns
filterdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/filterdata/first_returns.las ^
/return:1

REM filterdata ground_class
filterdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/filterdata/ground_class.las ^
/class:2

REM filterdata z_band
filterdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/filterdata/z_band.las ^
/minz:253 ^
/maxz:273
```

6.3 Options used

Table 6.1: Options used in the `filterdata` examples.

Option	Value	What it does
<code>/output</code>	<code>out/filterdata/<name>.las</code>	The filtered point cloud. A <code>.las</code> extension writes compressed output.
<code>/return</code>	<code>1</code>	Keep first returns only. <code>/return:first</code> does the same.
<code>/class</code>	<code>2</code>	Keep only points classified as ground (ASPRS class 2).
<code>/minz, /maxz</code>	<code>253, 273</code>	Keep points with elevations from 253 to 273 ft.

6.4 Outputs

One LAS or LAZ file per run, with the same point format, scale, offsets, and coordinate system as the input. Withheld points and, unless `/class` says otherwise, noise classes 7 and 18 are always left out.

6.5 Results

Figure 6.1 shows a west-east side view through the tile. Grey points are the whole tile; blue points are what each filter kept.

- **First returns** mostly trace the top of the canopy, with some reaching the ground through gaps.
- **Ground class** points trace one thin line along the bottom of the point cloud, with nothing in the canopy.
- **Elevation band** points fill the space between the two dashed lines and nothing outside them.

filterdata: kept points (blue) over the whole tile (grey), side view

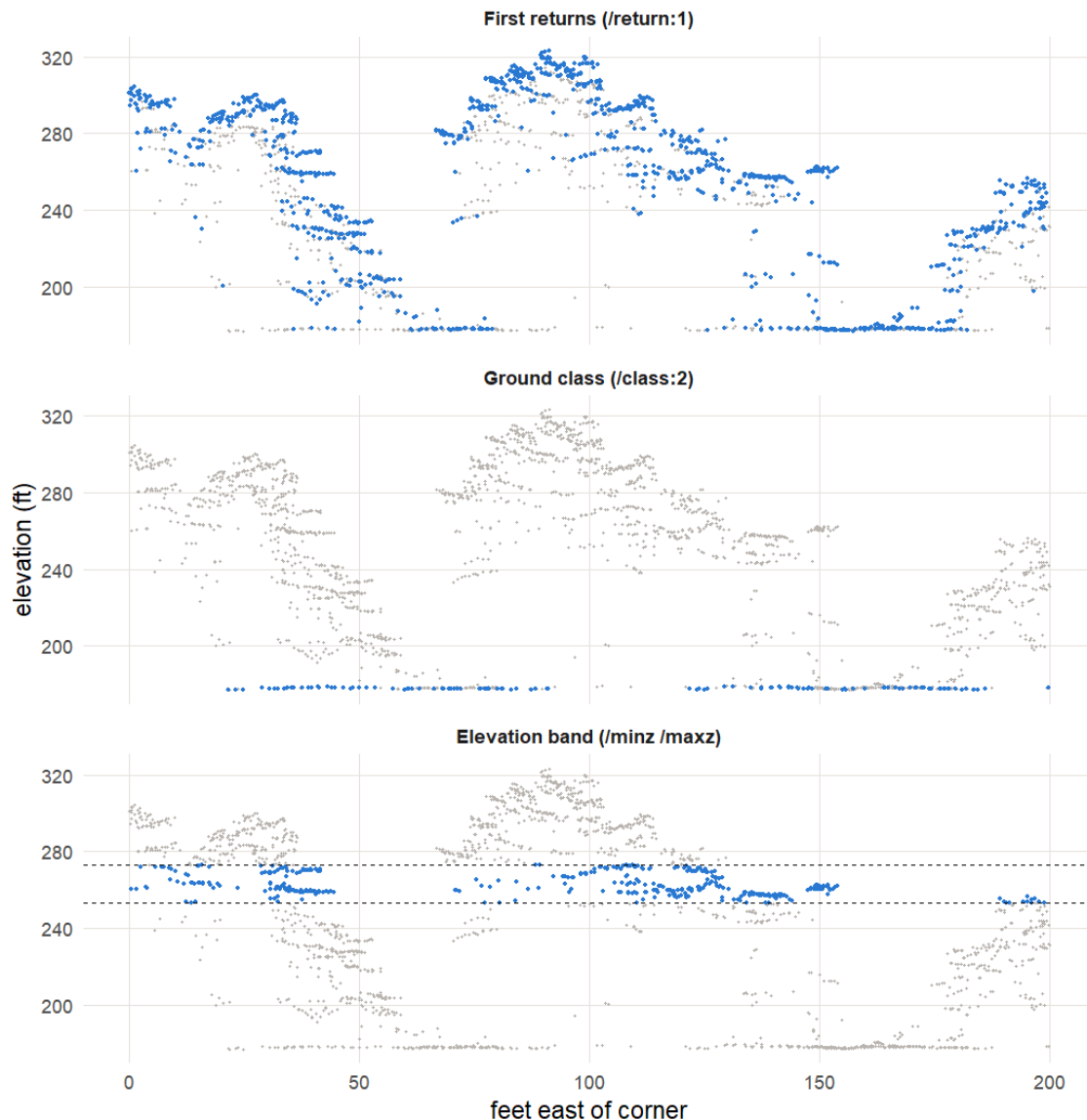


Figure 6.1: `filterdata`: points kept by each filter (blue) over all points in the side-view strip (grey).

6.6 Checks

For each filter, the checks in Table 6.2 confirm that the number of points kept matches the same filter applied in R, that every kept point satisfies the filter, that the coordinate system is kept, and that withheld points are dropped.

Table 6.2: Automated checks on the example run of `filterdata`.

Check	Result	Observed	Expected
<code>filterdata first_returns</code> : exits with status 0	pass	0	0
<code>first_returns</code> : output readable	pass	ok	ok
<code>first_returns</code> : kept count matches R	pass	39120	39120
<code>first_returns</code> : every kept point satisfies the filter	pass	0	0 violations
<code>first_returns</code> : coordinate system kept	pass	present	present
<code>first_returns</code> : withheld points dropped	pass	0	0
<code>filterdata ground_class</code> : exits with status 0	pass	0	0
<code>ground_class</code> : output readable	pass	ok	ok
<code>ground_class</code> : kept count matches R	pass	3672	3672
<code>ground_class</code> : every kept point satisfies the filter	pass	0	0 violations
<code>ground_class</code> : coordinate system kept	pass	present	present
<code>ground_class</code> : withheld points dropped	pass	0	0
<code>filterdata z_band</code> : exits with status 0	pass	0	0
<code>z_band</code> : output readable	pass	ok	ok
<code>z_band</code> : kept count matches R	pass	11893	11893
<code>z_band</code> : every kept point satisfies the filter	pass	0	0 violations
<code>z_band</code> : coordinate system kept	pass	present	present
<code>z_band</code> : withheld points dropped	pass	0	0

7. **thindata**: keep at most one point per cell

7.1 What it does

thindata reduces point density by keeping at most one point in each grid cell. It is used to make data from different acquisitions comparable (for example, thinning a dense recent flight to match an older, sparser one), to test how sensitive a model is to point density, or to shrink a point cloud for quick viewing.

The grid lines fall on whole multiples of the cell size in the data's own coordinates, so thinning two adjacent tiles gives one consistent grid across both.

7.2 Example commands

```
REM thindata
thindata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/thindata.las ^
/cellsize:2
```

7.3 Options used

Table 7.1: Options used in the **thindata** example.

Option	Value	What it does
/output	out/thindata.las	The thinned point cloud.
/cellsize	2	Keep at most one point per 2 x 2 ft cell.

The **/class** and **/return** filters (Section 3.4) are applied before thinning, so **/return:first /cellsize:2** keeps at most one first return per cell.

7.4 Outputs

One LAS or LAZ file with the same point format and coordinate system as the input, holding one point for every occupied cell.

7.5 Results

Figure 7.1 zooms in on a 20 x 20 ft patch in the middle of the tile. Grey points are the original tile, orange points are the ones kept, and the grid lines are the 2 ft thinning grid. Every square that holds grey points holds exactly one orange point.

thindata: 100 points kept of 907 in this window

Orange = kept, grey = original, lines = 2 ft thinning grid

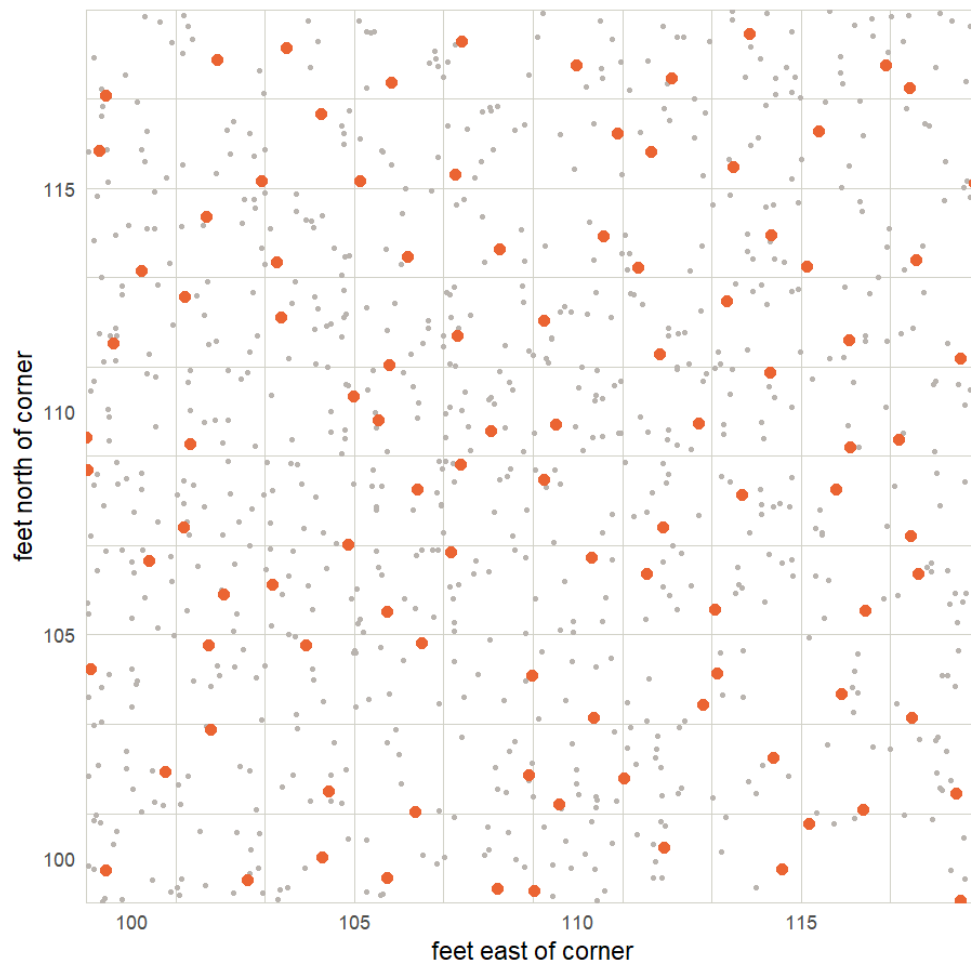


Figure 7.1: **thindata**: points kept (orange) and original points (grey) in a 20 x 20 ft window, with the 2 ft thinning grid.

7.6 Checks

Table 7.2 confirms that no cell keeps more than one point, that the number kept equals the number of occupied cells, and that every kept point is an original point rather than a new, averaged one.

Table 7.2: Automated checks on the example run of **thindata**.

Check	Result	Observed	Expected
thindata : exits with status 0	pass	0	0
output readable	pass	ok	ok
no cell keeps more than one point	pass	0	0 duplicate cells
kept count equals occupied cells	pass	10092	10092
every kept point exists in the tile	pass	0	0 new points

8. `groundfilter`: build the ground surface

8.1 What it does

`groundfilter` finds the points that hit the ground and interpolates them into a ground elevation raster (a digital elevation model, or DEM). Every tool that reports height above ground reads this surface, so its quality limits the quality of canopy heights, gridded metrics, and tree detection.

`groundfilter` ignores any classification already in the file and classifies ground itself, so it works on unclassified data. It uses the iterative robust filter of Kraus and Pfeifer, the method behind legacy FUSION's GroundFilter program:

1. **Coarse pass.** A rough ground surface is fitted on large cells (`/coarsecell`). Any point more than `/coarsecut` above it cannot be ground. This removes canopy over gaps in the ground returns that are too wide for the fine pass to see across.
2. **Fine passes.** On smaller cells (`/filtercell`), the filter fits a weighted plane through each cell and its eight neighbours, then weights each point by how far it sits above that plane. Points well above the surface lose weight, so over `/iterations` passes the surface sinks through the canopy to the ground.
3. **Classify and grid.** Points close enough to the final surface are ground. The DEM is each `/cellsize` cell's mean ground elevation, and cells with no ground point are filled from their neighbours, so the DEM has no empty cells.

Chapter 21. gives the weight function and every tuning option. The defaults follow legacy FUSION and were used unchanged here.

8.2 Example commands

```
REM groundfilter
groundfilter data/raw/lidar/TC_1372_forest200ft.laz ^
/output-raster:out/ground_dem.tif ^
/output-points:out/ground_points.las ^
/cellsize:5
```

8.3 Options used

Table 8.1: Options used in the `groundfilter` example.

Option	Value	What it does
<code>/output-raster</code>	<code>out/ground_dem.tif</code>	The ground DEM (GeoTIFF).
<code>/output-points</code>	<code>out/ground_points.las</code>	Also write the points classified as ground, as class 2.
<code>/cellsize</code>	5	DEM cell size, 5 ft.

A 5 ft DEM is a common choice for forest work: fine enough to follow terrain, coarse enough that most cells hold a ground return even under closed canopy.

8.4 Outputs

- `ground_dem.tif`: one band, `ground_elevation`, with no empty cells.

- **ground_points.las**: the points classified as ground, each written with class 2.

8.5 Results

Figure 8.1 shows the ground surface as a map with contour lines. The contours are continuous, with no single-cell pits or spikes.

groundfilter: ground elevation with 0.5 ft contours

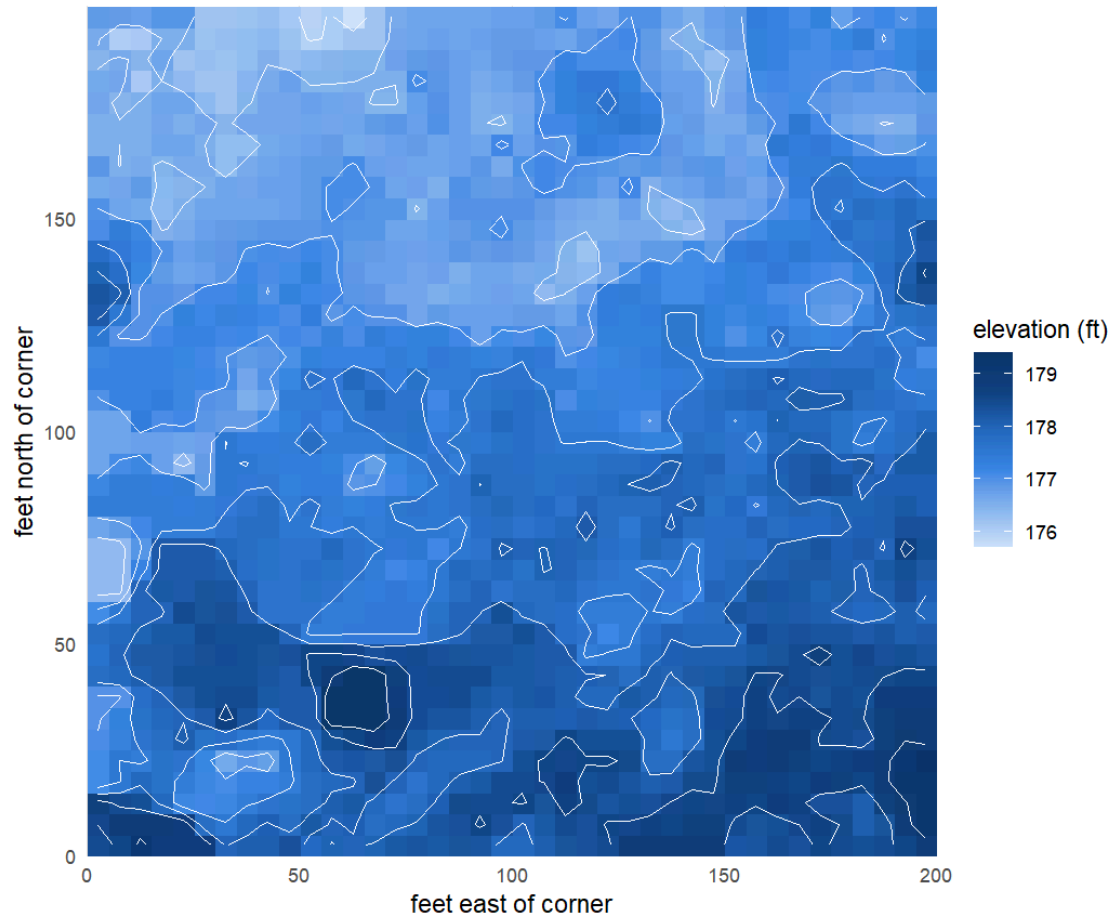


Figure 8.1: groundfilter: ground elevation with contour lines.

groundfilter: ground surface along the side-view strip

Aqua = points written to ground_points.las, grey = all points

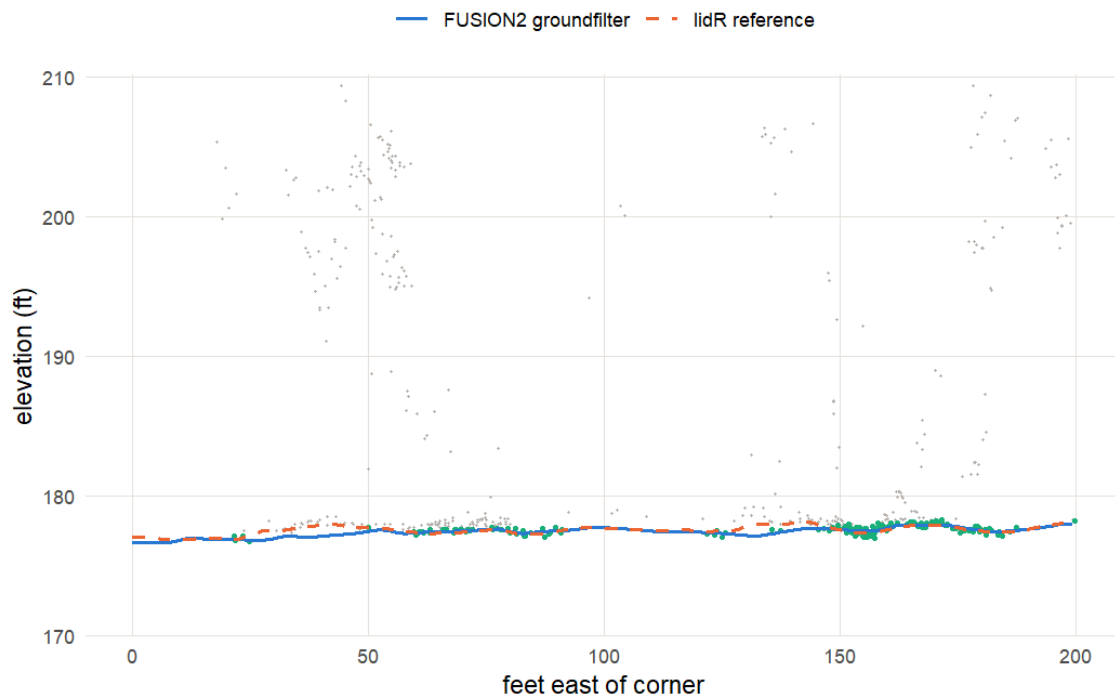


Figure 8.2: groundfilter: FUSION2 and lidR reference ground surfaces along the side-view strip, with the points written as ground.

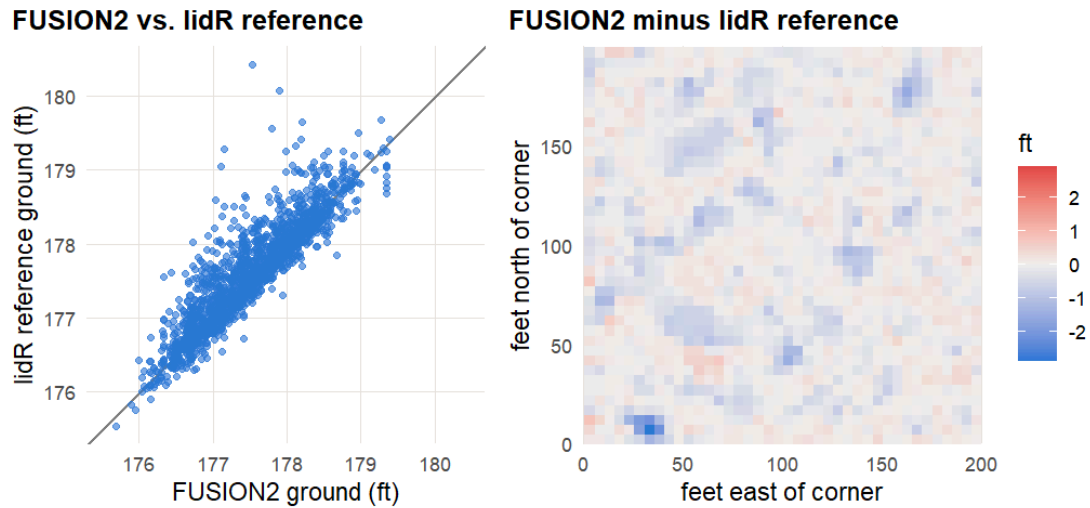


Figure 8.3: **groundfilter**: FUSION2 ground against the **lidR** reference, cell by cell (left) and as a difference map (right).

Figure 8.2 shows a west-east side view. The FUSION2 ground line (blue) runs along the bottom of the point cloud and through the points it classified as ground (aqua), and it stays close to a reference surface built independently in R with the **lidR** package (orange, dashed). Figure 8.3 compares the two surfaces cell by cell: points fall along the 1:1 line, and the difference map is mostly near zero.

8.6 Checks

Table 8.2 confirms that the DEM covers the tile with no empty cells, stays within the elevation range of the ground points, and agrees with the **lidR** reference surface: the median difference must be under 1 ft and the 95th-percentile difference under 3 ft. The ground points file must hold only class 2 points.

Table 8.2: Automated checks on the example run of **groundfilter**.

Check	Result	Observed	Expected
groundfilter : exits with status 0	pass	0	0
ground DEM written	pass	ground_dem.tif	ground_dem.tif
ground DEM: raster sits on the tile's real coordinates	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
ground DEM: coordinate system attached	pass	present	present
ground DEM has no empty cells	pass	0	0
ground DEM stays within the ground points' elevation range (+/- 1 ft)	pass	175.69, 179.39	175.45, 180.76

Check	Result	Observed	Expected
median FUSION2 - lidR ground difference under 1 ft	pass	0.134	< 1 ft
95th percentile FUSION2 - lidR ground difference under 3 ft	pass	0.681	< 3 ft
/output-points LAS written	pass	present	present
/output-points holds only ground-classed points	pass	2	2

9. clipdata: cut out boxes, plots, and height ranges

9.1 What it does

`clipdata` writes the points that fall inside a rectangle, inside (or outside) a set of polygons, or within a range of heights above ground. Its most common use is cutting field plots out of a lidar acquisition so their points can be summarized and matched to field measurements.

With a polygon shapefile and `/multifile`, one run writes a separate file for every polygon, named from one of the shapefile's fields. With `/ground`, `/minz` and `/maxz` are heights above the ground surface rather than raw elevations.

9.2 Example commands

The example runs four clips: a 100 x 100 ft box; the same box keeping only points 2 to 200 ft above ground; one file per 30 ft radius plot; and everything outside both plots.

```
REM clipdata box
clipdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/clipdata/box.las ^
/extent:1023780,598900,1023880,599000

REM clipdata box + height
clipdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/clipdata/box_height_2_200.las ^
/extent:1023780,598900,1023880,599000 ^
/ground:out/ground_dem.tif ^
/minz:2 ^
/maxz:200

REM clipdata /multifile
clipdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/clipdata/per_plot ^
/shape:out/clipdata/example_plots.shp ^
/multifile ^
/field:plot_id

REM clipdata /outside
clipdata data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/clipdata/outside_plots.las ^
/shape:out/clipdata/example_plots.shp ^
/outside
```

9.3 Options used

Table 9.1: Options used in the `clipdata` examples.

Option	Value	What it does
<code>/output</code>	<code>out/clipdata/box.las</code>	The clipped point cloud. With <code>/multifile</code> , a folder that receives one file per polygon.

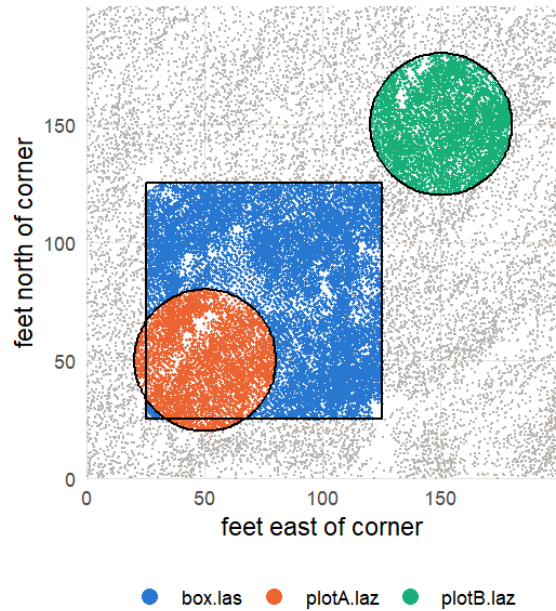
Option	Value	What it does
<code>/extent</code>	<code>1023780,598900,1023880,599000</code>	Keep points inside this rectangle, given as lower-left X, lower-left Y, upper-right X, upper-right Y.
<code>/ground</code>	<code>out/ground_dem.tif</code>	Ground surface; makes <code>/minz</code> and <code>/maxz</code> heights above ground.
<code>/minz, /maxz</code>	<code>2, 200</code>	Keep points 2 to 200 ft above ground.
<code>/shape</code>	<code>out/clipdata/example_plots.shp</code>	Keep points inside the shapefile's polygons. Combined with <code>/extent</code> , both apply.
<code>/multifile</code>		With <code>/shape</code> , write one file per polygon. A polygon with no points gets no file.
<code>/field</code>	<code>plot_id</code>	With <code>/multifile</code> , name each file from this attribute (<code>plotA.laz</code> , <code>plotB.laz</code>).
<code>/outside</code>		Keep points outside every polygon instead of inside.

9.4 Outputs

One LAS or LAZ file per run, or with `/multifile` one file per polygon (`<field value>.laz`), each with the input's point format and coordinate system.

9.5 Results

Figure 9.1 shows the clips from above. On the left, the box and plot points fill their black outlines exactly. On the right, the outside-plots clip covers the tile except for two empty circles that match the plot outlines. Figure 9.2 shows the height-limited clip from the side, in height above ground: every kept point (blue) sits above the dashed 2 ft line, and the grey points below it were in the box but too low.

Box and per-plot clips**/outside: everything but the plots**

one point in five shown

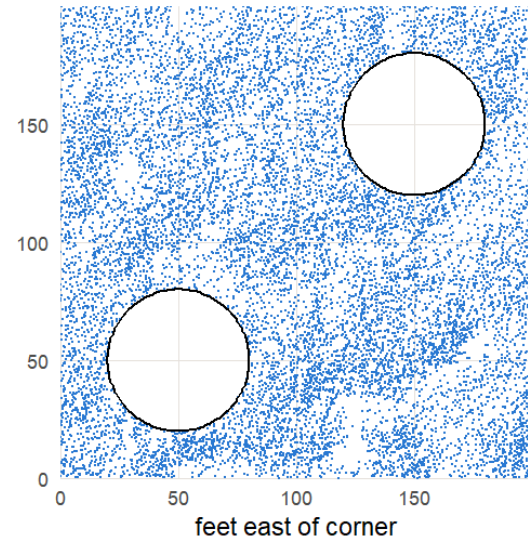


Figure 9.1: clipdata: box and per-plot clips over the tile (left), and everything outside the plots (right).

clipdata /minz:2 /maxz:200 with /ground: side view through the box

Blue = kept, grey = in the box but outside the height limits, dashed line = 2 ft

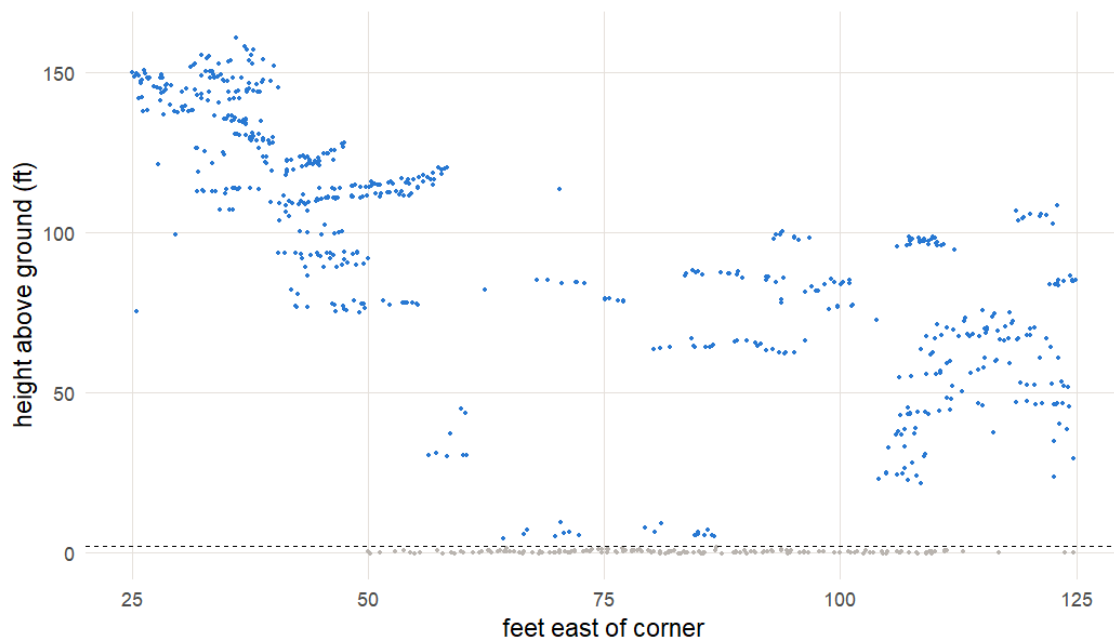


Figure 9.2: clipdata: the height-limited clip through the box, in height above ground.

9.6 Checks

Table 9.2 compares each clip with the same clip made in R. The box clip must match exactly. The per-plot and outside-plot clips must be within 0.5%, because a point lying exactly on a plot boundary can legitimately fall on either side. The height-limited clip is checked point by point against the 2-200 ft limits.

Table 9.2: Automated checks on the example run of `clipdata`.

Check	Result	Observed	Expected
clipdata box: exits with status 0	pass	0	0
box: output readable	pass	ok	ok
box: kept count matches R	pass	18079	18079
clipdata box + height: exits with status 0	pass	0	0
box + height: output readable	pass	ok	ok
box + height: kept count within 10% of R	pass	16584	16584
box + height: every kept point is 2-200 ft above ground (+/- 0.5 ft)	pass	2.00, 177.83	1.5 to 200.5 ft
clipdata /multifile: exits with status 0	pass	0	0
/multifile: plotA file named from /field	pass	1	1
/multifile: plotA count within 0.5% of R	pass	4799	4799
/multifile: plotB file named from /field	pass	1	1
/multifile: plotB count within 0.5% of R	pass	5505	5505
clipdata /outside: exits with status 0	pass	0	0
/outside: output readable	pass	ok	ok
/outside: count equals tile minus in-plot points (within 0.5%)	pass	59982	59982

10. `canopymodel`: canopy height surface

10.1 What it does

`canopymodel` builds a canopy height model (CHM): a raster in which each cell holds the height of the highest point in that cell above the ground surface. A CHM is the starting point for tree detection (`canopymaxima`, `treeseg`, `pipeline`), for canopy gap mapping, and for any analysis of canopy surface shape (`gridsurfacestats`).

A cell that no point falls in is left empty (NoData) rather than filled, so empty cells show exactly where the data has no returns.

10.2 Example commands

The example builds three CHMs from the same points and ground surface: plain, smoothed, and slope-adjusted.

```
REM canopymodel
canopymodel data/raw/lidar/TC_1372_forest200ft.laz ^
/ground:out/ground_dem.tif ^
/cellsize:3 ^
/output:out/chm.tif

REM canopymodel /smooth:3
canopymodel data/raw/lidar/TC_1372_forest200ft.laz ^
/ground:out/ground_dem.tif ^
/cellsize:3 ^
/output:out/chm_smooth3.tif ^
/smooth:3

REM canopymodel /slope
canopymodel data/raw/lidar/TC_1372_forest200ft.laz ^
/ground:out/ground_dem.tif ^
/cellsize:3 ^
/output:out/chm_slope.tif ^
/slope
```

10.3 Options used

Table 10.1: Options used in the `canopymodel` examples.

Option	Value	What it does
<code>/ground</code>	<code>out/ground_dem.tif</code>	Ground surface subtracted from each point's elevation. Without it, the raster holds raw elevations (a surface model rather than a height model).
<code>/cellsize</code>	3	CHM cell size, 3 ft.
<code>/output</code>	<code>out/chm.tif</code>	The CHM (GeoTIFF). Required.
<code>/smooth</code>	3	Smooth with a 3 x 3 cell moving average. Empty cells stay empty; they are not filled from their neighbours.

Option	Value	What it does
<code>/slope</code>		Measure each point's height perpendicular to the local slope of the ground surface rather than straight up. The two differ only where the ground is sloped.

10.4 Outputs

One GeoTIFF per run with one band, `canopy_height`, in feet above ground.

10.5 Results

Figure 10.1 shows the three CHMs on one height scale. All three show distinct crowns separated by gaps. The smoothed version is softer. On this gently sloping tile, the slope-adjusted version is nearly identical to the plain one. Single empty cells (dark grey) inside the canopy are cells no point fell in; they stay in the same place after smoothing.

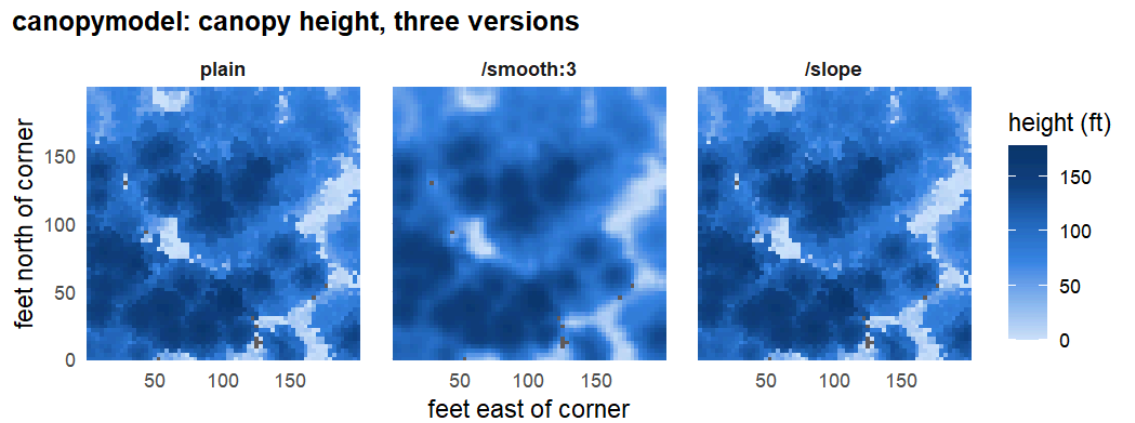


Figure 10.1: `canopymodel`: plain, smoothed, and slope-adjusted canopy height models on one height scale.

canopymodel: canopy surface along the side-view strip

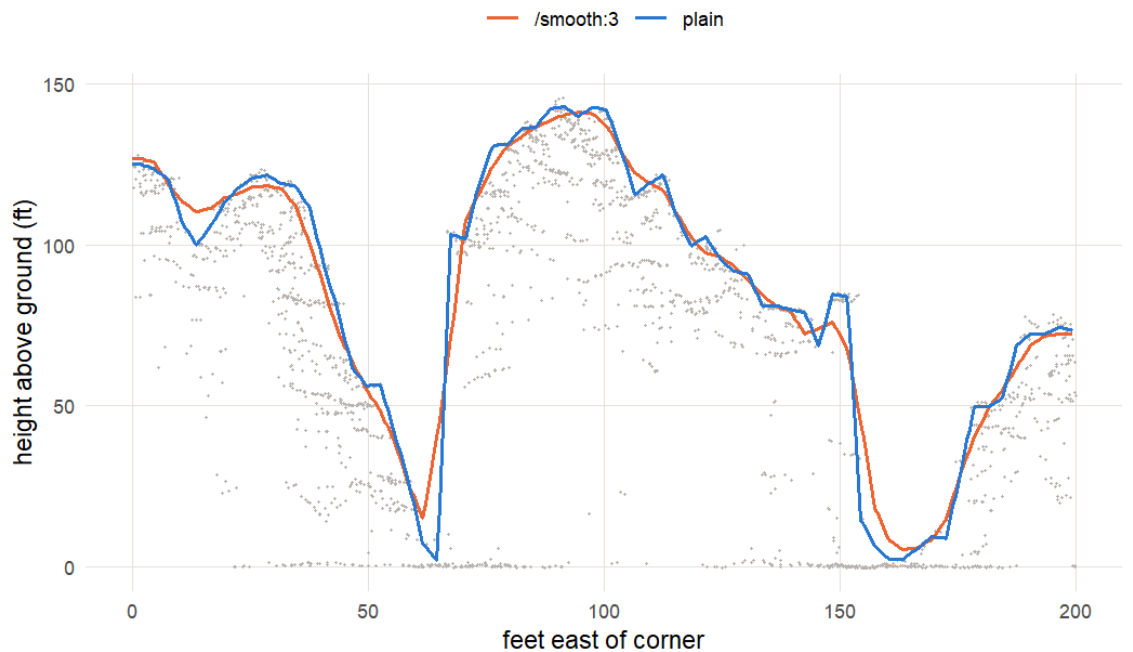


Figure 10.2: canopymodel: plain and smoothed canopy surfaces over the points in the side-view strip.

In the side view in Figure 10.2, the plain CHM line (blue) rides along the top of the point cloud, and the smoothed line (orange) follows it with softened peaks.

10.6 Checks

Table 10.2 confirms that no cell is taller than the tallest point above ground or more than 1 ft below ground, that the CHM agrees with a CHM built in R from the same points, that `/smooth:3` reduces cell-to-cell variation, and that `/slope` changes the surface.

Table 10.2: Automated checks on the example run of `canopymodel`.

Check	Result	Observed	Expected
canopymodel: exits with status 0	pass	0	0
canopymodel /smooth:3: exits with status 0	pass	0	0
canopymodel /slope: exits with status 0	pass	0	0
canopy height model written	pass	chm.tif	chm.tif
CHM: raster sits on the tile's real coordinates	pass	1023755.00, 1023956.00, 598873.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
CHM: coordinate system attached	pass	present	present
no CHM cell exceeds the tallest point above ground	pass	0 cells; max 177.83	0 cells above 181.06 ft
no CHM cell more than 1 ft below ground	pass	-0.17	>= -1 ft

Check	Result	Observed	Expected
median CHM - R reference under 0.5 ft (plausible cells)	pass	0	< 0.5 ft
/smooth:3 CHM written	pass	chm_smooth3.tif	chm_smooth3.tif
/smooth:3 lowers cell-to-cell variance	pass	1383.46	< 1599.93
/slope CHM written	pass	chm_slope.tif	chm_slope.tif
/slope changes at least one cell	pass	2.527	> 0

11. returndensity: point density and first-return share

11.1 What it does

`returndensity` maps how many returns fell in each cell per unit area, and what percent of them were first returns. Density maps are the standard check that an acquisition meets its specified point density. Low-density patches, stripes from overlapping flight lines, and water drop-outs all show up here. The first-return percent is lower under dense canopy, where each pulse produces several returns.

11.2 Example commands

```
REM returndensity
returndensity data/raw/lidar/TC_1372_forest200ft.laz ^
/output:out/returndensity.tif ^
/cellsize:10
```

11.3 Options used

Table 11.1: Options used in the `returndensity` example.

Option	Value	What it does
<code>/output</code>	<code>out/returndensity.tif</code>	The two-band density raster.
<code>/cellsize</code>	10	Cell size, 10 ft. Larger cells give smoother density estimates.

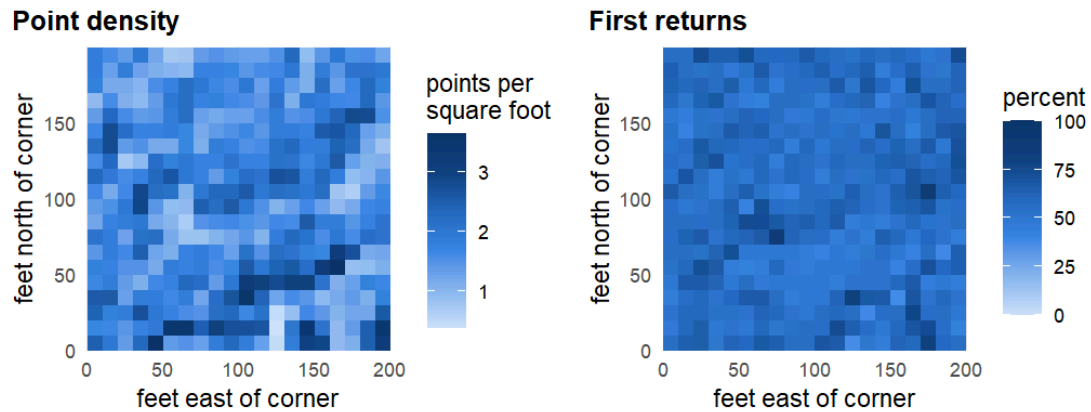
11.4 Outputs

One GeoTIFF with two bands:

- `point_density`: returns per square unit of the input's coordinate system (per square foot for the example tile).
- `first_return_ratio_pct`: first returns as a percent of all returns.

11.5 Results

Figure 11.1 shows both bands. Density is fairly even. The first-return percent stays between 0 and 100 and is lowest where the canopy is densest.

Figure 11.1: `returndensity`: point density (left) and first-return percent (right).

11.6 Checks

Table 11.2 confirms that each cell's count matches `R` exactly, that density times cell area adds back up to the tile's point count, that the first-return percent stays between 0 and 100, and that the band's unit label names the tile's own units (feet).

Table 11.2: Automated checks on the example run of `returndensity`.

Check	Result	Observed	Expected
<code>returndensity</code> : exits with status 0	pass	0	0
density raster written	pass	<code>returndensity.tif</code>	<code>returndensity.tif</code>
density raster: raster sits on the tile's real coordinates	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
density raster: coordinate system attached	pass	present	present
density x cell area sums to the point count (within 0.5%)	pass	70286	70286
per-cell counts match <code>R</code> exactly	pass	0	0 mismatched cells
first-return percent within 0-100	pass	36.96, 88.89	0 to 100
density band's unit label matches the tile's linear units (feet)	pass	<code>point_density</code>	no m2 label on a feet-based tile

12. **densitymetrics**: returns by height layer

12.1 What it does

densitymetrics counts returns in a set of height layers above ground, in every grid cell. The share of returns in each layer describes the vertical structure of the vegetation: a stand with a dense understory has many returns in the low layers, while a stand with a high, closed canopy has most returns in the top layer. These layer counts are common predictors in models of forest structure and wildlife habitat.

It computes the same per-layer density bands as **gridmetrics** with `/strata` and `/strataraster` (Chapter 13.), as a smaller, faster tool. It also keeps the name of legacy FUSION's DensityMetrics program.

12.2 Example commands

```
REM densitymetrics
densitymetrics data/raw/lidar/TC_1372_forest200ft.laz out/ground_dem.tif ^
  /outdir:out/densitymetrics ^
  /output:dm ^
  /cellsize:10 ^
  /output-table:out/densitymetrics/dm_densitymetrics.csv
```

12.3 Options used

Table 12.1: Options used in the **densitymetrics** example.

Option	Value	What it does
(second input)	out/ground_dem.tif	Ground surface, given as the second positional argument. <code>/ground:</code> works too.
<code>/outdir</code>	out/densitymetrics	Folder for the outputs.
<code>/output</code>	dm	Base name of the outputs (<code>dm_densitymetrics.tif</code>). Without it, the name comes from the input file.
<code>/cellsize</code>	10	Cell size, 10 ft.
<code>/output-table</code>	out/densitymetrics/dm_densitymetrics.csv	Also write the per-cell counts as a CSV table.

The layers come from `/strata`, which was left at its default of `0.5,2,5,10,20`. Those five break heights give six layers: below 0.5 ft, 0.5-2, 2-5, 5-10, 10-20, and above 20 ft.

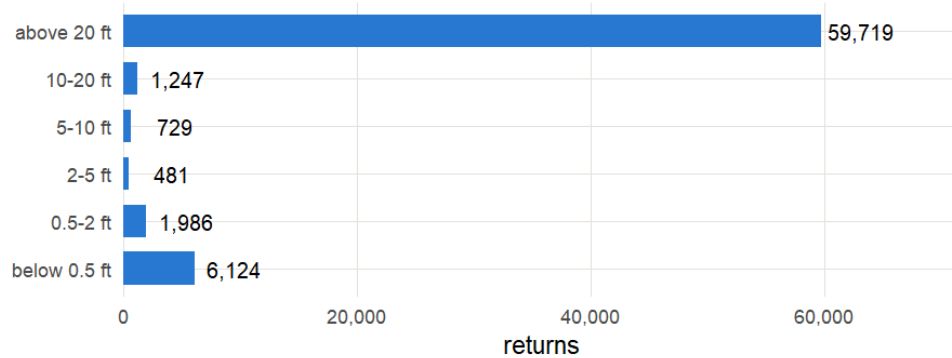
12.4 Outputs

- **dm_densitymetrics.tif**: one band per layer, `density_stratum_00` (lowest) to `density_stratum_05` (highest), each holding that layer's returns per square unit.
- **dm_densitymetrics.csv**: one row per cell with its column, row, center coordinates, total returns (`TotalReturns`), and the count in each layer (`StrataCnt_0` to `StrataCnt_5`).

12.5 Results

Figure 12.1 shows the returns per layer for the whole tile (top) and each cell's share of returns by layer (bottom). On this closed-canopy tile, most returns fall in the top layer, above 20 ft. Cells with a lower share there line up with the canopy gaps visible in Figure 10.1.

Returns per height layer, whole tile



Share of each cell's returns, by layer

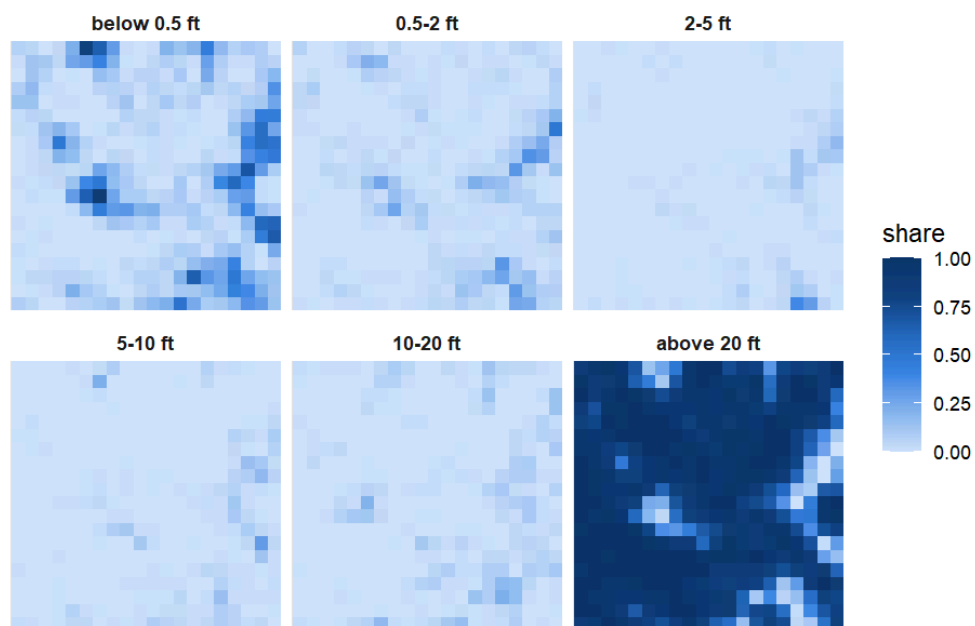


Figure 12.1: *densitymetrics*: returns per height layer for the whole tile (top) and each cell's share of returns by layer (bottom).

12.6 Checks

Table 12.2 confirms that each cell's layer counts add up to its total returns, that the totals across cells add up to the tile's point count, and that the top layer's count matches R.

Table 12.2: Automated checks on the example run of *densitymetrics*.

Check	Result	Observed	Expected
<i>densitymetrics</i> : exits with status 0	pass	0	0
stratum raster written	pass	dm_densitymetrics.tif	dm_densitymetrics.tif

Check	Result	Observed	Expected
stratum raster: raster sits on the tile's real coordinates	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
stratum raster: coordinate system attached	pass	present	present
per-cell CSV table written	pass	present	present
per cell, stratum counts add to total returns	pass	0	0 mismatched cells
total returns across cells equals the point count	pass	70286	70286
top stratum (> 20 ft) count matches R (within 5 points)	pass	59719	59719

13. `gridmetrics`: the full metric grid

13.1 What it does

`gridmetrics` is the workhorse of area-based forest inventory. For every grid cell, it computes a large set of statistics from the heights and intensities of the returns in that cell: height percentiles, mean and variability, canopy cover, return counts, per-layer summaries, and surface roughness. Models fit between these cell metrics and field plot measurements (volume, biomass, basal area) are then applied to every cell to map those attributes across a whole project area.

`gridmetrics` runs on one file, as here, or on a whole folder of tiles in a batch mode that tiles, buffers, and processes them in parallel (Chapter 20.). Chapter 23. defines every metric.

13.2 Example commands

```
REM gridmetrics
gridmetrics data/raw/lidar/TC_1372_forest200ft.laz out/ground_dem.tif ^
  /cellsize:10 ^
  /outdir:out/gridmetrics ^
  /strata:0.5,2,5,10,20 ^
  /strataraster ^
  /surfstats ^
  /output-table:out/gridmetrics/gridmetrics_table.csv
```

13.3 Options used

Table 13.1: Options used in the `gridmetrics` example.

Option	Value	What it does
(second input)	out/ground_dem.tif	Ground surface, given as the second positional argument. <code>/ground:</code> works too.
<code>/cellsize</code>	10	Cell size, 10 ft. Match it to the size of your field plots when the grid will be used for plot-based models.
<code>/outdir</code>	out/gridmetrics	Folder for the raster, named <code><input name>_gridmetrics.tif</code> .
<code>/strata</code>	0.5,2,5,10,20	Also summarize the returns in each of six height layers (Chapter 12.).
<code>/strataraster</code>		Write the per-layer summaries as raster bands, not only as table columns.
<code>/surfstats</code>		Add <code>surface_area_ratio</code> and <code>roughness</code> bands computed from the highest return in each cell (Chapter 16.).
<code>/output-table</code>	out/gridmetrics/gridmetrics_table.csv	Also write every metric as a CSV table, one row per cell.

Options left at their defaults include `/minht:2` (returns below 2 ft are left out of the height statistics), `/nodata:NA`, and `/noheight:0` (Chapter 3.).

13.4 Outputs

- **TC_1372_forest200ft_gridmetrics.tif**: one multiband GeoTIFF with 176 named bands on this run. They include height statistics (`elev_min` to `elev_p99`, L-moments, and others), canopy cover and its variants (`canopy_cover`, `allcover`, `afcover`, and others), `point_density`, counts by return number (`r1count` to `r9count`), intensity statistics (`int_*`), `surface_area_ratio`, `roughness`, and seven bands per height layer (density, count, proportion, mean, standard deviation, minimum, maximum), plus six intensity bands per layer.
- **gridmetrics_table.csv**: one row per cell. It starts with the cell's column, row, and center coordinates (`Col`, `Row`, `X`, `Y`), its `TotalReturns` and `FirstReturns`, and `CanopyCover` and `PointDensity`, followed by the raster's metrics with the same names. The surface-roughness bands and the per-layer density bands are in the raster only.

`/output-mode:singleband` writes each metric as its own GeoTIFF instead.

13.5 Results

Figure 13.1 shows six representative bands, each on its own scale. Maximum and 95th-percentile height follow the same crown pattern as the canopy height model (Figure 10.1) at a coarser cell size. Canopy cover is high across this closed-canopy tile. Height variability is highest where tall crowns sit next to gaps.

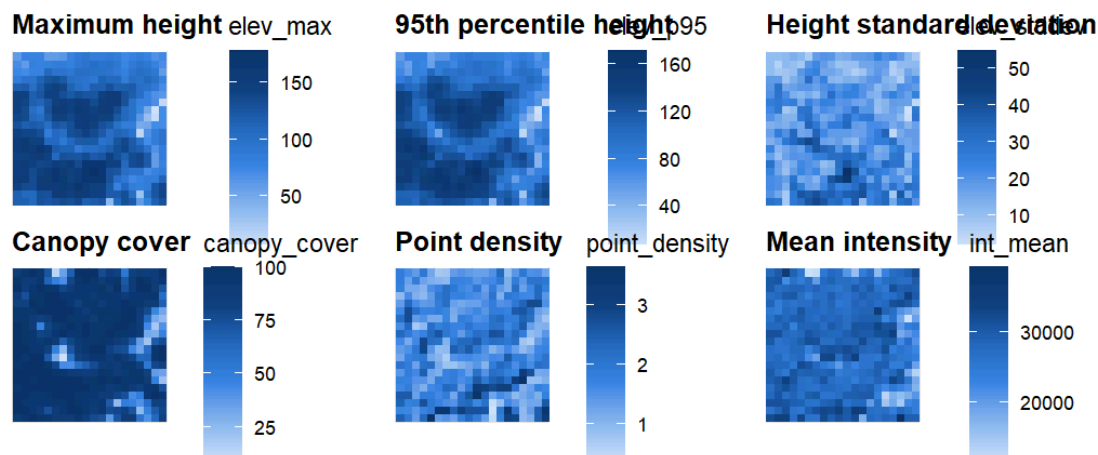


Figure 13.1: `gridmetrics`: six representative metric bands, each on its own scale.

13.6 Checks

Table 13.2 confirms that the grid has the expected rows and columns and sits on the tile, that no band is empty everywhere, that the per-cell return counts match R, that no maximum height exceeds the tallest point, and that the table has one row per cell and agrees with the raster.

Table 13.2: Automated checks on the example run of `gridmetrics`.

Check	Result	Observed	Expected
gridmetrics: exits with status 0	pass	0	0
/output-table path honored	pass	written	written
metrics raster written	pass	TC_1372_forest200ft_ gridmetrics.tif	TC_1372_forest200ft_ gridmetrics.tif
metrics raster: raster sits on the tile's real coordinates	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
metrics raster: coordinate system attached	pass	present	present
no band is empty in every cell	pass	none	none
grid has the expected rows x columns for the tile	pass	20x20	20x20
per-cell return counts match R	pass	0	0 mismatched cells
no elev_max cell exceeds the tallest point above ground	pass	0 cells; max 177.83	0 cells above 181.06 ft
CSV table has one row per raster cell	pass	400	400
CSV elev_mean matches the raster band	pass	0	all cells equal

14. cloudmetrics: one summary for a file or for each plot

14.1 What it does

`cloudmetrics` computes the same statistics as `gridmetrics`, but for a whole point cloud (one row per file) or for each polygon in a shapefile (one row per polygon) instead of for each grid cell. Its main use is computing lidar metrics for field plots, the other half of the plot-to-grid modelling workflow described in Chapter 13..

With `/shape`, `cloudmetrics` reads the full point cloud once and routes each point to the polygon that contains it. There is no need to clip each plot to its own file first, as legacy FUSION's workflow required.

14.2 Example commands

The example summarizes the whole tile, then each of the two plots.

```
REM cloudmetrics tile
cloudmetrics data/raw/lidar/TC_1372_forest200ft.laz ^
  /ground:out/ground_dem.tif ^
  /output:out/cloudmetrics_tile.csv ^
  /minht:2 ^
  /strata:0.5,2,5,10,20

REM cloudmetrics /shape
cloudmetrics data/raw/lidar/TC_1372_forest200ft.laz ^
  /ground:out/ground_dem.tif ^
  /output:out/cloudmetrics_plots.csv ^
  /shape:out/clipdata/example_plots.shp ^
  /field:plot_id
```

14.3 Options used

Table 14.1: Options used in the `cloudmetrics` examples.

Option	Value	What it does
<code>/ground</code>	<code>out/ground_dem.tif</code>	Ground surface for heights above ground.
<code>/output</code>	<code>out/cloudmetrics_tile.csv</code>	The metrics table.
<code>/minht</code>	<code>2</code>	Leave returns below 2 ft out of the height statistics (the default, stated here explicitly).
<code>/strata</code>	<code>0.5,2,5,10,20</code>	Also summarize the returns in each of six height layers.
<code>/shape</code>	<code>out/clipdata/example_plots.shp</code>	Compute one row per polygon rather than one for the whole file.
<code>/field</code>	<code>plot_id</code>	Label each polygon's row from this attribute.

14.4 Outputs

One CSV table per run. Each row starts with an identifier (`Filename`, or with `/shape` the polygon's `FeatureIndex` and `Label`), then `TotalPoints`, `CanopyPoints`, `CanopyCoverPct`, and the full set of height and intensity statistics (`elev_*`, `int_*`). The per-plot table on this run has 84 columns.

14.5 Results

Figure 14.1 plots the height percentiles for the tile and for each plot. Each curve rises steadily from low to high percentiles. The plot curves resemble the tile curve without matching it exactly, since each plot samples a different patch of forest.

cloudmetrics: height percentiles

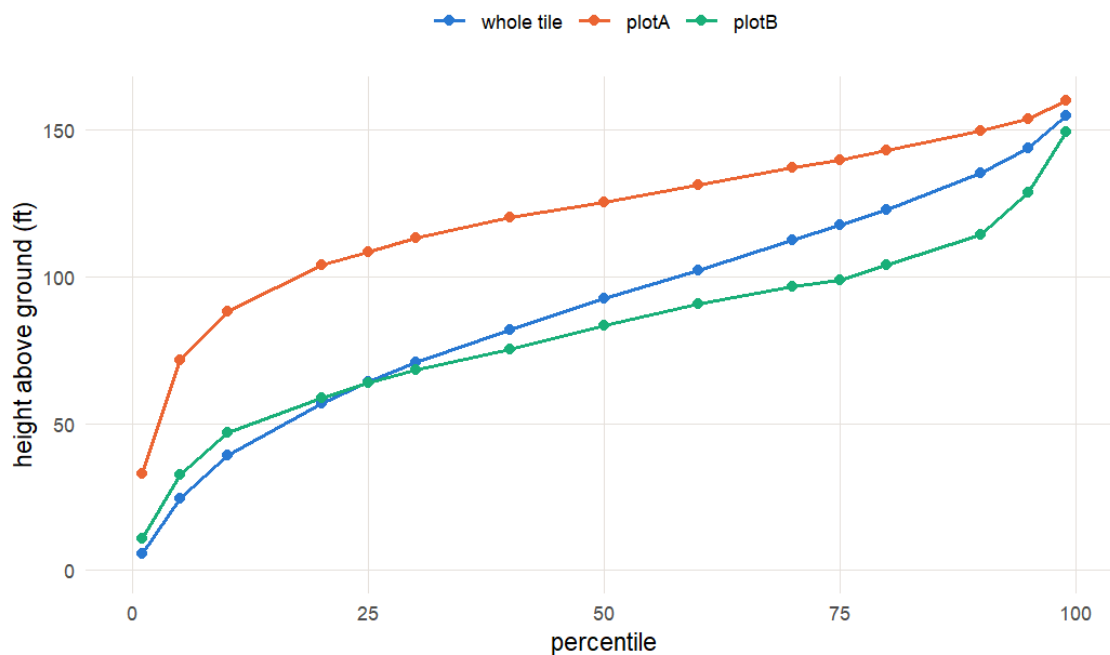


Figure 14.1: `cloudmetrics`: height percentiles for the whole tile and each plot.

14.6 Checks

Table 14.2 confirms that the tile's point count and canopy point count match R, that its maximum height is within the tallest point, and that the per-plot table has one row per plot, labelled from `plot_id`, with point counts within 0.5% of R.

Table 14.2: Automated checks on the example run of `cloudmetrics`.

Check	Result	Observed	Expected
cloudmetrics tile: exits with status 0	pass	0	0
tile metrics CSV written	pass	cloudmetrics_tile.csv	cloudmetrics_tile.csv
tile: total points matches the tile	pass	70286	70286
tile: elev_max within the tallest point above ground	pass	177.83	<= 181.06 ft
tile: canopy points (> 2 ft) match R (within 1%)	pass	62176	62176

Check	Result	Observed	Expected
cloudmetrics /shape: exits with status 0	pass	0	0
per-plot metrics CSV written	pass	cloudmetrics_plots.csv	cloudmetrics_plots.csv
/shape: one row per plot, labelled from /field	pass	plotA,plotB	plotA,plotB
/shape: plotA point count within 0.5% of R	pass	4799	4799
/shape: plotB point count within 0.5% of R	pass	5505	5505

15. topometrics: slope and aspect

15.1 What it does

`topometrics` computes terrain slope and aspect from a ground elevation raster. Slope and aspect affect site productivity, solar exposure, and snow and moisture, so they are standard covariates alongside lidar metrics in forest models. `topometrics` works on any ground raster, not only one made by `groundfilter`.

Slope is in degrees from horizontal. Aspect is the compass direction the slope faces, in degrees clockwise from north (0 = north, 90 = east). Both are computed from each cell's eight neighbours with Horn's method, the same method GIS software commonly uses. Cells on the outer edge have no full set of neighbours and are left empty.

15.2 Example commands

The example runs `topometrics` twice: on the ground surface from Chapter 8., and on a USGS 3DEP bare-earth DTM of the same ground that FUSION2 did not produce.

```
REM topometrics ground DEM
topometrics out/ground_dem.tif ^
/ ^
/output:out/topometrics_ground.tif

REM topometrics external DTM
topometrics data/raw/lidar_dtm_examples/site_tc_1372_forest/usgs_3dep_dtm.tif ^
/ ^
/output:out/topometrics_external_dtm.tif
```

15.3 Options used

Table 15.1: Options used in the `topometrics` examples.

Option	Value	What it does
(input)	out/ground_dem.tif or the USGS DTM	The ground raster. Its cell size and coordinate system carry through to the output.
/output	out/topometrics_ground.tif	The slope and aspect raster.

15.4 Outputs

One GeoTIFF on the input raster's grid, with two bands: `slope_degrees` and `aspect_degrees`.

15.5 Results

Figure 15.1 shows slope as color, with arrows pointing in the direction each slope faces (downhill) and contour lines of elevation. Both maps share one slope scale.

- Arrows cross the contour lines at right angles and point from high ground to low ground, and slopes are steepest where contours bunch together.
- Inside the dashed square (the tile), the two maps show the same broad pattern. The USGS DTM is an independent model with 3 ft cells, so the goal is broad agreement, not an exact match.

- On nearly flat ground, small bumps in a ground surface can show up as rings of higher slope one or two cells across. These follow the ground surface's own cells rather than real terrain.

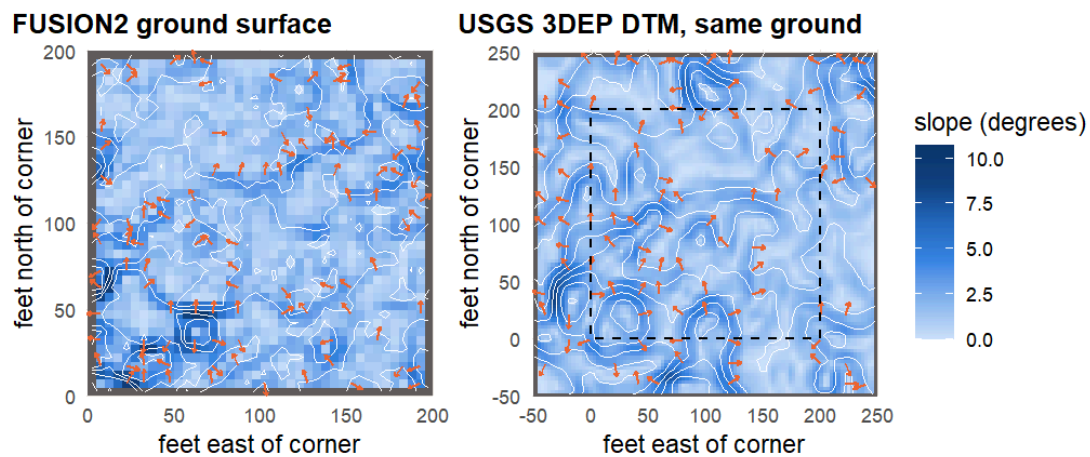


Figure 15.1: topometrics: slope with downhill arrows and contours, from the FUSION2 ground surface (left) and the USGS 3DEP DTM of the same ground (right). The dashed square marks the tile.

15.6 Checks

For both inputs, Table 15.2 confirms that the output sits on the input's grid and keeps its coordinate system, that interior cells have values, that slope matches the `terra` R package's slope within a median of 0.5 degrees, and that aspect matches `terra` and is reported as a compass bearing.

Table 15.2: Automated checks on the example run of `topometrics`.

Check	Result	Observed	Expected
topometrics ground DEM: exits with status 0	pass	0	0
topometrics ground DEM raster written	pass	topometrics_ground.tif	topometrics_ground.tif
topometrics ground DEM: output on the input DEM's grid	pass	1023755.00, 1023955.00, 598874.99, 59907...	1023755.00, 1023955.00, 598874.99, 59907...
topometrics ground DEM: keeps the input's coordinate system	pass	present	present

Check	Result	Observed	Expected
topometrics ground DEM: interior cells have values	pass	0	0
topometrics ground DEM: median slope - terra under 0.5 degrees	pass	0	< 0.5
topometrics ground DEM: aspect matches terra (compass or math conve...	pass	compass 0; math 98.14	< 5 under one convention
topometrics ground DEM: aspect reported as a compass bearing (0 = n...	pass	compass bearing	compass bearing
topometrics external DTM: exits with status 0	pass	0	0
topometrics external DTM raster written	pass	topometrics_external_dtm.tif	topometrics_external_dtm.tif
topometrics external DTM: output on the input DEM's grid	pass	1023705, 1024005, 598825, 599125	1023705, 1024005, 598825, 599125
topometrics external DTM: keeps the input's coordinate system	pass	present	present
topometrics external DTM: interior cells have values	pass	0	0
topometrics external DTM: median slope - terra under 0.5 degrees	pass	0	< 0.5
topometrics external DTM: aspect matches terra (compass or math con...	pass	compass 0; math 96.12	< 5 under one convention
topometrics external DTM: aspect reported as a compass bearing (0 =...	pass	compass bearing	compass bearing

16. gridsurfacestats: surface roughness and volume

16.1 What it does

`gridsurfacestats` measures how rough a surface is. It works on any raster surface: a canopy height model, where roughness reflects the canopy's vertical complexity, or a ground surface, where it reflects terrain ruggedness. Given a second, reference surface, it also measures the volume between the two in each cell, which serves for cut-and-fill estimates or for comparing canopy surfaces from two dates.

For each cell, it computes:

- **Surface area ratio:** the 3D area of the surface over the cell, divided by the cell's flat area. A flat surface gives 1; rougher surfaces give more.
- **Roughness:** the standard deviation of the surface's values in the cell's 3 x 3 neighbourhood.
- **Volume difference** (with `/reference`): the surface minus the reference, multiplied by the cell's area, so each cell holds a true volume and the cells can simply be added up.

`gridmetrics /surfstats` and `cloudmetrics /surfstats` compute the first two from points directly, without a separate raster.

16.2 Example commands

The example runs on the canopy height model from Chapter 10., with the smoothed canopy model as the reference surface.

```
REM gridsurfacestats
gridsurfacestats out/chm.tif ^
/output:out/gridsurfacestats.tif ^
/reference:out/chm_smooth3.tif
```

16.3 Options used

Table 16.1: Options used in the `gridsurfacestats` example.

Option	Value	What it does
(input)	out/chm.tif	The surface to measure.
/output	out/gridsurfacestats.tif	The output raster.
/reference	out/chm_smooth3.tif	A second surface on the same grid; adds the volume-difference band.

16.4 Outputs

One GeoTIFF on the input's grid with the bands `surface_area_ratio`, `roughness`, and (with `/reference`) `volume_diff`, in cubic units of the input (cubic feet here). Cells that are empty in the input stay empty.

16.5 Results

In Figure 16.1, the surface area ratio is 1 or more everywhere and highest at crown edges, where the canopy drops steeply into gaps. Roughness is highest in the same places. The volume difference is near

zero on even canopy and has opposite signs on crown peaks, which smoothing lowers, and in narrow gaps, which smoothing fills.

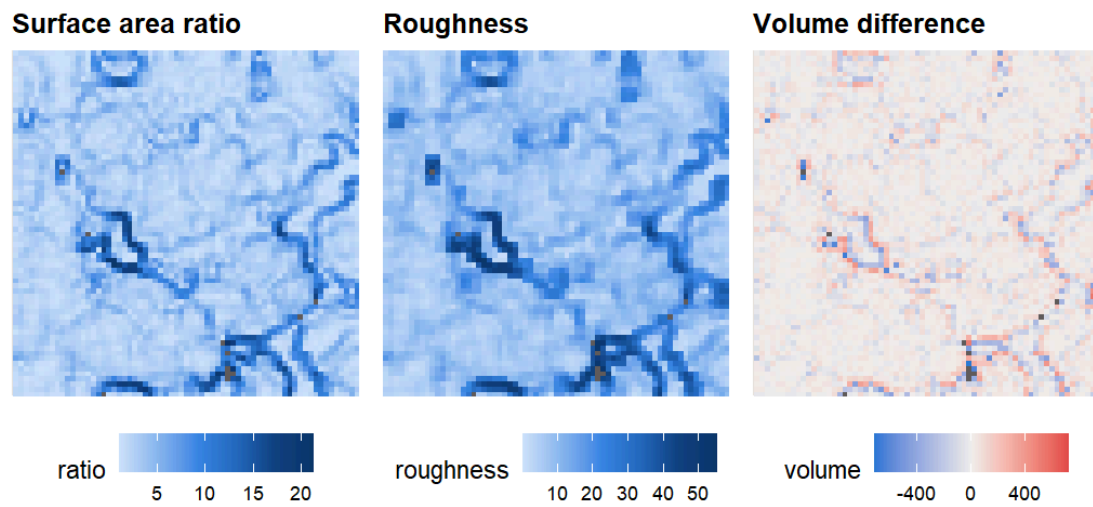


Figure 16.1: `gridsurfacestats`: surface area ratio, roughness, and volume difference between the canopy and its smoothed version.

16.6 Checks

Table 16.2 confirms that the surface area ratio is at least 1 and roughness at least 0 everywhere, and that the volume difference follows the surface-minus-reference difference and includes the cell area.

Table 16.2: Automated checks on the example run of `gridsurfacestats`.

Check	Result	Observed	Expected
<code>gridsurfacestats</code> : exits with status 0	pass	0	0
surface stats raster written	pass	<code>gridsurfacestats.tif</code>	<code>gridsurfacestats.tif</code>
surface stats raster: raster sits on the tile's real coordinates	pass	1023755.00, 1023956.00, 598873.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
surface stats raster: coordinate system attached	pass	present	present
<code>surface_area_ratio</code> ≥ 1 everywhere	pass	1.0001	≥ 1
<code>roughness</code> ≥ 0 everywhere	pass	0.1754	≥ 0
<code>volume_diff</code> tracks the surface - reference difference	pass	height difference x cell area	one of the two
<code>volume_diff</code> includes cell area (a true volume)	pass	volume	height difference x cell area

17. canopymaxima: find tree tops

17.1 What it does

`canopymaxima` finds individual tree tops as local height peaks on a canopy height model, and reports each one's location and height. Tree tops are the basis of individual-tree inventories: counting stems, building tree lists, and matching lidar trees to field-measured trees.

A cell counts as a tree top when it is the highest cell within a search window around it. The window grows with height, since taller trees have wider crowns: at height H , its radius is $\text{/window-a} + \text{/window-b} \times H$, in the canopy model's units, rounded to whole cells and never less than one cell. A small window finds more tops (and splits large crowns); a large window finds fewer (and merges small neighbouring trees).

17.2 Example commands

```
REM canopymaxima
canopymaxima out/chm.tif ^
/output:out/tree_tops.csv ^
/minht:2
```

17.3 Options used

Table 17.1: Options used in the `canopymaxima` example.

Option	Value	What it does
(input)	out/chm.tif	The canopy height model from Chapter 10..
/output	out/tree_tops.csv	The tree-top table.
/minht	2	Ignore peaks lower than 2 ft.

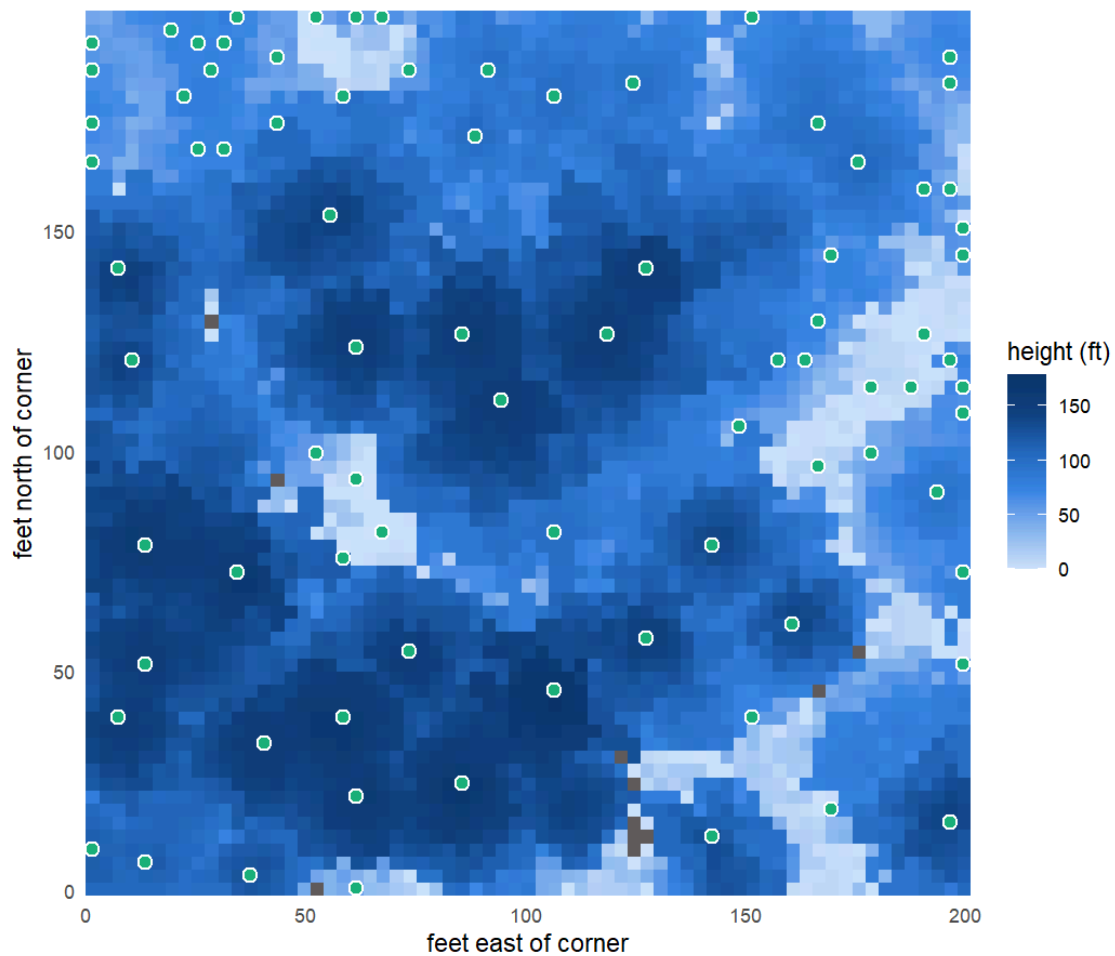
The window coefficients were left at their defaults (`/window-a:0.5`, `/window-b:0.05`). Tune them to the forest type and the canopy model's cell size; a smoothed canopy model (`canopymodel /smooth`) also reduces false tops on bumpy crowns.

17.4 Outputs

A CSV table with one row per tree top: `TreeID`, `X`, `Y` (in the input's coordinate system), `Height`, and `CrownDiameter` (twice the search window radius used at that height, a rough guide to crown size rather than a measured crown).

17.5 Results

In Figure 17.1, each tree top (aqua) sits on a bright local peak of the canopy model, with roughly one top per visible crown and no tops in gaps below 2 ft. Several tops clustered on one crown would suggest the window is too small; a large crown with no top would suggest it is too large. Tops along the tile's outer edge are likely edge effects, since an edge cell has no neighbours on one side.

canopymaxima: 82 tree tops over the canopy height modelFigure 17.1: `canopymaxima`: tree tops over the canopy height model.

17.6 Checks

Table 17.2 confirms that at least one top was found, that every top lies inside the tile and is at least 2 ft tall but no taller than the tallest point, that coordinates are written in full rather than rounded, and that each top’s height equals the canopy model’s value at its location.

Table 17.2: Automated checks on the example run of `canopymaxima`.

Check	Result	Observed	Expected
<code>canopymaxima</code> : exits with status 0	pass	0	0
tree tops CSV written	pass	tree_tops.csv	tree_tops.csv
at least one tree top found	pass	82	> 0
every tree top inside the tile	pass	0	0 outside
every tree top at least /minht (2 ft)	pass	2.31	>= 2

Check	Result	Observed	Expected
no tree top exceeds the tallest point above ground	pass	0 tops; max 177.83	0 above 181.06 ft
X/Y written without scientific-notation rounding	pass	0 rows, e.g. X = 1023789.50	0 rows
each top's height equals the CHM at its X/Y	pass	0	0 mismatched tops

18. treeseg: outline tree crowns

18.1 What it does

`treeseg` divides a canopy height model into individual tree crowns. It starts from each height peak and grows a region outward and downward (a watershed segmentation) until neighbouring regions meet along the low ground between crowns. Each crown gets an ID, an area, and a maximum height, which support crown-size distributions, tree lists with crown dimensions, and crown-level metrics.

`treeseg` finds its own peaks, so its crown count can differ from `canopymaxima`'s tree-top count on the same canopy model.

18.2 Example commands

```
REM treeseg
treeseg out/chm.tif ^
/output-raster:out/crown_segments.tif ^
/output-table:out/crown_summary.csv ^
/minht:2
```

18.3 Options used

Table 18.1: Options used in the `treeseg` example.

Option	Value	What it does
(input)	out/chm.tif	The canopy height model from Chapter 10..
/output-raster	out/crown_segments.tif	Raster of crown IDs.
/output-table	out/crown_summary.csv	One row per crown.
/minht	2	Cells lower than 2 ft belong to no crown.

18.4 Outputs

- **crown_segments.tif**: one band, `tree_segment_id`, holding each cell's crown ID (empty for cells in no crown).
- **crown_summary.csv**: one row per crown with `TreeID`, `MaxHeight`, `CrownArea` (in square units of the input, square feet here), and `PixelCount`.

18.5 Results

Figure 18.1 draws the crown outlines (orange) over the canopy height model, with the `canopymaxima` tree tops from Chapter 17. (aqua) for comparison. Outlines follow the visible crown edges and meet along the low ground between crowns. Crowns with no aqua top inside show where the two tools disagree. Figure 18.2 plots crown area against crown height. Very small crowns at low heights are the most likely to be fragments rather than trees.

treeseg: 165 crowns over the canopy height model

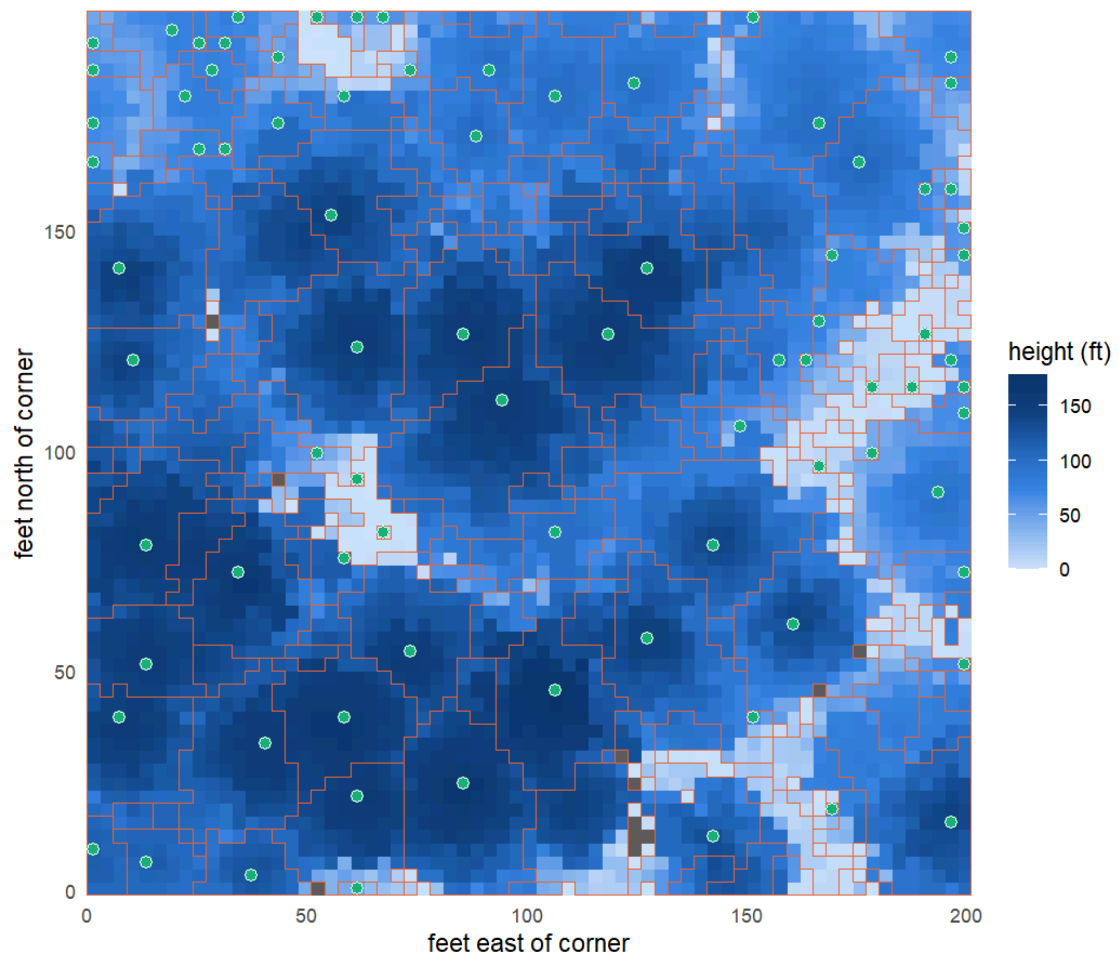


Figure 18.1: `treeseg`: crown outlines (orange) and `canopymaxima` tree tops (aqua) over the canopy height model.

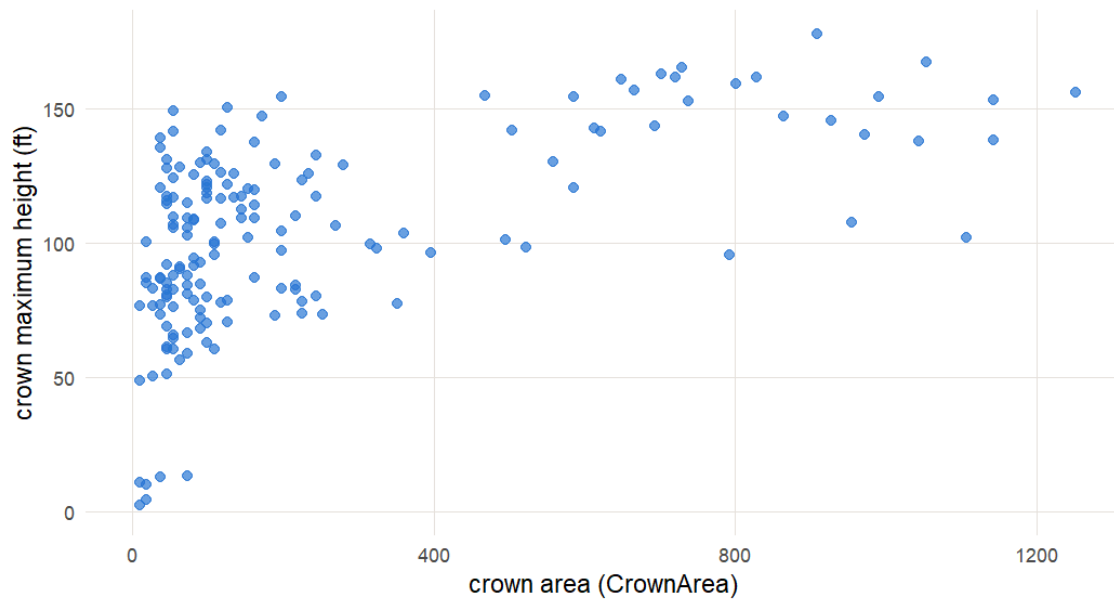
treeseq: crown size and height

Figure 18.2: treeseq: crown area against crown maximum height.

18.6 Checks

Table 18.2 confirms that the table and the raster list the same crowns, that each crown's pixel count and maximum height match the raster and the canopy model, that crown area equals pixel count times cell area, and that the area's unit label names the tile's own units.

Table 18.2: Automated checks on the example run of `treeseq`.

Check	Result	Observed	Expected
treeseq: exits with status 0	pass	0	0
segment raster written	pass	crown_segments.tif	crown_segments.tif
crown table written	pass	crown_summary.csv	crown_summary.csv
segment raster: raster sits on the tile's real coordinates	pass	1023755.00, 1023956.00, 598873.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
segment raster: coordinate system attached	pass	present	present
same crowns in the table and the raster	pass	165	165
PixelCount matches the segment raster	pass	0	0 mismatched crowns
MaxHeight matches the CHM within each crown	pass	0	0 mismatched crowns
no crown exceeds the tallest point above ground	pass	0 crowns; max 177.83	0 above 181.06 ft
crown area = pixel count x cell area	pass	0	0 mismatched crowns

Check	Result	Observed	Expected
crown area column's unit label matches the tile's units (feet)	pass	CrownArea	no m2 label on a feet-based tile

19. pipeline: run tools tile by tile

19.1 What it does

`pipeline` runs a chain of FUSION2 tools across an area too large to process as one piece. It splits the area into tiles, adds a buffer of extra points around each tile so that edge cells and edge trees see their full neighbourhood, runs the chain of tools on each tile in parallel, trims each result back to its tile, and stitches the tiles together.

A `groundfilter` stage's ground surface is passed automatically to every later stage that needs one, so a chain such as `groundfilter,canopymodel,canopymaxima` goes from raw points to tree tops in one command. `pipeline` records each tile's progress, so an interrupted run resumes where it stopped. Chapter 20. covers large-area runs in more depth.

19.2 Example commands

The example splits the example tile into four 100 x 100 ft tiles and runs ground filtering, canopy height modelling, and tree-top detection on each.

```
REM pipeline
pipeline ^
  /pipeline:groundfilter,canopymodel,canopymaxima ^
  /input:data/raw/lidar/TC_1372_forest200ft.laz ^
  /output:out/pipeline ^
  /extent:1023755,598875,1023954.98,599074.99 ^
  /tilesize:100,100 ^
  /buffer:20 ^
  /cellsize:3 ^
  /minht:2 ^
  /merge
```

19.3 Options used

Table 19.1: Options used in the `pipeline` example.

Option	Value	What it does
<code>/pipeline</code>	<code>groundfilter,canopymodel,canopymaxima</code>	The tools to run on each tile, in order. / <code>tool:<name></code> runs a single tool.
<code>/input</code>	<code>TC_1372_forest200ft.laz</code>	A LAS/LAZ file or a folder of them.
<code>/output</code>	<code>out/pipeline</code>	Folder for the stitched results.
<code>/extent</code>	the tile's bounds	The area to tile, as lower-left X, lower-left Y, upper-right X, upper-right Y.
<code>/tilesize</code>	<code>100,100</code>	Tile width and height, 100 ft.
<code>/buffer</code>	<code>20</code>	Extra points read around each tile, 20 ft.
<code>/cellsize</code>	<code>3</code>	Passed on to every stage that accepts it.
<code>/minht</code>	<code>2</code>	Passed on to every stage that accepts it.
<code>/merge</code>		Also merge each raster stage's tiles into one GeoTIFF, besides the virtual raster (<code>.vrt</code>).

Options that belong to individual tools (`/cellsize`, `/minht`, `/class`, `/strata`, and others) are forwarded to whichever stages accept them. Chapter 21. lists every forwarded option.

19.4 Outputs

- **Per raster stage:** `<stage>.vrt`, a virtual raster joining the tiles, and with `/merge` a single `<stage>_merged.tif` (`groundfilter_merged.tif`, `canopymodel_merged.tif`).
- **Per table stage:** one combined table (`canopymaxima_all.csv`).
- **_processing/:** each tile's intermediate files and log, plus `pipeline_state.csv`, which records the status, exit code, and times of every tile and stage. This is what lets a run resume.

19.5 Results

Figure 19.1 shows the stitched canopy model with tree tops (left), and the difference between the stitched and single-run canopy model (middle) and ground surface (right). Dashed lines mark the tile boundaries. The stitched map shows no seam along them, the difference maps are mostly near zero, and trees near a boundary appear once, not twice.

When the tile size is not a whole multiple of the cell size, as with 100 ft tiles and 3 ft cells, a tile's grid can sit a fraction of a cell away from the single-run grid. Cells then sample slightly different spots, and the canopy difference map shows scattered differences at crown edges in the shifted tiles. Choosing a tile size that is a multiple of the cell size avoids this.

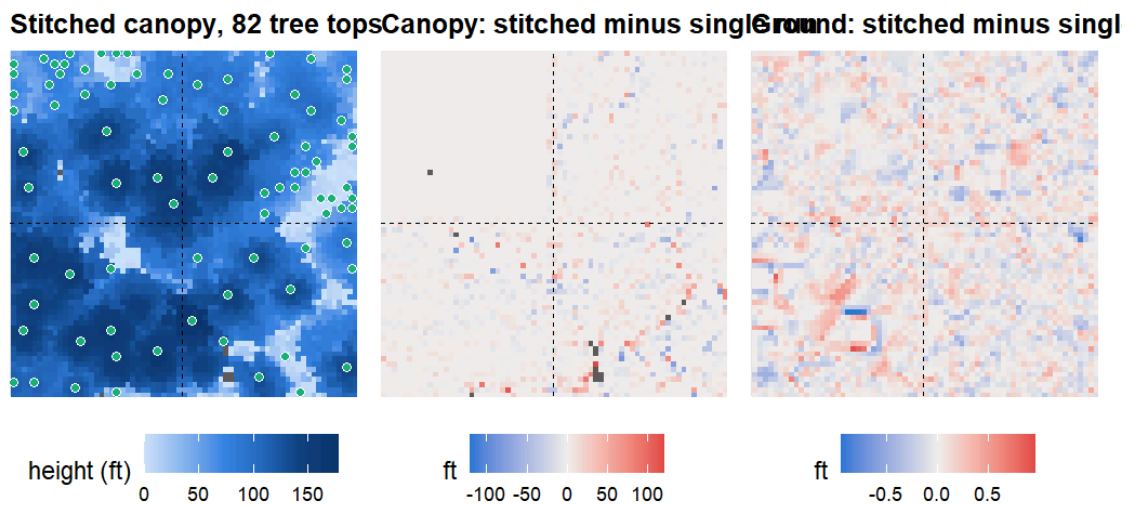


Figure 19.1: pipeline: stitched canopy with tree tops (left), and differences from the single-run canopy (middle) and ground (right). Dashed lines mark tile boundaries.

19.6 Checks

Table 19.2 confirms that all four tiles ran with no failed stage, that the stitched canopy model sits on the tile and matches the single-run canopy model within a median of 0.5 ft, and that no tree top is duplicated across tile seams.

Table 19.2: Automated checks on the example run of `pipeline`.

Check	Result	Observed	Expected
pipeline: exits with status 0	pass	0	0
run state table written	pass	pipeline_state.csv	pipeline_state.csv
no tile x stage failed	pass	done	no failed status
four tiles processed	pass	4	4
mosaicked canopy model written	pass	canopymodel_ merged.tif	a canopymodel .tif or .vrt
mosaicked CHM: raster sits on the tile's real coordinates	pass	1023755.00, 1023956.00, 598873.99, 59907...	1023755.00, 1023954.98, 598875.00, 59907...
mosaicked CHM matches single-run CHM (median diff < 0.5 ft)	pass	0.217	< 0.5 ft
no mosaicked CHM cell exceeds the tallest point above ground	pass	0	0
combined tree tops table written	pass	1	>= 1
no duplicate tree tops across tile seams	pass	0	0

20. Processing many tiles

This guide covers two related things: `gridmetrics.exe`'s own batch/tiled mode (for when you only need one tool run across a large area), and `pipeline.exe`, the general orchestrator for chaining several tools per tile. Both tile and buffer the input the same way; they differ in what they can run and how.

See Chapter 21. for the full option reference for every tool named here.

20.1 When you only need gridmetrics: its own batch/tiled mode

If your workflow is “run gridmetrics across a large area,” you don’t need `pipeline.exe` at all – point `gridmetrics.exe` at a directory instead of a file:

```
gridmetrics D:\project\las_tiles /outdir:D:\project\output\gridmetrics /
tilesize:1000,1000 /buffer:50 /threads:8 /cellsize:10 /minht:2
```

This tiles and buffers every LAS/LAZ file in `las_tiles`, computes grid metrics per tile in-process (no child processes, no `_processing` subfolder), and mosaics the results into `D:\project\output\gridmetrics\project_gridmetrics.vrt`.

20.2 Single-stage runs of any other tool: `pipeline.exe /tool:<name>`

Every other tool (`canopymodel`, `groundfilter`, `returndensity`, `filterdata`, `thindata`, `canopymaxima`, `topometrics`, `treeseg`) can also be run tiled across a whole project this way, one stage at a time:

```
pipeline /tool:groundfilter /input:D:\project\las_tiles /output:D:\project\output /
cellsize:1
```

This produces `D:\project\output\groundfilter.vrt` (the mosaicked ground DEM) plus the interim per-tile files under `D:\project\output\_processing\`.

20.3 The canonical pipeline: ground, then canopy height, then tree tops

```
pipeline /pipeline:groundfilter,canopymodel,canopymaxima ^
/input:D:\project\las_tiles /output:D:\project\output ^
/tilesize:1000,1000 /buffer:50 /threads:8 ^
/cellsize:1 /minht:2 /window-a:0.5 /window-b:0.05
```

Per tile, this: buffer-clips the input points once; runs `groundfilter` on that clip to get a DEM; runs `canopymodel` on the *same* clip, automatically passed `/ground:<the DEM groundfilter just produced>`, to get a CHM; runs `canopymaxima` on the CHM to detect tree tops. You never pass `/ground` yourself here – it’s wired through automatically because `groundfilter` precedes `canopymodel` in the chain.

Output: `groundfilter.vrt`, `canopymodel.vrt`, and `canopymaxima_all.csv` (every tile’s tree-top table concatenated, `TreeID` prefixed with the tile name so IDs stay unique) in `D:\project\output\`.

20.4 Crown segmentation instead of tree tops

```
pipeline /pipeline:groundfilter,canopymodel,treeseg /input:D:\project\las_tiles /
output:D:\project\output /minht:2
```

Output: `groundfilter.vrt`, `canopymodel.vrt`, `treeseg.vrt` (mosaicked crown-segment rasters), and `treeseg_table_all.csv`. Unlike `canopymaxima`’s table, `treeseg`’s crown-summary table has no per-

crown X/Y coordinates, so its rows can't be filtered to each tile's core extent the way `canopymaxima`'s can – a crown centered in a tile's buffer zone may show up in more than one tile's rows. The `treeseq.vrt` raster itself has no such overlap (it's cropped to core extent before mosaicking); only the table carries this limitation.

20.5 Terrain metrics only

```
pipeline /pipeline:groundfilter,topometrics /input:D:\project\las_tiles /output:D:\project\output
```

Output: `groundfilter.vrt` (DEM) and `topometrics.vrt` (slope + aspect bands).

20.6 Point-cloud-only chain (no raster stage at all)

```
pipeline /pipeline:thindata,filterdata,gridmetrics /input:D:\project\las_tiles /output:D:\project\output /cellsize:2 /return:1
```

`thindata` and `filterdata` both consume and produce point clouds, so they can chain directly into each other before `gridmetrics` (or any other point-cloud-input stage) runs on the result. The intermediate thinned/filtered `.laz` files stay in `_processing/<tile>/`; only `gridmetrics.vrt` gets mosaicked into `/output`, since per-tile point cloud outputs aren't merged across tiles (each tile's `.laz` is still a separate file, listed at the end of the run).

20.7 The `_processing/` subfolder and the state tracking file

Every stage's per-tile output lives under `<output>/_processing/<tile_name>/`:

```
_processing/
pipeline_state.csv      <- tile x stage status, read/written by every run
pipeline.log
tile_0001/
  clip.laz              <- Stage 0: buffered spatial clip (always first)
  log.txt               <- combined stdout/stderr for every stage's subprocess,
this tile
  groundfilter.tif      <- cropped to tile_0001's core extent
  canopymodel_buffered.tif <- buffered; feeds the next raster-consuming stage
  canopymodel.tif       <- cropped to core extent; what gets mosaicked
  canopymaxima.csv      <- filtered to core extent, TreeID prefixed with "tile_0001_"
tile_0002/
...
```

`pipeline_state.csv` has one row per tile x stage that has actually run:

```
tile,stage,status,output_path,exit_code,started_at,finished_at
tile_0001,clip,done,_processing/
tile_0001/clip.laz,0,2026-09-02T09:00:01,2026-09-02T09:00:04
tile_0001,groundfilter,done,_processing/
tile_0001/groundfilter.tif,0,2026-09-02T09:00:04,2026-09-02T09:00:07
tile_0001,canopymodel,failed,_processing/
tile_0001/canopymodel.tif,1,2026-09-02T09:00:07,2026-09-02T09:00:08
```

This file is both the resume cache and the answer to “what actually happened.”

20.8 Resuming and retrying

Re-running the exact same command is safe and cheap – every tile/stage already marked `done` in `pipeline_state.csv`, with its output file still present, is skipped:

```
pipeline /pipeline:groundfilter,canopymodel,canopymaxima /input:D:\project\las_tiles /output:D:\project\output
```

To force everything to redo regardless of recorded state, add `/rebuild`. To process only the tiles that had a failure last time (say, `canopymodel` crashed on a couple of tiles because of a bad input file you’ve since fixed):

```
pipeline /pipeline:groundfilter,canopymodel,canopymaxima /input:D:\project\las_tiles /output:D:\project\output /retryfailed
```

Or name specific tiles directly with `/tiles:tile_0004,tile_0017`. `/cleanup` (delete `_processing/` once everything succeeds) refuses to run if any tile/stage is still recorded as failed, precisely so `/retryfailed` still has something to work from.

20.9 Valid chaining

Every stage reads either a point cloud or a raster, and produces a point cloud, a raster, or a table:

Stage	Input	Output
<code>gridmetrics</code>	point cloud	raster
<code>canopymodel</code>	point cloud	raster
<code>groundfilter</code>	point cloud	raster
<code>returndensity</code>	point cloud	raster
<code>filterdata</code>	point cloud	point cloud
<code>thindata</code>	point cloud	point cloud
<code>canopymaxima</code>	raster	table
<code>topometrics</code>	raster	raster
<code>treeseq</code>	raster	raster + table

Two rules, both checked before any tile is processed:

1. **The first stage must consume a point cloud.** Nothing produces a raster before the chain starts, so a pipeline starting with `canopymaxima`, `topometrics`, or `treeseq` is rejected immediately with an error naming the offending stage.
2. **A raster-input stage must directly follow a raster-output stage.** `/pipeline:filterdata,canopymaxima` is rejected – `filterdata` produces a point cloud, not a raster, so `canopymaxima` has nothing to read. `/pipeline:groundfilter,canopymaxima` is accepted – `groundfilter`’s DEM satisfies `canopymaxima`’s raster input (even though feeding a bare ground DEM into a tree-top detector isn’t a meaningful workflow – `pipeline.exe` checks artifact *kind* compatibility, not whether the chain makes scientific sense).

Point-cloud-input stages have no such adjacency requirement: they always fall back to the nearest preceding point-cloud output, or Stage 0’s buffered clip if none of the earlier stages produced one.

III

Reference

21 Command-line options, tool by tool . 91

21.1	gridmetrics	91
21.2	cloudmetrics	93
21.3	canopymodel	94
21.4	canopymaxima	94
21.5	treeseq	94
21.6	groundfilter	94
21.7	clipdata	95
21.8	filterdata	96
21.9	thindata	96
21.10	returndensity	96
21.11	topometrics	96
21.12	catalog	97
21.13	pipeline	97
21.14	gridsurfacestats	98
21.15	densitymetrics	98

22 Built-in help for every tool 99

22.1	catalog	99
22.2	filterdata	99
22.3	thindata	99
22.4	groundfilter	100
22.5	clipdata	100
22.6	canopymodel	101
22.7	returndensity	101
22.8	densitymetrics	102
22.9	gridmetrics	102
22.10	cloudmetrics	103
22.11	topometrics	104
22.12	gridsurfacestats	104
22.13	canopymaxima	105
22.14	treeseq	105
22.15	pipeline	105

23 Metric definitions 107

23.1	NA vs. 0 – the /nodata and /noheight sentinel convention	107
23.2	Elevation Statistics & Percentiles	107
23.3	Canopy Cover & Structure Metrics	108
23.4	Pulse Density & Return Type Ratios	109
23.5	Surface Area Ratio, Roughness & Cut/Fill Volume	110
23.6	Vertical Density Stratum Bands & Per-Stratum Statistic Bundles	110
23.7	Novel 2D & 3D Experimental Spatial Metrics	112

24 C++ library interface (libfusion_core) 115

24.1	Point Cloud I/O (fusion::lidar)	115
24.2	Raster I/O & GDAL Engine (fusion::raster)	117
24.3	Parallel Batch Engine (fusion::batch)	118
24.4	Status Logging (fusion::batch)	118
24.5	Experimental Lidar Metrics (fusion::metrics)	119
24.6	CLI Argument Parsing (fusion::cli)	119

21. Command-line options, tool by tool

This chapter lists every option of every FUSION2 tool, with its default. The rules every tool shares are in Chapter 3.: option syntax, the `/class` and `/return` point filters (Section 3.4), units, missing values (`/nodata`, `/noheight`), and table output (`/output-table`, `/noraster`). Each tool's built-in help, printed with `/?`, is reproduced in Chapter 22.. Where this chapter and the help text differ, the help text comes from the tools themselves and is current.

Distances, heights, and cell sizes are in the input's own units (Chapter 3.).

21.1 gridmetrics

Calculates statistical elevation, canopy cover, percentile, and experimental metrics across a 2D spatial grid, outputting single-band or multi-band GeoTIFF rasters. Has two modes, chosen by whether the positional input is a file or a directory.

21.1.1 Single-file mode

```
gridmetrics <input.las/laz> [optional raster ground path] [other /options]
```

Processes one point cloud file, writing its raster(s) and CSV summary directly into `/outdir`. The ground surface DEM can be given either as the second positional argument or via `/ground:<path>` — `/ground` takes precedence if both are given.

21.1.2 Batch/tiled mode

```
gridmetrics <input_directory> /outdir:<dir> [/tilesize:...] [/buffer:...] [/threads:...] [other /options]
```

Tiles and buffers every LAS/LAZ file in the input directory, computes the same grid metrics per tile in parallel (multithreaded, in-process — no child processes), and mosaics the tile rasters into a `.vrt` in `/outdir`. Batch mode always writes one multiband raster per tile (`/output-mode` is ignored, since single-band mode would write many files per tile). With `/output-table`, each tile's table is written in parallel and the tiles are then joined, in tile order, into one project-level table.

A COPC (Cloud Optimized Point Cloud) file in the input directory is detected automatically (no separate flag) — its own chunk index is used to seek directly to the chunks overlapping each tile's buffered extent, instead of reading the whole file sequentially per tile. A plain (non-indexed) LAS/LAZ file is read sequentially. This mainly pays off when the input directory holds one very large regional COPC file rather than pre-tiled LAS/LAZ — pre-tiled input is already read efficiently without it.

- `/extent:<LLX,LLY,URX,URY>`: Batch mode only. Project extent (default: a 0,0,5000,5000 grid if omitted).
- `/tilesize:<w,h>`: Batch mode only. Tile width,height in project units (default: 1000,1000).
- `/buffer:<val>`: Batch mode only. Tile buffer distance (default: 50).
- `/threads:<N>`: Batch mode only. Number of parallel worker threads (default: 4).
- `/vrt`: Batch mode only. Generate a GDAL Virtual Raster (`.vrt`) across tile rasters (on by default).
- `/merge`: Batch mode only. Merge the VRT into a single global GeoTIFF file.

The `elev_*` and `int_*` bands are the full statistic bundle (min, max, mean, standard deviation, variance, coefficient of variation, skewness, kurtosis, canopy relief ratio, mode, median, interquartile range, percentiles, mean and median absolute deviations, L-moments, quadratic and cubic means, plus `elev_profile_area`); Chapter 23. lists every column. Every run also writes `r1count` to `r9count` (counts by return number) and the cover variants `allcover`, `afcover`, `allabovemean`, `allabovemode`, `afabovemean`, and `afabovemode`, computed from the unfiltered returns. Single-file and batch mode compute the same metrics, including `/rgb`, `/strata`, `/intstrata`, `/rgbstrata`, `/surfstats`, and `/exp`. The extra raster bands

added by `/strataraster`, and `/rgbstrata`'s raster bands, are written in single-file mode only; batch mode reports those values in `/output-table`.

21.1.3 Options & Flags (both modes, except where noted)

- `/ground:<path>`: Path to ground surface DEM raster (GeoTIFF, ENVI, IMG).
- `/cellsize:<val>`: Output grid cell size in project units (default: 10.0).
- `/minht:<val>`: Minimum height above ground for canopy metrics calculation (default: 2.0).
- `/heightcut:<val>`: Height cutoff threshold for canopy cover calculations (defaults to `/minht`).
- `/outlier:<min,max>`: Trim elevation outliers outside `min,max` values (e.g. `/outlier:-5,150`).
- `/class:<spec> //return:<spec>`: Point classification and return-number filtering – see Section 3.4.
- `/first`: Use only first returns for metric calculations.
- `/all`: Single-file mode only. Use all returns for canopy cover and metric calculations.
- `/nointensity`: Skip computing intensity metrics.
- `/strata:<h1,h2,...>`: Comma-separated height strata thresholds (e.g. `/strata:0.5,2.0,5.0,10.0,20.0`). Each stratum bucket gets the simplified 6-metric summary as CSV columns – `stratum_N_count`, `stratum_N_proportion`, `stratum_N_mean`, `stratum_N_stddev`, `stratum_N_min`, `stratum_N_max` (see Chapter 23.) – for that bucket's points. A bucket with zero points in an otherwise non-empty cell gets `/noheight` for its mean/stddev/min/max columns and a real 0 for its own count/proportion.
- `/intstrata:<h1,h2,...>`: Comma-separated intensity strata height thresholds (defaults to `/strata`'s thresholds if omitted). Buckets points by elevation the same way as `/strata`, but each bucket's simplified 6-metric summary (see Chapter 23.) is computed from the bucket's **intensity** values instead of elevation – `intstratum_N_count`, `intstratum_N_proportion`, `intstratum_N_mean`, `intstratum_N_stddev`, `intstratum_N_min`, `intstratum_N_max` CSV columns, and (with `/strataraster`) matching raster bands. Ignored if `/nointensity` is set.
- `/rgb:<spec>`: Comma-separated spectral channels to compute a full statistic bundle for – R, G, B, N (near-infrared), or `all` (every channel the input file's LAS point format actually carries). RGB is populated in point formats 2, 3, 5, 7, 8, 10; NIR only in formats 8 and 10 – a requested channel the file's format doesn't carry prints a warning naming the channel and the point format, and is simply omitted (never a hard failure). Emits `red_*/green_*/blue_*/nir_*` bands, one full bundle per selected, available channel, gated by the same `/minht` cutoff and `/noheight//nodata` rule as `int_*`.
- `/rgbstrata`: With `/rgb` and `/strata` both set. Also reports a mean/stddev/min/max summary per selected spectral channel within each height-stratum bucket – `<channel>_stratum_N_mean/stddev/min/max` (e.g. `red_stratum_00_mean`), as both CSV columns and (with `/strataraster`) raster bands. Count/proportion aren't repeated here since the elevation `/strata` columns/bands sharing the same bucket boundaries already report them. Ignored (with a warning) if `/rgb` or `/strata` is missing.
- `/voxelsize:<val>`: Single-file mode only. 3D voxel resolution for voxel volume metrics (default: 20.0).
- `/exp`: Single-file mode only. Compute additional experimental metrics from RSForTools.
- `/outroot:<name>`: Single-file mode only. Base root name for output CSV summary metrics tables.
- `/outdir:<path>`: Output directory for rasters and CSV reports (single-file mode) or for tile rasters and the mosaicked VRT (batch mode). Default: ..
- `/output-mode:<mode>`: Single-file mode only. Output raster mode, `multiband` or `singleband` (default: `multiband`).
- `/surfstats`: Single-file mode only. Compute `surface_area_ratio` and `roughness` bands from the per-cell elevation grid gridmetrics already holds in memory (no second raster round-trip). See `/surfstats-source` and Chapter 23. for the underlying math.
- `/surfstats-source:<max|mean>`: With `/surfstats`. Which per-cell elevation to feed the surface-stats math – `max` (default) suits typical CHM top-surface use, `mean` suits ground-DTM-style runs.
- `/strataraster`: With `/strata`. Also append, per stratum bucket, a return-density band (`density_stratum_00, ...`), a count band (`stratum_00_count, ...`), a proportion band (`stratum_00_proportion, ...`), and the simplified mean/stddev/min/max band set (`stratum_00_mean, stratum_00_stddev, stratum_00_min, stratum_00_max`) to the multiband output – the same values `/strata` already writes to CSV, just also as raster bands. With `/intstrata` (and `/nointensity` not set), also appends a matching `intstratum_00_count/proportion/mean/stddev/min/max` band set per bucket – no density band, since return density is tied to the elevation bucketing above, not to intensity. With `/rgbstrata`, also appends that per-channel mean/stddev/min/max band set.
- `/nodata:<NA|number>`: Value for cells with zero returns at all (default: `NA`). Applies to every band and, in the CSV export, every column including `TotalReturns/FirstReturns`.

- `/noheight:<NA|number>`: Value for height-dependent bands (`elev_*`, `int_*`, and `/strataraster's//rgbstrata's` mean/stddev/min/max bands) when a cell (or stratum bucket) has returns but none clear `/minht/land` in that bucket (default: 0). Never applied to `canopy_cover`, `point_density`, or `/strataraster's` own density/count/proportion bands – those are well-defined directly from the unfiltered return count. If both `/nodata` and `/noheight` are explicitly set to different, non-NA values, `gridmetrics` prints a warning and uses `/nodata's` value as the GeoTIFF's registered NoData value (GDAL supports only one per file); `/noheight's` value is still written as an ordinary pixel, just not flagged as NoData by the file header.

21.2 cloudmetrics

Computes summary canopy, elevation, and intensity metrics for a single point cloud file or an entire directory of point clouds. Supports height normalization via a ground DTM raster file or a directory of DTM tiles.

```
cloudmetrics <input.las/laz or directory> [optional ground DTM file or directory]
[other /options]
```

21.2.1 Options & Flags

- `/output:<path>`: Output CSV file path (default: `cloud_metrics.csv`).
- `/ground:<path>`: Path to ground DEM raster for height normalization (GeoTIFF, ENVI, IMG), or a directory of DTM tiles to mosaic on the fly via an in-memory VRT. May also be supplied as an unflagged second positional argument.
- `/minht:<val>`: Minimum height cutoff for canopy metrics (default: 2.0).
- `/cellsize:<val>`: Grid cell size for 2D area/volume metrics (default: 10.0).
- `/voxelsize:<val>`: 3D voxel resolution for voxel volume metrics (default: 20.0).
- `/exp`: Compute additional experimental metrics from RSForTools.
- `/surfstats`: Compute `SurfaceAreaRatioMean`, `RoughnessMean`, `PlanimetricArea`, and `SurfaceArea3D` from a `/cellsize` elevation grid (max height per cell across the point cloud). Not available together with `/shape`.
- `/shape:<path.shp>`: Compute one metrics row per polygon feature instead of one row for the whole cloud – streams the full, uncropped point cloud exactly once, routing each point into whichever feature's polygon contains it. Replaces the legacy two-step `PolyClipData`-then-`CloudMetrics` workflow (no intermediate per-plot LAS files written or reread).
- `/field:<name>`: With `/shape`. Attribute field used to label each output row (`Label` column); falls back to a zero-padded feature index when omitted or missing on a feature.
- `/nodata:<NA|number>`: Value for a cloud/feature with zero points at all (default: NA). Applies to every CSV column, including `TotalPoints/CanopyPoints`.
- `/noheight:<NA|number>`: Value for height-dependent columns (the full `elev_*/int_*` statistic bundle – see below) when points exist but none clear `/minht` (default: 0). Never applied to `CanopyCoverPct`, which is well-defined directly from the unfiltered point count.
- `/class:<spec> /return:<spec>`: Point classification and return-number filtering – see Section 3.4.
- `/strata:<h1,h2,...>`: Comma-separated height strata thresholds – same syntax as `gridmetrics' /strata`. Each bucket gets the simplified 6-metric summary as CSV columns: `stratum_N_count`, `stratum_N_proportion`, `stratum_N_mean`, `stratum_N_stddev`, `stratum_N_min`, `stratum_N_max` (see Chapter 23.).
- `/intstrata:<h1,h2,...>`: Comma-separated height thresholds bucketing points the same way as `/strata` (defaults to `/strata's` thresholds if omitted), but reporting an `intstratum_N_*` **intensity** summary per bucket instead of elevation.
- `/rgb:<spec>`: Comma-separated spectral channels to compute a full statistic bundle for – R, G, B, N, or all – same semantics as `gridmetrics' /rgb` (see above), emitting `red_*/green_*/blue_*/nir_*` CSV columns.
- `/rgbstrata`: With `/rgb` and `/strata` both set. Also reports a mean/stddev/min/max summary per selected spectral channel within each height-stratum bucket as CSV columns – `<channel>_stratum_N_mean/stddev/min/max` (e.g. `red_stratum_00_mean`). Ignored (with a warning) if `/rgb` or `/strata` is missing.

`cloudmetrics`' output CSV carries the same full `elev_*/int_*` statistic bundle (plus `elev_profile_area`) that `gridmetrics` writes per cell – see Chapter 23. for the column list. The `/strata/intstrata/rgbstrata` per-bucket columns use the simplified 6-metric (or 4-metric, for `/rgbstrata`) summary instead, not this full bundle.

21.3 canopymodel

Interpolates point cloud data to create a continuous Digital Canopy Height Model (CHM) GeoTIFF raster. Accepts a single `.las/.laz` file or an entire directory of point clouds.

```
canopymodel <input.las/laz or directory> [other /options]
```

`/output` is required — the tool errors out if it is not given.

21.3.1 Options & Flags

- `/output:<path>`: Output GeoTIFF CHM file path. **Required.**
- `/cellsize:<val>`: Output CHM cell size (default: 1.0).
- `/ground:<path>`: Path to ground DEM raster file for height normalization, or a directory of DTM tiles to mosaic on the fly.
- `/slope`: Normalize heights perpendicular to the local terrain slope plane.
- `/smooth:<n>`: Spatial smoothing window size (e.g. `/smooth:3` for a 3x3 filter).
- `/class:<spec> //return:<spec>`: Point classification and return-number filtering – see Section 3.4.

21.4 canopymaxima

Detects individual tree tops on CHM rasters using Variable Window Local Maxima (VLM) filtering.

```
canopymaxima <input_chm.tif> [other /options]
```

The search window's radius at height `H` is `window-a + window-b * H`, in the canopy model's units, rounded to whole cells and never less than one cell.

21.4.1 Options & Flags

- `/output:<path>`: Output CSV file path for tree tops (default: `tree_tops.csv`).
- `/minht:<val>`: Minimum tree height threshold (default: 2.0).
- `/window-a:<val>`: Window polynomial coefficient A, the constant term (default: 0.5).
- `/window-b:<val>`: Window polynomial coefficient B, the linear term (default: 0.05).

21.5 treeseq

Performs watershed region-growing individual tree crown segmentation on a CHM raster.

```
treeseq <input_chm.tif> [other /options]
```

21.5.1 Options & Flags

- `/output-raster:<path>`: Output GeoTIFF raster path for tree segments (default: `crown_segments.tif`).
- `/output-table:<path>`: Output CSV summary table path (default: `crown_summary.csv`).
- `/minht:<val>`: Minimum height cutoff for segmentation (default: 2.0).

21.6 groundfilter

Classifies ground points in a LAS/LAZ point cloud file or directory with the Kraus & Pfeifer iterative robust filter (the algorithm behind legacy FUSION's GroundFilter), and generates a bare-earth GeoTIFF DEM from them. The input's existing classification is not used, so unclassified data works.

```
groundfilter <input.las/laz or directory> [other /options]
```

How it works, in brief: 1. A coarse stage fits a rough ground surface on `/coarsecell` cells. Any point more than `/coarsecut` above it is ruled out as ground. This removes canopy over gaps in the ground returns that are too wide for the fine stage to see across. 2. The fine stage repeats `/iterations` passes on `/filtercell` cells. Each pass fits, per cell, a weighted least-squares plane through the points in that cell and its 8 neighbours. Each point then gets a weight from its residual v (elevation minus surface): 1 for $v \leq g$, 0 for $v > g + w$, and $1 / (1 + (a(v - g))^b)$ in between. Points above the surface lose weight, so the surface sinks through the canopy to the ground. 3. Points with $v \leq g + w$ against the final surface (or within `/tolerance` of it) are ground. The DEM is each `/cellsize` cell's mean ground elevation. Cells without a ground point are filled outward from their neighbours, so the DEM has no empty cells.

`g`, `w`, `/coarsecut`, and `/tolerance` are in the point cloud's vertical units. The defaults follow legacy FUSION.

All points passing `/class//return` are held in memory (about 24 bytes each). For very large inputs, run it per tile through `pipeline`.

21.6.1 Options & Flags

- `/cellsize:<val>`: Output DEM cell size (default: 1.0).
- `/filtercell:<val>`: Cell size of the fine stage's surfaces. Each cell's plane is fitted over its 3 x 3 neighbourhood (default: 10.0).
- `/coarsecell:<val>`: Cell size of the coarse stage (default: 3 x `/filtercell`; 0 disables the coarse stage).
- `/coarsecut:<val>`: Height above the coarse surface beyond which a point cannot be ground (default: 4 x `/wparam`).
- `/gparam:<val>`: g , the residual at or below which a point gets full weight (default: -2.0).
- `/wparam:<val>`: w , the width above g over which the weight falls to 0 (default: 2.5).
- `/aparam:<val>`: a , steepness of the weight function (default: 1.0).
- `/bparam:<val>`: b , exponent of the weight function (default: 4.0).
- `/iterations:<n>`: Number of fine-stage passes (default: 5).
- `/tolerance:<val>`: Classify as ground every point within this distance of the final surface, instead of every point with residual $\leq g + w$.
- `/output-raster:<path>`: Output GeoTIFF ground DEM file path.
- `/output-points:<path>`: Output LAS/LAZ of the points classified as ground, written with classification 2.
- `/class:<spec>` / `//return:<spec>`: Point classification and return-number filtering – see Section 3.4. Applied before filtering, so noise (classes 7/18) and withheld returns never shape the ground surface.

21.7 clipdata

Subsets point cloud data from a file or directory by 2D spatial extent or height range, with optional height normalization.

```
clipdata <input.las/laz or directory> [other /options]
```

`/output` is required — the tool errors out if it is not given.

21.7.1 Options & Flags

- `/output:<path>`: Output LAS/LAZ file path. **Required.**
- `/extent:<LLX,LLY,URX,URY>`: Bounding box to clip to.
- `/ground:<path>`: Path to ground surface raster (GeoTIFF, ENVI, IMG) or directory of DTM tiles for height normalization.
- `/minz:<val>`: Minimum height above ground (or elevation, if no `/ground` given).
- `/maxz:<val>`: Maximum height above ground (or elevation, if no `/ground` given).
- `/shape:<path.shp>`: Polygon shapefile to clip against, in addition to (not instead of) `/extent` – both apply if both are given.
- `/multifile`: With `/shape`. Write one output LAS/LAZ per polygon feature instead of one merged output, into the directory given by `/output`. Each feature's writer is opened lazily on its first matched point, so polygons with no returns don't create empty files.

- `/field:<name>`: With `/multifile`. Attribute field used to name each output file (`<field-value>.laz`); falls back to a zero-padded feature index when omitted or missing on a feature.
- `/outside`: Keep points outside every polygon instead of inside. Single-clip mode only (no `/multifile`).
- `/class:<spec> / /return:<spec>`: Point classification and return-number filtering, applied before the spatial/height clip – see Section 3.4.

21.8 filterdata

Filters a point cloud file or directory by elevation range, return number, or classification.

```
filterdata <input.las/laz or directory> [other /options]
```

21.8.1 Options & Flags

- `/output:<path>`: Output filtered LAS/LAZ file path (default: `filtered_output.laz`).
- `/minz:<val>`: Minimum Z elevation threshold.
- `/maxz:<val>`: Maximum Z elevation threshold.
- `/class:<spec> / /return:<spec>`: Point classification and return-number filtering – see Section 3.4.

21.9 thindata

Spatially thins a point cloud file or directory by keeping a single point per grid cell.

```
thindata <input.las/laz or directory> [other /options]
```

21.9.1 Options & Flags

- `/output:<path>`: Output thinned LAS/LAZ file path (default: `thinned_output.laz`).
- `/cellsize:<val>`: Grid cell size for 2D spatial thinning (default: `1.0`).
- `/class:<spec> / /return:<spec>`: Point classification and return-number filtering, applied before thinning – see Section 3.4.

21.10 returndensity

Calculates point density (points per square unit) and return-type ratio rasters from a point cloud file or directory, saved as a multi-band GeoTIFF.

```
returndensity <input.las/laz or directory> [other /options]
```

21.10.1 Options & Flags

- `/output:<path>`: Output multi-band GeoTIFF raster path (default: `density_metrics.tif`).
- `/cellsize:<val>`: Output grid cell size (default: `5.0`).

21.11 topometrics

Computes topographic terrain slope and aspect from an input DEM GeoTIFF, saved as a multi-band GeoTIFF.

```
topometrics <input_dem.tif> [other /options]
```

21.11.1 Options & Flags

- `/output:<path>`: Output multi-band GeoTIFF raster path (default: `topo_metrics.tif`).

21.12 catalog

Scans a point cloud file or directory to produce a summary acquisition CSV report and an optional density coverage raster.

```
catalog <input.las/laz or directory> [other /options]
```

21.12.1 Options & Flags

- `/density:<cellsize>`: Generates a pulse density coverage raster at the given cell size.
- `/output:<path>`: Output CSV summary report file path.

21.13 pipeline

Multi-tool batch pipeline orchestrator. Tiles and buffers the input point cloud once, then – per tile – chains any of 10 tools (`groundfilter`, `canopymodel`, `gridmetrics`, `densitymetrics`, `returndensity`, `filterdata`, `thindata`, `canopymaxima`, `topometrics`, `treeseg`) by running each one's own `.exe` as a child process, in order. For `gridmetrics` alone, `gridmetrics`' own batch mode (above) is simpler and runs in a single process. See Chapter 20. for worked end-to-end examples, the valid-chaining rule, and the state/resume model.

```
pipeline /pipeline:<stage1,stage2,...> /input:<dir> /output:<dir> [other /options]
```

`/input` and `/output` are both required. Either `/pipeline:<stage1,stage2,...>` (a chain) or `/tool:<name>` (a single stage) is required.

21.13.1 Options & Flags

- `/pipeline:<stage1,stage2,...>`: Ordered list of stages to chain per tile, e.g. `/pipeline:groundfilter,canopymodel,canopymaxima`.
- `/tool:<name>`: Shorthand for a single-stage `/pipeline:<name>`.
- `/input:<dir>`: Input directory containing LAS/LAZ files. **Required.**
- `/output:<dir>`: Output directory for each stage's finalized (mosaicked/concatenated) result. **Required.**
- `/extent:<LLX,LLY,URX,URY>`: Project extent (default: a 0,0,5000,5000 grid if omitted).
- `/tilesize:<w,h>`: Tile width,height in project units (default: 1000,1000).
- `/buffer:<val>`: Tile buffer distance (default: 50).
- `/threads:<N>`: Number of parallel worker threads – each one runs one tile's stage chain at a time, so this is also how many child processes run concurrently (default: 4).
- `/processingdir:<path>`: Interim-product and state subfolder (default: `<output>/_processing`). Holds every stage's per-tile output plus `pipeline_state.csv`, the file recording each tile and stage's status.
- `/rebuild`: Ignore recorded state and redo every tile and stage, instead of the default resume-from-state behavior.
- `/tiles:<name1,name2,...>`: Process only the named tiles instead of the whole extent.
- `/retryfailed`: Process only tiles with a recorded failed stage in `pipeline_state.csv`.
- `/toolsdir:<path>`: Directory containing the sibling tool `.exe` files (default: the directory `pipeline.exe` itself is running from).
- `/cleanup`: Delete the processing subfolder after a fully successful run (skipped if any tile/stage failed, so `/retryfailed` still has something to resume from).
- `/merge`: Also merge each raster stage's VRT into a single global GeoTIFF.

The following options are forwarded to whichever stage(s) in the chain accept them (see each stage's own section above for what it does): `- /cellsize, /minht, /heightcut, /outlier, /class, /strata, /intstrata, /ground, /window-a, /window-b, /smooth, /minz, /maxz, /return - /first, /nointensity, /slope`

If `groundfilter` runs earlier in the chain, its DEM output is passed automatically as `/ground` to any later stage that accepts it (`canopymodel`, `gridmetrics`, `densitymetrics`) – an explicit `/ground:<path>` is only needed when no `groundfilter` stage precedes it.

21.14 gridsurfacestats

Computes surface area ratio and roughness from a DEM/CHM GeoTIFF (a standalone raster-in/raster-out tool – the same math is also available inline from `gridmetrics/cloudmetrics` via `/surfstats`, see above). With `/reference`, also computes per-cell cut/fill volume against a second, independently produced surface.

```
gridsurfacestats <input_surface.tif> [other /options]
```

21.14.1 Options & Flags

- `/output:<path>`: Output multi-band GeoTIFF raster path (default: `surface_stats.tif`).
- `/reference:<path>`: A second surface GeoTIFF, the same cols x rows as the input, to diff against for cut/fill volume_diff. Omit to skip cut/fill entirely (2-band output instead of 3).

The output raster inherits its NoData value directly from the input surface's own registered NoData – there is no separate `/nodata` option, since a cell already flagged NoData in the source DEM/CHM has no basis for a surface-stats value either.

21.15 densitymetrics

Computes a return-density raster stack across vertical height slices – one band per `/strata` bucket (`density_stratum_00`, `density_stratum_01`, ...), each cell holding that bucket's return count per unit area. It computes the same per-layer density bands as `gridmetrics /strata /strataraster`, as a smaller, faster tool, keeps legacy FUSION's `DensityMetrics` name, and can run as a pipeline stage.

```
densitymetrics <input.las/laz or directory> [optional raster ground path] [other / options]
```

21.15.1 Options & Flags

- `/strata:<h1,h2,...>`: Comma-separated height strata thresholds, same syntax as `gridmetrics' /strata` (default: `0.5,2.0,5.0,10.0,20.0`).
- `/cellsize:<val>`: Output grid cell size in project units (default: `10.0`).
- `/ground:<path>`: Path to ground surface DEM raster (GeoTIFF, ENVI, IMG), or a directory of DTM tiles to mosaic on the fly. May also be supplied as an unflagged second positional argument.
- `/class:<spec> /return:<spec>`: Point classification and return-number filtering – see Section 3.4.
- `/output:<name>`: Base output name (stem) for the raster/CSV files (default: derived from the input filename).
- `/outdir:<path>`: Output directory for the raster and CSV report (default: `.`).
- `/nodata:<NA|number>`: Value for cells with zero returns at all (default: `NA`). Applies to every stratum band and, in the CSV export, every column including `TotalReturns`. A non-empty cell's stratum bands are always real computed counts (including a legitimate `0` for an empty bucket) – `densitymetrics` has no `/noheight` option, since nothing it computes is height-filtered the way `elev_*/int_*` bands are.

22. Built-in help for every tool

Each FUSION2 tool prints its own list of options when run with `/?`, `-h`, or `--help`. This chapter reproduces that output for all 15 tools. `build.ps1` captures it from the freshly built tools on every build, so it always matches the release this manual ships with. Chapter 21. explains the options in more detail. The `/class` and `/return` options that appear under many tools are described once in Section 3.4.

22.1 catalog

```
Usage: catalog <input.las/laz or directory> [other /options]
Summarizes LAS/LAZ point cloud acquisition stats and density rasters
```

Options:

```
/output:<value>   Output CSV summary report file path (default: )
/density:<value>  Output GeoTIFF point density raster cell size (default: )
```

22.2 filterdata

```
Usage: filterdata <input.las/laz or directory> [other /options]
Point Cloud Filtering Tool (Elevation, Return Type, Classification, Scan Angle)
```

Options:

```
/return:<value>   Return numbers to keep: /return:1 or /return:1,2 for an explicit list
(ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /
return:only, /return:intermediate. Omit to keep every return. (default: )
/class:<value>    Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/maxz:<value>     Maximum Z elevation threshold (default: )
/minz:<value>     Minimum Z elevation threshold (default: )
/output:<value>   Output filtered LAS/LAZ file path (default: filtered_output.laz)
```

22.3 thindata

```
Usage: thindata <input.las/laz or directory> [other /options]
Spatial Point Cloud Thinning / Decimation Tool
```

Options:

```
/return:<value>   Return numbers to keep: /return:1 or /return:1,2 for an explicit list
(ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /
return:only, /return:intermediate. Omit to keep every return. (default: )
/class:<value>    Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/cellsize:<value> Grid cell size for 2D spatial thinning (same units as the input)
(default: 1.0)
/output:<value>   Output thinned LAS/LAZ file path (default: thinned_output.laz)
```

22.4 groundfilter

Usage: groundfilter <input.las/laz or directory> [other /options]
Classifies ground points with the Kraus & Pfeifer iterative filter and generates a GeoTIFF ground DEM

Options:

/noraster Skip writing the ground DEM GeoTIFF -- only valid together with /output-table, since a run must produce at least one output
/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list (ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /return:only, /return:intermediate. Omit to keep every return. (default:)
/cellsize:<value> Output DEM cell size (default: 1.0)
/filtercell:<value> Cell size of the filter's intermediate surfaces (horizontal units); each surface cell averages its 3 x 3 neighbourhood (default: 10.0)
/gparam:<value> Kraus & Pfeifer g: residual at or below which a point gets full weight (vertical units) (default: -2.0)
/output-table:<value> Also write a one-band multicolumn table alongside the ground DEM raster (path ending in .csv or .sqlite) -- omit to skip table output entirely (default:)
/tolerance:<value> Classify as ground every point within this distance of the final surface (default: every point with residual <= g + w) (default:)
/wparam:<value> Kraus & Pfeifer w: width above g over which a point's weight falls to 0 (vertical units) (default: 2.5)
/aparam:<value> Kraus & Pfeifer a: steepness of the weight function (default: 1.0)
/iterations:<value> Number of surface/weight passes (default: 5)
/output-raster:<value> Output GeoTIFF ground DEM file path (default:)
/coarsecell:<value> Cell size of a first, coarse filter stage; points more than /coarsecut above its surface are dropped before the fine passes (default 3 x /filtercell; 0 disables) (default:)
/bparam:<value> Kraus & Pfeifer b: exponent of the weight function (default: 4.0)
/coarsecut:<value> Height above the coarse surface beyond which a point cannot be ground (vertical units; default 4 x /wparam) (default:)
/output-points:<value> Output LAS/LAZ of the points classified as ground (written with classification 2) (default:)
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every other class). (default:)

22.5 clipdata

Usage: clipdata <input.las/laz or directory> [other /options]
Clips LAS/LAZ point clouds by bounding box or spatial extents

Options:

/outside Keep points outside every polygon instead of inside (single-clip mode only)
/multifile With /shape, write one output LAS/LAZ per polygon feature instead of one merged output
/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list (ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /return:only, /return:intermediate. Omit to keep every return. (default:)
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every other class). (default:)
/field:<value> Attribute field used to name each /multifile output (<field-value>.laz); falls back to a zero-padded feature index when omitted or missing on a feature (default:)

```

/maxz:<value> Maximum height above ground or elevation (default: )
/minz:<value> Minimum height above ground or elevation (default: )
/shape:<value> Polygon shapefile to clip against, in addition to (not instead
of) /extent (default: )
/ground:<value> Path to ground surface raster (GeoTIFF, ENVI, IMG) for height
normalization, or a directory of DTM tiles to mosaic on the fly (default: )
/output:<value> Output LAS/LAZ file path (single-clip mode), or output directory
(/multifile mode) (default: )
/extent:<value> Bounding box LLX,LLY,URX,URY (default: )

```

22.6 canopymodel

Usage: canopymodel <input.las/laz or directory> [other /options]
 Generates Canopy Height Model (CHM) GeoTIFF from point cloud

Options:

```

/noraster Skip writing the GeoTIFF raster -- only valid together with /output-table,
since a run must produce at least one output
/slope Normalize heights perpendicular to local terrain slope plane
/output-table:<value> Also write a one-band multicolumn table alongside the raster
(path ending in .csv or .sqlite) -- omit to skip table output entirely (default: )
/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list
(ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /
return:only, /return:intermediate. Omit to keep every return. (default: )
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/smooth:<value> Spatial smoothing window size (e.g. 3 for 3x3 filter) (default: )
/output:<value> Output GeoTIFF CHM file path (default: )
/ground:<value> Path to ground DEM raster for height normalization, or a directory
of DTM tiles to mosaic on the fly (default: )
/cellsize:<value> Output CHM cell size (default: 1.0)

```

22.7 returndensity

Usage: returndensity <input.las/laz or directory> [other /options]
 Generates Point Density and Return Ratio GeoTIFF Rasters

Options:

```

/noraster Skip writing the GeoTIFF raster -- only valid together with /output-table,
since a run must produce at least one output
/output-table:<value> Also write a multicolumn table alongside the raster (path
ending in .csv or .sqlite) -- omit to skip table output entirely (default: )
/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list
(ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /
return:only, /return:intermediate. Omit to keep every return. (default: )
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/cellsize:<value> Output grid cell size (default: 5.0)
/output:<value> Output multi-band GeoTIFF raster path (default: density_metrics.tif)

```

22.8 densitymetrics

Usage: densitymetrics <input.las/laz or directory> [optional raster ground path] [other /options]

Computes a return-density raster stack across vertical height slices (one band per /strata bucket)

Options:

/noraster Skip writing the GeoTIFF raster -- only valid together with /output-table, since a run must produce at least one output

/output-table:<value> Write a multicolumn table of per-cell stratum counts (path ending in .csv or .sqlite) -- omit to skip table output entirely (default:)

/nodata:<value> Value for cells with zero returns at all: NA, or a number such as 0, -9999, or inf. A non-empty cell's stratum bands are always real computed counts, never this sentinel. (default: NA)

/output:<value> Base output name (stem) for the raster/CSV files (default:)

/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list (ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /return:only, /return:intermediate. Omit to keep every return. (default:)

/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every other class). (default:)

/ground:<value> Path to ground surface DEM raster (GeoTIFF, ENVI, IMG), or a directory of DTM tiles to mosaic on the fly (default:)

/outdir:<value> Output directory for the raster and CSV report (default: .)

/cellsize:<value> Output grid cell size in project units (default: 10.0)

/strata:<value> Comma-separated height strata thresholds (e.g. 0.5,2.0,5.0,10.0,20.0) -- same syntax as gridmetrics' /strata (default: 0.5,2.0,5.0,10.0,20.0)

22.9 gridmetrics

Usage: gridmetrics <input.las/laz or directory> [optional raster ground path] [other /options]

Computes comprehensive canopy elevation and intensity metrics grid from point clouds

Options:

/merge Batch/tiled mode: merge the VRT into a single global GeoTIFF file

/vrt Batch/tiled mode: generate a GDAL Virtual Raster (.vrt) across tile rasters (on by default)

/noraster Skip writing the GeoTIFF raster(s) -- only valid together with /output-table, since a run must produce at least one output

/exp Compute additional experimental metrics from RSForTools

/strataraster With /strata, also append one return-density band per stratum bucket to the multiband output (density_stratum_NN)

/surfstats Compute surface_area_ratio and roughness bands from the per-cell elevation grid (see /surfstats-source)

/nointensity Skip computing intensity metrics

/all Use all returns for canopy cover and metric calculations

/rgbstrata With /rgb and /strata both set, also append a mean/stddev/min/max band set (and matching CSV columns) per selected spectral channel within each height-stratum bucket (<channel>_stratum_NN_*).

/first Use only first returns for metric calculations

/buffer:<value> Batch/tiled mode: tile buffer distance (default: 50)

/tilesize:<value> Batch/tiled mode: tile width,height in project units (default: 1000,1000)

/extent:<value> Batch/tiled mode: project extent LLX,LLY,URX,URY (default:)

```

/output-table:<value> Write the full per-cell metrics table alongside the raster
(path ending in .csv or .sqlite) -- omit to skip table output entirely (default: )
/output-mode:<value> Output raster mode: multiband or singleband (default: multiband)
/outdir:<value> Output directory for rasters and CSV reports (also the batch/tiled
mode output directory) (default: .)
/outroot:<value> Base root name for output CSV summary metrics tables (default: )
/surfstats-source:<value> Elevation source for /surfstats: max (typical CHM top-
surface use) or mean (ground-DTM-style runs) (default: max)
/rgb:<value> Comma-separated spectral channels to compute a statistic bundle for:
R, G, B, N, or all (every channel the input file's LAS point format actually carries)
(default: )
/cellsize:<value> Output grid cell size in project units (default: 10.0)
/strata:<value> Comma-separated height strata thresholds (e.g.
0.5,2.0,5.0,10.0,20.0) (default: )
/outlier:<value> Trim elevation outliers outside min,max values (e.g. -5,150)
(default: )
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/ground:<value> Path to ground surface DEM raster (GeoTIFF, ENVI, IMG), or a
directory of DTM tiles to mosaic on the fly (default: )
/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list
(ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /
return:only, /return:intermediate. Omit to keep every return. (default: )
/heightcut:<value> Height cutoff threshold for canopy cover calculations (defaults
to minht) (default: )
/noheight:<value> Value for height-dependent bands (elev_*, int_*) when a cell has
returns but none clear the height cutoff: NA, or a number such as 0, -9999, or inf
(default: 0)
/intstrata:<value> Comma-separated intensity strata height thresholds (default: )
/threads:<value> Batch/tiled mode: number of parallel worker threads (default: 4)
/minht:<value> Minimum height above ground for canopy metrics calculation (default:
2.0)
/voxelsize:<value> 3D voxel resolution for voxel volume metrics (same units as
the input) (default: 20.0)
/nodata:<value> Value for cells with zero returns at all: NA, or a number such as
0, -9999, or inf (default: NA)

```

22.10 cloudmetrics

Usage: cloudmetrics <input.las/laz or directory> [optional ground DTM path] [other / options]

Computes Summary Metrics for Point Cloud Clips

Options:

/rgbstrata With /rgb and /strata both set, also report a mean/stddev/min/max bundle per selected spectral channel within each height-stratum bucket (<channel>_stratum_NN_*).

/surfstats Compute surface area ratio, roughness, planimetric area, and 3D surface area from a /cellsize elevation grid (top-of-cloud max per cell). Not available with /shape.

/exp Compute additional experimental metrics from RSForTools

/output:<value> Output CSV file path (default: cloud_metrics.csv)

/return:<value> Return numbers to keep: /return:1 or /return:1,2 for an explicit list (ranges like /return:1-2 also work), or the mnemonics /return:first, /return:last, /return:only, /return:intermediate. Omit to keep every return. (default:)

/ground:<value> Path to ground DEM raster for height normalization, or a directory of DTM tiles to mosaic on the fly (default:)

```

/rgb:<value> Comma-separated spectral channels to compute a statistic bundle for:
R, G, B, N, or all (every channel the input file's LAS point format actually carries)
(default: )
/cellsize:<value> Grid cell size for 2D area/volume metrics (same units as the input)
(default: 10.0)
/shape:<value> Polygon shapefile -- compute one metrics row per polygon feature
instead of one row for the whole cloud (default: )
/strata:<value> Comma-separated height strata thresholds (e.g.
0.5,2.0,5.0,10.0,20.0) -- same syntax as gridmetrics' /strata. Appends stratum_N_count/
proportion plus a full elevation statistic bundle per bucket. (default: )
/field:<value> Attribute field used to label each /shape polygon's output row; falls
back to a zero-padded feature index when omitted or missing on a feature (default: )
/class:<value> Point classifications to keep. Omit for the default (excludes ASPRS
noise classes 7 and 18); /class:all or /class:* keeps every class; /class:2,3,4,5 or /
class:1-5 whitelists the listed classes; /class:~7,9,18 blacklists them (keeps every
other class). (default: )
/minht:<value> Minimum height cutoff for canopy metrics (same units as the input)
(default: 2.0)
/voxelsize:<value> 3D voxel resolution for voxel volume metrics (same units as
the input) (default: 20.0)
/nodata:<value> Value for a cloud/feature with zero points at all: NA, or a number
such as 0, -9999, or inf (default: NA)
/intstrata:<value> Comma-separated height thresholds bucketing points the same way
as /strata, but reporting an intensity statistic bundle per bucket instead of elevation
(defaults to /strata's thresholds if omitted). (default: )
/noheight:<value> Value for height-dependent columns when points exist but none clear
the height cutoff: NA, or a number such as 0, -9999, or inf (default: 0)

```

22.11 topometrics

```

Usage: topometrics <input_dem.tif> [other /options]
Computes Topographic Terrain Metrics (Slope, Aspect) from DEM GeoTIFF

Options:
/noraster Skip writing the GeoTIFF raster -- only valid together with /output-table,
since a run must produce at least one output
/output-table:<value> Also write a multicolumn table alongside the raster (path
ending in .csv or .sqlite) -- omit to skip table output entirely (default: )
/output:<value> Output multi-band GeoTIFF raster path (default: topo_metrics.tif)

```

22.12 gridsurfacestats

```

Usage: gridsurfacestats <input_surface.tif> [other /options]
Computes surface area ratio, roughness, and (with /reference) cut/fill volume from a
DEM/CHM GeoTIFF

Options:
/noraster Skip writing the GeoTIFF raster -- only valid together with /output-table,
since a run must produce at least one output
/output-table:<value> Also write a multicolumn table alongside the raster (path
ending in .csv or .sqlite) -- omit to skip table output entirely (default: )
/reference:<value> Second surface GeoTIFF (same cols x rows) to diff against for
cut/fill volume_diff (default: )
/output:<value> Output multi-band GeoTIFF raster path (default: surface_stats.tif)

```

22.13 canopymaxima

Usage: canopymaxima <input_chm.tif> [other /options]
Individual Tree Top Detection via Variable Window Local Maxima (VLM) on CHM GeoTIFF

Options:

/window-a:<value> Window polynomial coefficient A (constant) (default: 0.5)
/minht:<value> Minimum tree height threshold (same units as the input) (default: 2.0)
/window-b:<value> Window polynomial coefficient B (linear) (default: 0.05)
/output:<value> Output CSV file path for tree tops (default: tree_tops.csv)

22.14 treeseg

Usage: treeseg <input_chm.tif> [other /options]
Individual Tree Crown Watershed Segmentation on CHM GeoTIFF

Options:

/minht:<value> Minimum height cutoff for segmentation (same units as the input) (default: 2.0)
/output-table:<value> Output CSV summary table path (default: crown_summary.csv)
/output-raster:<value> Output GeoTIFF raster path for tree segments (default: crown_segments.tif)

22.15 pipeline

Usage: pipeline <arguments> [other /options]
Multi-tool batch pipeline: tiles/buffers the input once and chains any of the FUSION2 tools per tile

Options:

/slope Forwarded to whichever stage(s) accept it
/nointensity Forwarded to whichever stage(s) accept it
/first Forwarded to whichever stage(s) accept it
/cleanup Delete the processing subfolder after a fully successful run
/merge Also merge each raster stage's VRT into a single global GeoTIFF
/retryfailed Process only tiles with a recorded failed stage
/rebuild Ignore recorded state and redo every tile/stage
/minz:<value> Forwarded to whichever stage(s) accept it (default:)
/maxz:<value> Forwarded to whichever stage(s) accept it (default:)
/window-a:<value> Forwarded to whichever stage(s) accept it (default:)
/return:<value> Forwarded to whichever stage(s) accept it (default:)
/ground:<value> Forwarded to whichever stage(s) accept it (default:)
/window-b:<value> Forwarded to whichever stage(s) accept it (default:)
/intstrata:<value> Forwarded to whichever stage(s) accept it (default:)
/strata:<value> Forwarded to whichever stage(s) accept it (default:)
/outlier:<value> Forwarded to whichever stage(s) accept it (default:)
/tool:<value> Shorthand for a single-stage /pipeline:<name> (default:)
/tilesize:<value> Tile width,height in project units (default: 1000,1000)
/class:<value> Forwarded to whichever stage(s) accept it (default:)
/input:<value> Input directory of LAS/LAZ files, or a single LAS/LAZ file (default:)
/smooth:<value> Forwarded to whichever stage(s) accept it (default:)
/toolsdir:<value> Directory containing the sibling tool .exe files (default: this executable's own directory) (default:)
/output:<value> Output directory for finalized rasters/tables (default:)
/extent:<value> Project extent LLX,LLY,URX,URY (default:)
/pipeline:<value> Comma-separated ordered list of stages to chain per tile, e.g. groundfilter,canopymodel,canopymaxima (default:)

```
/buffer:<value>   Tile buffer distance (default: 50)
/heightcut:<value> Forwarded to whichever stage(s) accept it (default: )
/processingdir:<value> Interim-product/state subfolder (default: <output>/
_processing) (default: )
/cellsize:<value> Forwarded to whichever stage(s) accept it (default: )
/threads:<value>   Number of parallel worker threads (parallel child processes)
(default: 4)
/minht:<value>     Forwarded to whichever stage(s) accept it (default: )
/tiles:<value>     Comma-separated tile names to process (default: all) (default: )
```

23. Metric definitions

This reference manual documents the statistical elevation, canopy structure, return intensity, pulse density, surface, and experimental spatial metrics computed by `gridmetrics`, `cloudmetrics`, `gridsurfacestats`, `densitymetrics`, and `ExperimentalMetrics`.

Every metric below is computed only from points admitted by each tool's shared `/class//return` point filtering (excluding ASPRS noise classes 7/18 and withheld points by default) – see Section 3.4.

23.1 NA vs. 0 – the `/nodata` and `/noheight` sentinel convention

`gridmetrics` and `cloudmetrics` distinguish two different “empty” conditions, each with its own independently-overridable placeholder value:

- **`/nodata` (default NA)** – a cell or cloud with **zero total points**: nothing landed there at all. Every band/column for that cell/cloud gets this value, including counts (`TotalReturns`, `TotalPoints`) and density (`point_density`) – there is no real computed answer here, only an absence of data.
- **`/noheight` (default 0)** – a cell or cloud with **points present but none passing the height cutoff** (`/minht`): a real, meaningful “no canopy here” answer. This applies specifically to the elevation- and intensity-derived bands whose only data source is the height-filtered point set (`elev_*`, `int_*`, and per-stratum stats). Metrics that are well-defined directly from the *unfiltered* return count – `canopy_cover`, `point_density`, per-return-number counts – are **never** overridden by `/noheight`; a computed 0% cover when points exist but none clear the cutoff is a real answer, not a placeholder.

Both options accept NA (case-insensitive) or a literal number (0, -9999, `inf`, ...). CSV output writes the literal text NA when a sentinel resolves to NA; both `fread()` (R `data.table`, default `na.strings` already includes “NA”) and `terra/raster` (a GeoTIFF's registered NoData value, including NaN) pick this up automatically on read, with no reader-side code change needed.

GDAL raster caveat: GDAL registers exactly one NoData value per file. When `/nodata` resolves to NA (the default), the GeoTIFF is created with NaN as its registered NoData value, and `/noheight`'s default 0 is written as an ordinary, valid pixel – the clean default case. If both options are explicitly set to different, non-NA values, only one can be the file's registered NoData; the tool prints a warning and uses `/nodata`'s value, with `/noheight`'s value still written as a real (not NoData-flagged) pixel.

23.2 Elevation Statistics & Percentiles

Calculated across normalized height values ($Z_i = Z_{point} - Z_{ground}$) for all points or canopy points ($Z_i \geq h_{min}$).

Metric	Name	Formula / Description
<code>elev_min</code>	Minimum Height	$\min(Z_i)$
<code>elev_max</code>	Maximum Height	$\max(Z_i)$
<code>elev_mean</code>	Mean Height	$\bar{Z} = \frac{1}{N} \sum_{i=1}^N Z_i$
<code>elev_mode</code>	Mode Height	Peak location of height distribution histogram.
<code>elev_stddev</code>	Standard Deviation	$S = \sqrt{\frac{1}{N-1} \sum (Z_i - \bar{Z})^2}$
<code>elev_variance</code>	Variance	S^2
<code>elev_cv</code>	Coefficient of Variation	$CV = \frac{S}{\bar{Z}}$
<code>elev_skewness</code>	Skewness	$\frac{1}{NS^3} \sum (Z_i - \bar{Z})^3$

Metric	Name	Formula / Description
<code>elev_kurtosis</code>	Kurtosis	$\frac{1}{NS^4} \sum (Z_i - \bar{Z})^4 - 3$
<code>elev_iqr</code>	Interquartile Range	$IQR = P_{75} - P_{25}$
<code>elev_p01-elev_p99</code>	Height Percentiles	1st, 5th, 10th, 20th, 25th, 30th, 40th, 50th, 60th, 70th, 75th, 80th, 90th, 95th, 99th height percentiles.
<code>elev_aad</code>	Average Absolute Deviation	$\frac{1}{N} \sum Z_i - \bar{Z} $ (mean absolute deviation about the mean).
<code>elev_mad_median</code>	MAD about the Median	$\frac{1}{N} \sum Z_i - \text{median}(Z) $ (mean absolute deviation about the median).
<code>elev_mad_mode</code>	MAD about the Mode	$\frac{1}{N} \sum Z_i - \text{mode}(Z) $ (mean absolute deviation about the mode).
<code>elev_l1-elev_l4</code>	L-Moments	Sample L-moments (Hosking 1990), computed from the probability-weighted moments of the sorted values. <code>elev_l1</code> equals <code>elev_mean</code> .
<code>elev_l_cv, elev_l_skewness, elev_l_kurtosis</code>	L-Moment Ratios	$L - CV = L_2/L_1$, $L - skew = L_3/L_2$, $L - kurt = L_4/L_2$ - robust analogues of <code>elev_cv/elev_skewness/elev_kurtosis</code> .
<code>elev_quadratic_mean</code>	Quadratic Mean (RMS)	$\sqrt{\frac{1}{N} \sum Z_i^2}$
<code>elev_cubic_mean</code>	Cubic Mean	$\sqrt[3]{\frac{1}{N} \sum Z_i^3}$
<code>elev_profile_area</code>	Profile Area	Trapezoidal-rule area under the percentile-vs-percentile-rank curve (<code>elev_p01...elev_p99</code> , each height offset from <code>elev_min</code>) - the one legacy elevation column that needs the full percentile curve rather than a single-vector reduction.

The same full bundle (all rows above) is computed for return intensity under the `int_` prefix (`int_min, int_mean, ... int_cubic_mean`; no `int_profile_area`), and, when `/rgb`: selects one or more spectral channels, once more per selected channel under `red/green/blue/nir_` prefixes - same columns as `int_*`, one full set per selected, available spectral channel (RGB populated in LAS point formats 2, 3, 5, 7, 8, 10; NIR only in formats 8 and 10 - see `gridmetrics/cloudmetrics'/rgb`: option in Chapter 21).

23.3 Canopy Cover & Structure Metrics

Metric	Name	Formula / Description
<code>canopy_cover / cover_2m</code>	Canopy Cover Fraction	Fraction of returns above the canopy height cutoff (h_{min} , default 2.0 m):

Metric	Name	Formula / Description
		$\text{Cover} = \frac{N_{Z \geq h_{min}}}{N_{total}} \quad (23.1)$
<code>canopy_relief_ratio</code>	Canopy Relief Ratio (<i>CRR</i>)	Relative position of mean height within total height range: $CRR = \frac{\bar{Z} - Z_{min}}{Z_{max} - Z_{min}} \quad (23.2)$
<code>strata_fraction_i</code>	Height Strata Fraction	Proportion of returns falling into specified height interval $[h_a, h_b]$.
<code>r1count-r9count</code>	Per-Return-Number Counts	<code>gridmetrics</code> only. Raw count of returns with return number 1 through 8, plus an <code>r9count</code> bucket for return number 9 or higher. Computed from the unfiltered return set – <code>/nodata</code> when the cell is empty, never <code>/noheight</code> .
<code>allcover</code>	All>Returns Cover	$100 \times N_{\text{above heightcut}} / N_{\text{total}}$ – the cover fraction using every return as the denominator, regardless of <code>/first</code> .
<code>afcover</code>	All-First Cover	$100 \times N_{\text{above heightcut}} / N_{\text{first}}$ – returns above the height cutoff (from the full return set) as a fraction of first returns only.
<code>allabovemean, allabovemode</code>	All>Returns Above Mean/Mode	$100 \times N_{\text{above elev_mean or elev_mode}} / N_{\text{total}}$.
<code>afabovemean, afabovemode</code>	All-First Above Mean/Mode	Same numerator as <code>allabovemean/allabovemode</code> , but as a fraction of first returns only.

23.4 Pulse Density & Return Type Ratios

Calculated by `returndensity.exe` and `gridmetrics.exe`:

Metric	Name	Description
<code>pulse_density</code>	Pulse Density	Total first-returns or unique pulses per square meter (pts/m ²).
<code>point_density</code>	Total Point Density	Total returns of all return types per square meter (pts/m ²).
<code>first_return_ratio</code>	First Return Percentage	$\frac{N_{first}}{N_{total}} \times 100\%$
<code>ground_return_ratio</code>	Ground Return Percentage	Percentage of classified ground returns (<i>Class</i> = 2).

Metric	Name	Description
<code>int_mean_first</code>	First Return Mean Intensity	Mean return intensity of first-returns.
<code>int_mean_all</code>	All Return Mean Intensity	Mean return intensity across all returns.

23.5 Surface Area Ratio, Roughness & Cut/Fill Volume

Computed by `fusion::metrics::ComputeSurfaceStatsGrid/SummarizeSurfaceStats` (`SurfaceStats.h`), and exposed from three places: the standalone `gridsurfacestats.exe`, `gridmetrics.exe`'s `/surfstats` flag (per-cell raster bands, fed from the same `elev_max/elev_mean` grid `gridmetrics` already holds), and `cloudmetrics.exe`'s `/surfstats` flag (scalar summary over a scratch / `cellsize` grid). No TIN/Delaunay triangulation is used – surface area ratio is a direct closed-form derivation from Horn's-method slope (the same 3x3-neighbor calculation `topometrics.exe` uses for slope/aspect).

Metric	Name	Formula / Description
<code>surface_area_ratio</code>	Surface Area Ratio	3D surface area / 2D planimetric area for one cell's 3x3 neighborhood: $\frac{1}{\cos(\text{slope})}$. Only computable for interior cells with a full, noData-free 3x3 neighborhood – border/edge cells get <code>/nodata</code> .
<code>roughness</code>	Roughness	Local elevation standard deviation across the same 3x3 window.
<code>volume_diff</code>	Cut/Fill Volume Difference	Per-cell <code>elevation - reference</code> , only present when a second surface is supplied (<code>gridsurfacestats' /reference</code>). Positive = fill, negative = cut.
<code>SurfaceAreaRatioMean</code> , <code>RoughnessMean</code>	(<code>cloudmetrics /surfstats</code>)	Mean of the per-cell values above, over cells with a computed value.
<code>PlanimetricArea</code> , <code>SurfaceArea3D</code>	(<code>cloudmetrics /surfstats</code>)	Total 2D footprint area and total 3D surface area (border cells assumed flat) over the point cloud's <code>/cellsize</code> grid.

23.6 Vertical Density Stratum Bands & Per-Stratum Statistic Bundles

`densitymetrics.exe` (standalone), `gridmetrics.exe`'s `/strataraster` add-on, and `gridmetrics' /cloudmetrics' /strata`, `/intstrata`, and `/rgbstrata` all share the same bucket-assignment rule (`fusion::metrics::AssignStratumIndex`): a point's stratum bucket is the first `/strata` threshold its (ground-normalized) elevation is still $\geq$, or the bucket past the last threshold if it clears all of them.

Per-stratum output uses a simplified 6-field summary (`fusion::metrics::StrataStatBundle`) rather than the full ~38-field bundle in Section 1 – a stratum bucket is one of several repeated per row/cell, so the full percentile/L-moment set would multiply out to an impractical column count.

Metric	Name	Description
density_stratum_00, density_stratum_01, ...	Per-Stratum Return Density	<code>densitymetrics/</code> <code>gridmetrics /strataraster</code> raster bands: that bucket's return count per unit area (<code>count / cellsize^2</code>) for the cell. A cell with zero total points gets <code>/nodata</code> across every stratum band; a non- empty cell's individual bands are real computed values (including a legitimate 0 for an empty bucket), never <code>/</code> <code>noheight</code> .
stratum_N_count, stratum_N_proportion, stratum_N_mean, stratum_N_stddev, stratum_N_min, stratum_N_max	Per-Stratum Elevation Summary	<code>gridmetrics/cloudmetrics /</code> <code>strata</code> : bucket N's raw return count and its proportion of the cell/cloud's total returns (always real values, including 0, once the cell/cloud has any returns at all), plus the mean/ stddev/min/max of bucket N's own elevation values. A bucket with zero points in an otherwise non-empty cell/ cloud gets <code>/noheight</code> for the mean/stddev/min/max columns only.
intstratum_N_count, intstratum_N_proportion, intstratum_N_mean, intstratum_N_stddev, intstratum_N_min, intstratum_N_max	Per-Stratum Intensity Summary	<code>gridmetrics/cloudmetrics /</code> <code>intstrata</code> : same bucket assignment as <code>/strata</code> (elevation-based, defaults to <code>/</code> <code>strata</code> 's thresholds), but computed from each bucket's intensity values instead of elevation. <code>cloudmetrics</code> and <code>gridmetrics</code> both emit these as CSV columns whenever <code>/</code> <code>intstrata</code> is given (and <code>/</code> <code>nointensity</code> is not set); <code>gridmetrics</code> additionally emits matching raster bands when <code>/</code> <code>strataraster</code> is also set – no density band, since return density is a spatial-return concept tied to the elevation bucketing, not to intensity.
<channel>_stratum_N_mean, <channel>_stratum_N_stddev, <channel>_stratum_N_min, <channel>_stratum_N_max	Per-Stratum Spectral Summary	<code>gridmetrics/cloudmetrics /</code> <code>rgbstrata</code> (with <code>/rgb</code> and <code>/</code> <code>strata</code> both set): mean/stddev/ min/max of bucket N's spectral values for each selected, format-carried channel (<code>red</code> , <code>green</code> , <code>blue</code> , <code>nir</code>), using the same elevation-based buckets as <code>/strata</code> . Count/proportion

Metric	Name	Description
		aren't repeated here since the elevation <code>stratum_N_*</code> columns/bands sharing the same buckets already report them; a bucket with zero points gets <code>/noheight</code> for these four columns.

23.7 Novel 2D & 3D Experimental Spatial Metrics

Computed by `fusion::metrics::ComputeExperimentalMetrics` in `ExperimentalMetrics.h` using 2D grid resolution (`cellSize`) and 3D voxel resolution (`voxelSize`).

23.7.1 Relative Height Ratio Metrics

Ratio of canopy height parameters ($Z \geq h_{min}$) relative to overall plot height statistics:

- **zMinRat**: Relative minimum canopy height ratio ($|\frac{Z_{min,canopy} - Z_{min,all}}{Z_{min,all}}| \times 100$).
- **zMaxRat**: Canopy maximum relative ratio ($100 \times \frac{Z_{max,canopy}}{Z_{max,all}}$).
- **zSdRat**: Canopy height standard deviation ratio ($100 \times \frac{S_{canopy}}{S_{all}}$).
- **zMeanRat**: Canopy mean height ratio ($100 \times \frac{\bar{Z}_{canopy}}{\bar{Z}_{all}}$).
- **zCvRat**: Canopy height CV ratio ($100 \times \frac{CV_{canopy}}{CV_{all}}$).

23.7.2 2D & 3D Spatial Radial Distances & Correlations

- **2D Radial Distance**: $R_{xy} = \sqrt{X^2 + Y^2}$
 - `xyMinRt`, `xyMaxRt`, `xySdRt`, `xyMnRt`, `xyCvRt`: Min, Max, StdDev, Mean, and CV of 2D radial distances.
 - `xyMinRtRat`, `xyMaxRtRat`, `xySdRtRat`, `xyMnRtRat`, `xyCvRtRat`: Ratio of canopy subset to overall 2D radial metrics ($100 \times \frac{M_{canopy}}{M_{all}}$).
 - `xyCor`, `xyCorRat`: Pearson correlation coefficient between X and Y coordinates ($\rho_{X,Y}$).
- **3D Radial Distance**: $R_{xyz} = \sqrt{X^2 + Y^2 + Z^2}$
 - `xyzMinRt`, `xyzMaxRt`, `xyzSdRt`, `xyzMnRt`, `xyzCvRt`: Min, Max, StdDev, Mean, and CV of 3D radial distances.
 - `xyzCor`, `xyzCorRat`: Pearson correlation coefficient between 2D radial distance R_{xy} and height Z ($\rho_{R_{xy},Z}$).

23.7.3 2D Area & Columnar Grid Volume

Grid cells of size `cellSize` × `cellSize` ($A_{cell} = \text{cellSize}^2$).

- **gridAreaAll**: Total footprint area of occupied 2D grid cells ($N_{cells,all} \times A_{cell}$).
- **gridArea**: Canopy footprint area of grid cells containing canopy returns ($N_{cells,canopy} \times A_{cell}$).
- **areaCover**: Canopy area coverage ratio ($100 \times \frac{\text{gridArea}}{\text{gridAreaAll}}$).
- **gridVolAll**: Total column grid volume ($\sum_{\text{occupied cells}} A_{cell} \times (Z_{max,cell} - Z_{min,cell})$).
- **gridVol**: Canopy column grid volume ($\sum_{\text{canopy cells}} A_{cell} \times (Z_{max,cell} - Z_{min,cell})$).
- **gridVolRat**: Canopy column volume percentage ($100 \times \frac{\text{gridVol}}{\text{gridVolAll}}$).

23.7.4 3D Voxel Volume

Voxels of size `voxelSize` × `voxelSize` × `voxelSize` ($V_{voxel} = \text{voxelSize}^3$).

- **voxVolAll**: Total occupied 3D voxel volume ($N_{voxels,all} \times V_{voxel}$).

- **voxVol**: Occupied canopy voxel volume ($N_{voxels,canopy} \times V_{voxel}$).
- **voxVolRat**: Voxel volume canopy ratio ($100 \times \frac{voxVol}{voxVolAll}$).
- **voxVolScl**: Scaled voxel occupancy metric ($\frac{voxVol}{cellSize \times voxelSize}$).

24. C++ library interface (`libfusion_core`)

This reference documents the C++ classes, structs, namespaces, and functions provided by `libfusion_core`. The library is compiled with C++17 and relies on GDAL and LASlib/LASzip for robust geospatial point cloud and raster I/O.

24.1 Point Cloud I/O (`fusion::lidar`)

Header: `#include "fusion/lidar/LASPointCloud.h"`

24.1.1 `fusion::lidar::PointRecord`

Represents a single 3D point record extracted from a `.las` or `.laz` file.

```
struct PointRecord {
    double x{0.0};
    double y{0.0};
    double z{0.0};
    uint16_t intensity{0};
    uint8_t returnNumber{1};
    uint8_t numberOfReturns{1};
    uint8_t classification{0};
    int16_t scanAngle{0};
    uint16_t pointSourceID{0};
    double gpsTime{0.0};
    uint16_t red{0};
    uint16_t green{0};
    uint16_t blue{0};
    uint16_t nir{0};
    bool withheld{false};
    bool keypoint{false};
    bool synthetic{false};
    bool overlap{false};
    uint8_t scannerChannel{0};
};
```

24.1.1.1 Fields

- `x, y, z` (double): Georeferenced spatial coordinates (in project units, e.g., meters or feet).
- `intensity` (uint16_t): Pulse return intensity (0–65535).
- `returnNumber` (uint8_t): Return number of the pulse (1-indexed).
- `numberOfReturns` (uint8_t): Total number of returns generated by the laser pulse.
- `classification` (uint8_t): ASPRS classification code (e.g., 2 = Ground, 3 = Low Veg, 4 = Medium Veg, 5 = High Veg).
- `scanAngle` (int16_t): Scan angle rank (-90 to +90 degrees).
- `pointSourceID` (uint16_t): Flight line or sensor source identifier.
- `gpsTime` (double): Adjusted GPS timestamp or time of day.
- `red, green, blue, nir` (uint16_t): Multispectral/RGB channel values.
- `withheld, keypoint, synthetic, overlap` (bool): ASPRS point classification flags.
- `scannerChannel` (uint8_t): Laser channel ID (for multi-channel lidar systems).

24.1.2 `fusion::lidar::LASHeaderInfo`

Metadata header structure describing an open `.las` or `.laz` dataset.

```
struct LASHeaderInfo {
    uint64_t pointCount{0};
    double minX{0.0}, maxX{0.0};
    double minY{0.0}, maxY{0.0};
    double minZ{0.0}, maxZ{0.0};
    double xScaleFactor{0.001}, yScaleFactor{0.001}, zScaleFactor{0.001};
    double xOffset{0.0}, yOffset{0.0}, zOffset{0.0};
    uint8_t pointFormat{0};
    uint8_t versionMajor{1}, versionMinor{2};
    bool isCompressed{false};
    std::string systemID;
    std::string generatingSoftware;
};
```

24.1.3 `fusion::lidar::LASReader`

Sequential reader class for reading `.las` and `.laz` point cloud files.

24.1.3.1 Methods

- `bool Open(const std::filesystem::path& filePath)`
 - **Parameters:** `filePath` - Absolute or relative path to the `.las` or `.laz` file.
 - **Returns:** `true` if file opened successfully; `false` on file missing or corrupt header.
- `void Close()`
 - Closes the active file handle and frees internal reader memory.
- `bool IsOpen() const`
 - Returns `true` if a file is currently loaded and ready for reading.
- `const LASHeaderInfo& GetHeader() const`
 - Returns a reference to the parsed header metadata structure.
- `uint64_t GetPointCount() const`
 - Convenience method returning total point count in the dataset.
- `bool ReadNextPoint(PointRecord& pt)`
 - **Parameters:** `pt` - Output record populated with the next point attributes.
 - **Returns:** `true` if point was successfully read; `false` on reaching end of file (EOF).
- `void Rewind()`
 - Resets the reader stream cursor back to the first point.

24.1.4 `fusion::lidar::LASWriter`

Writer class for exporting modified or generated point clouds to `.las` or `.laz` files.

24.1.4.1 Methods

- `bool Open(const std::filesystem::path& filePath, const LASHeaderInfo& header)`
 - **Parameters:** `filePath` - Target output path. `header` - Metadata header defining point format and scale factors.
 - **Returns:** `true` if output file created successfully.
- `bool WritePoint(const PointRecord& pt)`
 - **Parameters:** `pt` - Point record to append to dataset.
 - **Returns:** `true` on success.
- `void Close()`
 - Flushes output stream buffers and updates point count metadata in file header.

24.2 Raster I/O & GDAL Engine (fusion::raster)

Header: #include "fusion/raster/GDALRaster.h"

24.2.1 Enums & Data Structures

```
enum class SampleMethod { Nearest, Bilinear };
enum class OutputMode { SingleBand, MultiBand };

struct RasterInfo {
    int width{0};
    int height{0};
    int numBands{0};
    double originX{0.0};
    double originY{0.0};
    double pixelWidth{1.0};
    double pixelHeight{1.0};
    double minX{0.0}, maxX{0.0};
    double minY{0.0}, maxY{0.0};
    double noDataValue{-9999.0};
    bool hasNoData{false};
    std::string projectionWKT;
};
```

24.2.2 fusion::raster::GDALRaster

Wrapper around GDAL datasets supporting single-band ground DEMs and multi-band canopy metric GeoTIFF rasters.

24.2.2.1 Methods

- `bool Open(const std::filesystem::path& filePath, bool readWrite = false)`
– Opens existing raster file (GeoTIFF, ERDAS Image, AAIGrid, etc.).
- `bool Create(const std::filesystem::path& filePath, int width, int height, int numBands = 1, const std::string& dataType = "Float32", const std::string& driverName = "GTiff", const std::string& projectionWKT = "", const double* geotransform = nullptr, double noDataValue = -9999.0)`
– Creates a new GDAL raster dataset with specified resolution, bands, driver, and coordinate system.
- `std::optional<double> GetElevation(double x, double y, SampleMethod method = SampleMethod::Bilinear, int band = 1) const`
– Samples continuous elevation or surface height at world coordinates (x, y) using nearest-neighbor or bilinear interpolation.
- `std::optional<double> GetCellValue(int col, int row, int band = 1) const`
– Reads specific cell value by 0-indexed column and row indices.
- `bool SetCellValue(int col, int row, double value, int band = 1)`
– Writes cell value at column and row indices.
- `bool SetBandDescription(int band, const std::string& description)`
– Sets descriptive string tag for a GDAL raster band (e.g. "elev_p95" or "canopy_cover").
- `std::string GetBandDescription(int band) const`
– Retrieves description string for band.
- `bool ReadBandData(int band, std::vector<float>& buffer) const`
– Reads full raster band block into a contiguous float vector buffer ($\text{width} * \text{height}$).
- `bool WriteBandData(int band, const std::vector<float>& buffer)`
– Writes float vector buffer to target band.
- `static bool BuildVRT(const std::filesystem::path& outputVRTPath, const std::vector<std::filesystem::path>& inputRasterPaths, bool relativePaths = true)`
– Compiles multiple tile rasters into a unified Virtual Raster (.vrt) catalog.
- `static bool MergeVRTToGeoTIFF(const std::filesystem::path& vrtPath, const std::filesystem::path& outputGeoTIFFPath, const std::string& compressOption = "LZW")`

- Merges tiles referenced in a VRT into a single GeoTIFF file with optional LZW compression.

24.3 Parallel Batch Engine (**fusion::batch**)

Header: `#include "fusion/batch/BatchPipeline.h"`

24.3.1 Data Structures

```
struct TileGridSpec {
    double minX{0.0}, maxX{0.0};
    double minY{0.0}, maxY{0.0};
    double tileSizeX{1000.0};
    double tileSizeY{1000.0};
    double bufferDistance{50.0};
    double resolution{1.0};
};

struct TileInfo {
    int tileID{0};
    std::string name;
    double minX{0.0}, maxX{0.0};
    double minY{0.0}, maxY{0.0};
    double bufferedMinX{0.0}, bufferedMaxX{0.0};
    double bufferedMinY{0.0}, bufferedMaxY{0.0};
};
```

24.3.2 **fusion::batch::BatchPipeline**

Engine for distributing tiled processing across multi-core CPUs.

24.3.2.1 Constructors & Methods

- `BatchPipeline(TileGridSpec gridSpec, PipelineJobOptions jobOptions)`
 - Initializes pipeline with spatial bounding extents and job options (thread counts, buffer overlaps, output paths).
- `bool ExecutePipeline(const std::function<bool(const TileInfo& tile, const PipelineJobOptions& opts)>& tileTask)`
 - Dispatches parallel execution across threads. Returns `true` if all tile tasks succeed.
- `const std::vector<std::filesystem::path>& GetGeneratedTileRasters() const`
 - Returns list of generated tile raster paths.
- `std::filesystem::path GetVRTPath() const`
 - Returns generated VRT catalog path.

24.4 Status Logging (**fusion::batch**)

Header: `#include "fusion/batch/StatusMessenger.h"`

24.4.1 **fusion::batch::StatusMessenger**

Thread-safe singleton logger for writing status updates to console stdout and optional log files.

24.4.1.1 Methods

- `static StatusMessenger& Instance()` - Retrieves singleton instance.
- `void SetLogFile(const std::filesystem::path& logPath)` - Sets optional destination file path.
- `void SendStatus(const std::string& statusMessage, bool quiet = false)` - Logs string message.

- `void SendProgress(const std::string& jobName, int current, int total)` - Formats and logs percentage progress string (e.g., Job Progress: 5/10 (50%)).

24.5 Experimental Lidar Metrics (fusion::metrics)

Header: `#include "fusion/metrics/ExperimentalMetrics.h"`

24.5.1 Data Structures

```
struct Point3D {
    double x{0.0};
    double y{0.0};
    double z{0.0};
};

struct ExperimentalMetricsOptions {
    double minHt{2.0};           // Minimum canopy height cutoff
    double cellSize{10.0};       // 2D grid cell resolution (input units)
    double voxelSize{20.0};      // 3D voxel resolution (input units)
};
```

24.5.2 Functions

- `ExperimentalMetricsResults ComputeExperimentalMetrics(const std::vector<Point3D>& points, const ExperimentalMetricsOptions& options)`
 - Calculates relative height metrics, 2D/3D radial distances, grid cell area/column volume, and 3D voxel occupancy volume for a point set.
- `std::map<std::string, double> GetExperimentalMetricsAsMap(const ExperimentalMetricsResults& res)`
 - Converts result struct into string key-value map for CSV export.
- `std::vector<std::string> GetExperimentalMetricsNames()`
 - Returns ordered list of metric column names.

24.6 CLI Argument Parsing (fusion::cli)

Header: `#include "fusion/cli/ArgumentParser.h"`

24.6.1 fusion::cli::ArgumentParser

Case-insensitive parser for command-line options and flags supporting Windows slash (`/option=val`), single dash (`-option val`), and double dash (`--option=val`) syntaxes.

24.6.1.1 Methods

- `ArgumentParser(std::string programName, std::string description)`
- `void AddFlag(const std::string& name, const std::string& description)`
- `void AddOption(const std::string& name, const std::string& description, const std::string& defaultValue = "")`
- `bool Parse(int argc, char* argv[])`
- `bool HasFlag(const std::string& name) const`
- `std::optional<std::string> GetOption(const std::string& name) const`
- `const std::vector<std::string>& GetPositionalArgs() const`
- `void PrintHelp() const`

Appendices

A. Unit tests

FUSION2's library code is covered by a set of unit tests in the repository's `tests/cpp/` folder: small programs that feed a function made-up data with a known correct answer and check that the function returns it. `build.ps1` builds and runs them on every build and stops the build if any test fails, so a released build has passed every test below.

Most tests were written alongside a specific fix and reproduce the problem that fix solved, so the same problem cannot return unnoticed. They complement the whole-tool checks in Appendix B., which run the finished tools on real data.

A.1 What each test group covers

Table A.1: Unit test groups and what each protects.

Test group	What it protects
PointFilter	The shared <code>/class</code> and <code>/return</code> filter (Section 3.4): the default noise exclusion, whitelists, blacklists, return-number lists and mnemonics, and that withheld points are always dropped.
StrataStatBundle	The per-height-layer statistics behind <code>/strata</code> : each layer's count, proportion, mean, standard deviation, minimum, and maximum match hand-computed values, an empty layer reports zero without error, and column names stay in step with values.
GetMode	The height mode statistic (<code>elev_mode</code>) finds the most common height bin and breaks ties consistently.
LASWriter/ LASReader	Point clouds written and read back unchanged: point formats 1, 3, 6, and 7 in LAS and compressed LAZ (including every point of a LAS 1.4 format 6 LAZ file), colour values, coordinate-system records (WKT and GeoTIFF keys), and the header's GPS-time and WKT flags.
ChmSmoothing	<code>canopymodel /smooth</code> gives the same result as a plain moving average, including at grid edges and around empty cells, which stay empty.
TableWriter	CSV and SQLite table output (<code>/output-table</code>) writes every row and value, including when per-tile tables are joined into one.
GroundFilter	The Kraus and Pfeifer ground filter (Chapter 8.) on a made-up sloped scene with dense canopy: all ground under the canopy is kept, no canopy is classified as ground, and wide canopy over a ground-return gap is removed by the coarse pass.
GDALRaster::GetElevation	Ground heights read from a raster: exact values between cell centers, edge cells used for points up to one cell outside the raster on every side, nothing returned further out, and a valid neighbour used next to an empty cell.
SurfaceStats	<code>gridsurfacestats</code> ' volume is height difference times cell area and is not multiplied twice when summed; grids with gaps still give values in every cell with data; a known slope gives the exact surface area ratio.
StageRegistry	<code>pipeline</code> knows all 10 of its stages, and the <code>densitymetrics</code> stage receives the ground surface and forwarded options and names its output correctly.
ArgumentParser	All three option spellings (<code>/name:value</code> , <code>-name value</code> , <code>--name=value</code>) set the same options (Chapter 3.).

Test group	What it protects
Point-count raster	<code>catalog /density</code> places its raster on the data's real coordinates with a coordinate system, and counts points lying exactly on the data's edges.

A.2 Results for this release

The output of the test run from the build that produced this manual:

```
PointFilter tests
  all passed
StrataStatBundle tests
  all passed
GetMode (via ComputePointStatBundle) tests
  all passed
LASWriter/LASReader round-trip tests
  all passed
ChmSmoothing tests
  all passed
TableWriter tests
  all passed
GroundFilter (Kraus & Pfeifer) tests
  ground kept 30625/30625 (under canopy 625/625), canopy misclassified 0/10000
  unsupported crown: fine only misclassifies 51 canopy points; with coarse stage 0
  (ground kept 36400/36400)
  all passed
GDALRaster::GetElevation tests
  all passed
SurfaceStats tests
  all passed
StageRegistry tests
  all passed
ArgumentParser tests
  all passed
Point-count raster (catalog /density) tests
  all passed

All fusion_tests passed.
```

To run the tests yourself after building, run `fusion_tests.exe` from the build folder, or `ctest` in that folder.

B. Checks on the example tile

The worked examples in this manual come from a scripted run of all 15 tools on the example tile (Chapter 4.). After running each tool, the script checks the tool's output against an independent calculation made in R from the same points, using the `lidR`, `terra`, and `sf` packages. Each check asks one of three questions:

1. Was the output written, and does it sit on the tile's real coordinates with a coordinate system attached?
2. Do the counts match exactly what the options asked for? For example, a first-returns filter must keep exactly the tile's first returns.
3. Are the values physically plausible? For example, no canopy height may exceed the tallest point in the tile.

Where FUSION2 and R legitimately differ slightly (a point lying exactly on a plot boundary, or two ground surfaces built by different methods), the check allows a stated tolerance, which appears in the check's name.

A check has one of two severities. A `hard` check that fails means the output is wrong or missing. A `soft` check flags something that works but deserves a look, and its failure is reported as `warn`.

B.1 Summary

Table B.1: Automated check results by tool on the example run.

Tool	Checks	Pass	Warn	Fail
reference	1	1	0	0
catalog	9	9	0	0
filterdata	18	18	0	0
thindata	5	5	0	0
groundfilter	10	10	0	0
clipdata	15	15	0	0
canopymodel	13	13	0	0
returndensity	8	8	0	0
densitymetrics	8	8	0	0
gridmetrics	11	11	0	0
cloudmetrics	10	10	0	0
topometrics	16	16	0	0
gridsurfacestats	8	8	0	0
canopymaxima	8	8	0	0
treeseq	11	11	0	0
pipeline	10	10	0	0

The `reference` row is the script's own first check: that the heights it computes in R for comparison fall within a plausible range.

B.2 Every check

Table B.2 lists every check in the order it ran. Long values are shortened with The complete log, with full values, is `docs/examples/tables/all_tools_check_log.csv` in the FUSION2 repository, and every command the run executed is in `docs/examples/tables/all_tools_commands.csv`.

Table B.2: Every automated check on the example run, in the order it ran.

Tool	Check	Severity	Result	Observed	Expected
reference	reference heights stay within a plausible range	hard	pass	-0.63, 176.06	-10 to 300 ft
catalog	catalog: exits with status 0	hard	pass	0	0
catalog	CSV report written	hard	pass	catalog.csv	catalog.csv
catalog	point count matches the tile	hard	pass	70532	70532
catalog	XY bounds match the tile within 0.01 ft	hard	pass	1023755.00, 1023954.98, 59...	1023755.00, 1023954.98, 59...
catalog	Z range matches the tile within 0.01 ft	hard	pass	175.38, 355.99	175.38, 355.99
catalog	/density raster written	hard	pass	1	>= 1 .tif
catalog	density raster: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023954.98, 59...
catalog	density raster: coordinate system attached	soft	pass	present	present
catalog	density raster sums to the point count	hard	pass	70532	70532
filterdata	filterdata first_returns: exits with status 0	hard	pass	0	0
filterdata	first_returns: output readable	hard	pass	ok	ok
filterdata	first_returns: kept count matches R	hard	pass	39120	39120
filterdata	first_returns: every kept point satisfies the filter	hard	pass	0	0 violations
filterdata	first_returns: coordinate system kept	hard	pass	present	present
filterdata	first_returns: withheld points dropped	soft	pass	0	0
filterdata	filterdata ground_class: exits with status 0	hard	pass	0	0
filterdata	ground_class: output readable	hard	pass	ok	ok
filterdata	ground_class: kept count matches R	hard	pass	3672	3672
filterdata	ground_class: every kept point satisfies the filter	hard	pass	0	0 violations

Tool	Check	Severity	Result	Observed	Expected
filterdata	ground_class: coordinate system kept	hard	pass	present	present
filterdata	ground_class: withheld points dropped	soft	pass	0	0
filterdata	filterdata z_band: exits with status 0	hard	pass	0	0
filterdata	z_band: output readable	hard	pass	ok	ok
filterdata	z_band: kept count matches R	hard	pass	11893	11893
filterdata	z_band: every kept point satisfies the filter	hard	pass	0	0 violations
filterdata	z_band: coordinate system kept	hard	pass	present	present
filterdata	z_band: withheld points dropped	soft	pass	0	0
thindata	thindata: exits with status 0	hard	pass	0	0
thindata	output readable	hard	pass	ok	ok
thindata	no cell keeps more than one point	hard	pass	0	0 duplicate cells
thindata	kept count equals occupied cells	hard	pass	10092	10092
thindata	every kept point exists in the tile	hard	pass	0	0 new points
groundfilter	groundfilter: exits with status 0	hard	pass	0	0
groundfilter	ground DEM written	hard	pass	ground_dem.tif	ground_dem.tif
groundfilter	ground DEM: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023954.98, 59...
groundfilter	ground DEM: coordinate system attached	soft	pass	present	present
groundfilter	ground DEM has no empty cells	hard	pass	0	0
groundfilter	ground DEM stays within the ground points' elevation rang...	hard	pass	175.69, 179.39	175.45, 180.76
groundfilter	median FUSION2 - lidR ground difference under 1 ft	hard	pass	0.134	< 1 ft
groundfilter	95th percentile FUSION2 - lidR ground difference under ...	hard	pass	0.681	< 3 ft
groundfilter	/output-points LAS written	hard	pass	present	present

Tool	Check	Severity	Result	Observed	Expected
groundfilter	/output-points holds only ground-classed points	hard	pass	2	2
clipdata	clipdata box: exits with status 0	hard	pass	0	0
clipdata	box: output readable	hard	pass	ok	ok
clipdata	box: kept count matches R	hard	pass	18079	18079
clipdata	clipdata box + height: exits with status 0	hard	pass	0	0
clipdata	box + height: output readable	hard	pass	ok	ok
clipdata	box + height: kept count within 10% of R	hard	pass	16584	16584
clipdata	box + height: every kept point is 2-200 ft above ground (...)	hard	pass	2.00, 177.83	1.5 to 200.5 ft
clipdata	clipdata /multifile: exits with status 0	hard	pass	0	0
clipdata	/multifile: plotA file named from /field	hard	pass	1	1
clipdata	/multifile: plotA count within 0.5% of R	hard	pass	4799	4799
clipdata	/multifile: plotB file named from /field	hard	pass	1	1
clipdata	/multifile: plotB count within 0.5% of R	hard	pass	5505	5505
clipdata	clipdata /outside: exits with status 0	hard	pass	0	0
clipdata	/outside: output readable	hard	pass	ok	ok
clipdata	/outside: count equals tile minus in-plot points (within ...)	hard	pass	59982	59982
canopymodel	canopymodel: exits with status 0	hard	pass	0	0
canopymodel	canopymodel /smooth:3: exits with status 0	hard	pass	0	0
canopymodel	canopymodel /slope: exits with status 0	hard	pass	0	0
canopymodel	canopy height model written	hard	pass	chm.tif	chm.tif
canopymodel	CHM: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023956.00, 59...	1023755.00, 1023954.98, 59...
canopymodel	CHM: coordinate system attached	soft	pass	present	present

Tool	Check	Severity	Result	Observed	Expected
canopymodel	no CHM cell exceeds the tallest point above ground	hard	pass	0 cells; max 177.83	0 cells above 181.06 ft
canopymodel	no CHM cell more than 1 ft below ground	hard	pass	-0.17	>= -1 ft
canopymodel	median CHM - R reference under 0.5 ft (plausible cells)	hard	pass	0	< 0.5 ft
canopymodel	/smooth:3 CHM written	hard	pass	chm_smooth3.tif	chm_smooth3.tif
canopymodel	/smooth:3 lowers cell-to-cell variance	hard	pass	1383.46	< 1599.93
canopymodel	/slope CHM written	hard	pass	chm_slope.tif	chm_slope.tif
canopymodel	/slope changes at least one cell	hard	pass	2.527	> 0
returndensity	returndensity: exits with status 0	hard	pass	0	0
returndensity	density raster written	hard	pass	returndensity.tif	returndensity.tif
returndensity	density raster: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023954.98, 59...
returndensity	density raster: coordinate system attached	soft	pass	present	present
returndensity	density x cell area sums to the point count (within 0.5%)	hard	pass	70286	70286
returndensity	per-cell counts match R exactly	hard	pass	0	0 mismatched cells
returndensity	first-return percent within 0-100	hard	pass	36.96, 88.89	0 to 100
returndensity	density band's unit label matches the tile's linear units...	soft	pass	point_density	no m2 label on a feet-based...
densitymetrics	densitymetrics: exits with status 0	hard	pass	0	0
densitymetrics	stratum raster written	hard	pass	dm_densitymetrics.tif	dm_densitymetrics.tif
densitymetrics	stratum raster: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023954.98, 59...
densitymetrics	stratum raster: coordinate system attached	soft	pass	present	present
densitymetrics	per-cell CSV table written	hard	pass	present	present
densitymetrics	per cell, stratum counts add to total returns	hard	pass	0	0 mismatched cells

Tool	Check	Severity	Result	Observed	Expected
densitymetrics	total returns across cells equals the point count	hard	pass	70286	70286
densitymetrics	top stratum (> 20 ft) count matches R (within 5 points)	hard	pass	59719	59719
gridmetrics	gridmetrics: exits with status 0	hard	pass	0	0
gridmetrics	/output-table path honored	soft	pass	written	written
gridmetrics	metrics raster written	hard	pass	TC_1372_forest200ft_gridmet...	TC_1372_forest200ft_gridmet...
gridmetrics	metrics raster: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023954.98, 59...
gridmetrics	metrics raster: coordinate system attached	soft	pass	present	present
gridmetrics	no band is empty in every cell	hard	pass	none	none
gridmetrics	grid has the expected rows x columns for the tile	hard	pass	20x20	20x20
gridmetrics	per-cell return counts match R	hard	pass	0	0 mismatched cells
gridmetrics	no elev_max cell exceeds the tallest point above ground	hard	pass	0 cells; max 177.83	0 cells above 181.06 ft
gridmetrics	CSV table has one row per raster cell	hard	pass	400	400
gridmetrics	CSV elev_mean matches the raster band	hard	pass	0	all cells equal
cloudmetrics	cloudmetrics tile: exits with status 0	hard	pass	0	0
cloudmetrics	tile metrics CSV written	hard	pass	cloudmetrics_tile.csv	cloudmetrics_tile.csv
cloudmetrics	tile: total points matches the tile	hard	pass	70286	70286
cloudmetrics	tile: elev_max within the tallest point above ground	hard	pass	177.83	<= 181.06 ft
cloudmetrics	tile: canopy points (> 2 ft) match R (within 1%)	hard	pass	62176	62176
cloudmetrics	cloudmetrics /shape: exits with status 0	hard	pass	0	0
cloudmetrics	per-plot metrics CSV written	hard	pass	cloudmetrics_plots.csv	cloudmetrics_plots.csv
cloudmetrics	/shape: one row per plot, labelled from /field	hard	pass	plotA,plotB	plotA,plotB

Tool	Check	Severity	Result	Observed	Expected
cloudmetrics	/shape: plotA point count within 0.5% of R	hard	pass	4799	4799
cloudmetrics	/shape: plotB point count within 0.5% of R	hard	pass	5505	5505
topometrics	topometrics ground DEM: exits with status 0	hard	pass	0	0
topometrics	topometrics ground DEM raster written	hard	pass	topometrics_ground.tif	topometrics_ground.tif
topometrics	topometrics ground DEM: output on the input DEM's grid	hard	pass	1023755.00, 1023955.00, 59...	1023755.00, 1023955.00, 59...
topometrics	topometrics ground DEM: keeps the input's coordinate system	soft	pass	present	present
topometrics	topometrics ground DEM: interior cells have values	hard	pass	0	0
topometrics	topometrics ground DEM: median slope - terra under 0.5 ...	hard	pass	0	< 0.5
topometrics	topometrics ground DEM: aspect matches terra (compass or ...	hard	pass	compass 0; math 98.14	< 5 under one convention
topometrics	topometrics ground DEM: aspect reported as a compass bear...	soft	pass	compass bearing	compass bearing
topometrics	topometrics external DTM: exits with status 0	hard	pass	0	0
topometrics	topometrics external DTM raster written	hard	pass	topometrics_external_dtm.tif	topometrics_external_dtm.tif
topometrics	topometrics external DTM: output on the input DEM's grid	hard	pass	1023705, 1024005, 598825, ...	1023705, 1024005, 598825, ...
topometrics	topometrics external DTM: keeps the input's coordinate sy...	soft	pass	present	present
topometrics	topometrics external DTM: interior cells have values	hard	pass	0	0
topometrics	topometrics external DTM: median slope - terra under 0....	hard	pass	0	< 0.5
topometrics	topometrics external DTM: aspect matches terra (compass o...	hard	pass	compass 0; math 96.12	< 5 under one convention

Tool	Check	Severity	Result	Observed	Expected
topometrics	topometrics external DTM: aspect reported as a compass be...	soft	pass	compass bearing	compass bearing
gridsurfacestats	gridsurfacestats: exits with status 0	hard	pass	0	0
gridsurfacestats	surface stats raster written	hard	pass	gridsurfacestats.tif	gridsurfacestats.tif
gridsurfacestats	surface stats raster: raster sits on the tile's real coord...	hard	pass	1023755.00, 1023956.00, 59...	1023755.00, 1023954.98, 59...
gridsurfacestats	surface stats raster: coordinate system attached	soft	pass	present	present
gridsurfacestats	surface_area_ratio >= 1 everywhere	hard	pass	1.0001	>= 1
gridsurfacestats	roughness >= 0 everywhere	hard	pass	0.1754	>= 0
gridsurfacestats	volume_diff tracks the surface - reference difference	hard	pass	height difference x cell area	one of the two
gridsurfacestats	volume_diff includes cell area (a true volume)	soft	pass	volume	height difference x cell area
canopymaxima	canopymaxima: exits with status 0	hard	pass	0	0
canopymaxima	tree tops CSV written	hard	pass	tree_tops.csv	tree_tops.csv
canopymaxima	at least one tree top found	hard	pass	82	> 0
canopymaxima	every tree top inside the tile	hard	pass	0	0 outside
canopymaxima	every tree top at least /minht (2 ft)	hard	pass	2.31	>= 2
canopymaxima	no tree top exceeds the tallest point above ground	hard	pass	0 tops; max 177.83	0 above 181.06 ft
canopymaxima	X/Y written without scientific-notation rounding	hard	pass	0 rows, e.g. X = 1023789.50	0 rows
canopymaxima	each top's height equals the CHM at its X/Y	hard	pass	0	0 mismatched tops
treeseq	treeseq: exits with status 0	hard	pass	0	0
treeseq	segment raster written	hard	pass	crown_segments.tif	crown_segments.tif
treeseq	crown table written	hard	pass	crown_summary.csv	crown_summary.csv
treeseq	segment raster: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023956.00, 59...	1023755.00, 1023954.98, 59...
treeseq	segment raster: coordinate system attached	soft	pass	present	present

Tool	Check	Severity	Result	Observed	Expected
treeseq	same crowns in the table and the raster	hard	pass	165	165
treeseq	PixelCount matches the segment raster	hard	pass	0	0 mismatched crowns
treeseq	MaxHeight matches the CHM within each crown	hard	pass	0	0 mismatched crowns
treeseq	no crown exceeds the tallest point above ground	hard	pass	0 crowns; max 177.83	0 above 181.06 ft
treeseq	crown area = pixel count x cell area	hard	pass	0	0 mismatched crowns
treeseq	crown area column's unit label matches the tile's units (...)	soft	pass	CrownArea	no m2 label on a feet-based...
pipeline	pipeline: exits with status 0	hard	pass	0	0
pipeline	run state table written	hard	pass	pipeline_state.csv	pipeline_state.csv
pipeline	no tile x stage failed	hard	pass	done	no failed status
pipeline	four tiles processed	hard	pass	4	4
pipeline	mosaicked canopy model written	hard	pass	canopymodel_merged.tif	a canopymodel.tif or .vrt
pipeline	mosaicked CHM: raster sits on the tile's real coordinates	hard	pass	1023755.00, 1023956.00, 59...	1023755.00, 1023954.98, 59...
pipeline	mosaicked CHM matches single-run CHM (median diff < 0.5 ft)	hard	pass	0.217	< 0.5 ft
pipeline	no mosaicked CHM cell exceeds the tallest point above ground	hard	pass	0	0
pipeline	combined tree tops table written	hard	pass	1	>= 1
pipeline	no duplicate tree tops across tile seams	hard	pass	0	0

C. Legacy FUSION programs not in FUSION2

This document inventories the functionality present in legacy FUSION, the USDA Forest Service suite (version 4.40, including the LiDAR Toolkit, LTK), that FUSION2 does not include.

Per guidance, format-conversion programs dedicated strictly to legacy PLANS `.dtm`, binary `.lda`, and ASCII raster formats (such as `ASCII2DTM`, `DTM2ASCII`, `DTM2ENVI`, `DTM2TIF`, `DTM2XYZ`, `XYZ2DTM`, `ClipDTM`, `MergeDTM`, `SplitDTM`, `RepairGridDTM`, `DTMDescribe`, `LDA2LAS`, `LDAtoASCII`, `LDAConvert`, `ASC2ASC`, `ASCII3D`, and `ASCImport`) are excluded from this comparison.

C.1 Summary of Current FUSION2 Toolset

The FUSION2 suite focuses on replacing the legacy `.dtm` raster format with native GDAL GeoTIFF support, streaming direct `.las` and `.laz` point cloud input and output, and multithreading batch execution. It provides 15 command-line tools:

1. `gridmetrics` (gridded canopy and elevation metric rasters, single-file and batch/tiled mode, with height and intensity strata)
2. `cloudmetrics` (point cloud summary metrics, for the whole cloud or one row per `/shape` polygon)
3. `canopymodel` (canopy height model / CHM interpolation)
4. `canopymaxima` (variable-window local maxima tree top detection)
5. `treeseq` (watershed crown segmentation)
6. `groundfilter` (iterative bare-earth ground filtering, DEM generation, and optional ground-point LAS/LAZ output)
7. `clipdata` (bounding-box and shapefile-polygon subsetting, elevation slicing)
8. `filterdata` (elevation, return number, and classification filtering)
9. `thindata` (spatial point cloud decimation per grid cell)
10. `returndensity` (pulse density and return ratio mapping)
11. `densitymetrics` (return counts in height slices above ground, per grid cell)
12. `gridsurfacestats` (surface area ratio, roughness, and cut/fill volume from a surface GeoTIFF)
13. `topometrics` (topographic slope and aspect calculation from DEMs)
14. `catalog` (acquisition coverage reports and point density summary)
15. `pipeline` (multi-stage tile and buffer batch orchestrator)

Every tool that writes a point cloud (`filterdata`, `thindata`, `clipdata`, `groundfilter /output-points`, and `pipeline`'s tile clips) copies the input's coordinate system records into the output, whether they are stored as WKT or as GeoTIFF keys. It also copies the header's global encoding bits: the GPS time type, and the WKT flag that readers such as `lidR`/`rlas` require before they will use a WKT record. Raster and table-writing tools take an opt-in `/output-table:<path.csv|.sqlite>` for a per-cell table. The raster-writing tools also take `/noraster`, which skips the GeoTIFF when only the table is wanted.

C.2 Legacy FUSION functionality missing from FUSION2

C.2.1 Interactive Graphical Environments

1. **FUSION.exe (Interactive 2D Desktop GIS & Workstation):**
 - **Multi-layer GIS Display:** Visual overlay of LiDAR point coverage, georeferenced aerial imagery (DOQs, ortho-imagery, TIFF/JPEG/MrSID/ECW), bare-earth DEMs, canopy height models, and vector GIS layers (ESRI shapefiles of stands, roads, plot centers, and streams).
 - **Interactive Sampling & Plot Extraction:** Interactive drawing of sample plots (circles, rectangles, polygons) to clip points and immediately send them to visualization or metric calculators.

- **2D/3D Profile Tool:** Interactive profile line drawing to inspect cross-sections of vegetation structure and ground surfaces.
 - **Interactive Tree Measurement:** On-screen stem mapping and manual digitizing of individual tree heights, crown widths, and lean.
 - **GUI Control Center:** Dialog-driven interface to parameterize and execute command-line LTK tools without manual shell scripting.
2. **LDV.exe (3D LiDAR Data Viewer):**
- **Hardware-Accelerated 3D OpenGL Visualization:** High-throughput rendering of raw point clouds with real-time rotation, pan, zoom, and fly-through modes.
 - **Flexible Color Modes:** Dynamic coloring of points by elevation, return intensity, return number, classification code, RGB color attributes, GPS timestamp, or user attributes.
 - **Dynamic Slicing Planes:** Interactive clipping boxes and slicing planes (X, Y, and Z axes) to strip away upper canopy layers and inspect understory or ground returns.
 - **3D Tree Crown & Stem Rendering:** Displaying 3D geometric tree models (conifer and hardwood crown shapes, stem locations, DBH, and lean) rendered directly in the point cloud scene over the terrain surface.
 - **3D Measurement:** Direct point-to-point distance, tree height, crown base height, and canopy dimension measurement tools.
 - **Animation & Trajectory Playback:** Flight-line playback from trajectory files and camera animation sequence recording.
-

C.2.2 Vector & GIS Spatial Subsetting

1. **PolyClipData (Shapefile Polygon Subsetting):**
- Clips point clouds using polygon geometries from ESRI shapefiles.
 - **Multi-file Clipping (/multifile):** Automatically iterates over features in a shapefile (such as forest inventory plots, harvest units, or stand boundaries) and writes a distinct, properly attributed LAS/LAZ file for each polygon, named using a designated attribute column (via /shape:field).
 - Supports inverted clipping (/outside) to extract points outside polygon boundaries.
 - *Status in FUSION2:* ported into clipdata. /shape:<file.shp> clips to polygon features (in addition to any /extent), /multifile writes one LAS/LAZ per feature named by /field:<attribute> (falling back to a zero-padded feature index), and /outside keeps points outside every polygon (single-clip mode only). cloudmetrics /shape similarly computes one metrics row per polygon.
-

C.2.3 Surface Interpolation, Triangulation (TIN), and Quality Control

1. **SurfaceCreate / TINSurfaceCreate:**
- Creates bare-earth or canopy surface models using Delaunay Triangulation (TIN) directly from point data, followed by interpolation onto a grid.
 - Preserves exact point elevations at sample locations without the smoothing or cell-binning artifacts common to raster grid binning.
 - Supports return-filtering (/return:) and classification filtering (/class:).
2. **GridSurfaceCreate:**
- Creates gridded elevation surface models directly from point data with configurable cell operations (minimum, maximum, median, mean) and explicit multi-cell void filling (/fillldist:).
3. **SurfaceFilter / Filter2:**
- Specialized surface filtering tools to despike, median filter, or smooth existing surface models to eliminate noise or vegetation remnants.
4. **fillsurfacevoids:**
- Algorithmic void and hole filling for surface models containing gaps due to water bodies, cloud shadows, or sparse returns.
-

C.2.4 Surface Analysis & Raster Map Algebra

1. **GridSurfaceStats & SurfaceStats:**
- **3D Surface Area vs. Planimetric Area:** Calculates the true 3D surface area of complex terrain (accounting for slope and micro-topography) compared to nominal 2D planimetric area.

- **Volume & Cut/Fill:** Calculates volume between a surface and a reference datum or bare-earth ground model.
 - **Roughness & Topographic Complexity:** Computes surface roughness and terrain texture metrics across user-specified sample factors.
 - *Status in FUSION2:* largely ported. `gridsurfacestats` writes per-cell surface area ratio and roughness bands from a surface GeoTIFF, plus a `volume_diff` cut/fill band when given a second surface with `/reference`. `cloudmetrics /surfstats` reports surface area ratio, roughness, planimetric area, and 3D surface area for a point cloud's top-of-cloud grid. Legacy's user-specified sample factors have no direct equivalent.
 - 2. **GridSample & SurfaceSample:**
 - Extracts surface model elevations at specific point coordinates provided in a CSV or text table (such as field inventory plot centers), with optional window neighborhood sampling.
 - 3. **ModelMath & SRSGridMath:**
 - Surface model map algebra operations (addition, subtraction, absolute difference, scaling, multiplication).
 - Used to subtract an earlier surface model from a later one to compute canopy growth, vegetation loss from disturbances, or terrain cut-and-fill.
 - 4. **diffimage:**
 - Raster differencing tool to compare two surface models and produce difference maps flagging changes exceeding a threshold.
 - 5. **SurfaceColor:**
 - Produces georeferenced color images from surface models based on user-defined elevation/height color ramp rules.
-

C.2.5 Point Cloud Processing & Derivative Products

1. **IntensityImage (Ortho-intensity Image Generation):**
 - Generates georeferenced grayscale or color raster images based on point return intensities.
 - Features histogram percentile contrast stretching (`/minint`, `/maxint`), pixel jittering/anti-aliasing, and return-type selection (`/allreturns`, `/lowest`).
 - Serves as a surrogate aerial photograph in areas where optical imagery is unavailable or misaligned with LiDAR data.
 2. **imagecreate & TiledImageMap:**
 - Generates tiled image pyramids and interactive HTML image map index pages linking master overviews to high-resolution tiles.
 3. **FirstLastReturn:**
 - Extracts first and last returns into separate files.
 - Includes the `/lastnotfirst` switch, which isolates true penetrated returns (last returns from pulses that had multiple returns) from single-return pulses.
 4. **DensityMetrics:**
 - Computes return density across vertical height slices (elevation bands above ground) on a spatial grid, evaluating vertical distribution and foliage layer density.
 - *Status in FUSION2:* ported as `densitymetrics`. Height-slice breaks are set with `/strata`, heights are taken above a `/ground` DEM, and output is a multi-band GeoTIFF plus an optional per-cell CSV or SQLite table (`/output-table`) of stratum counts and total returns.
 5. **Cover:**
 - Dedicated canopy closure tool calculating cover and penetration ratios across multiple height thresholds.
 6. **VegMask:**
 - Generates binary vegetation masks based on point height variance, standard deviation, and cover thresholds within grid cells.
 7. **MergeData:**
 - Standalone utility to merge multiple point cloud files into a single LAS/LAZ file.
 8. **Classify:**
 - Rule-based point reclassification tool based on height above ground, intensity, or spatial criteria.
-

C.2.6 Forestry-Specific Stem Modeling & Tree Inventory Utilities

1. **TreePoints & treebase:**
 - Converts tabular tree lists (with coordinates, heights, crown ratios, and species) into synthetic 3D point clouds representing stems and crown envelopes.
 - Identifies tree base ground elevations from bare-earth models.
 2. **Stem Mapping with Lean & DBH:**
 - In legacy FUSION 4.40, tree data structures incorporated explicit support for DBH, lean angle from vertical, and lean azimuth, with visual representation in LDV.
 3. **TreeSeg Individual Tree Point Extraction:**
 - Legacy TreeSeg supported clipping and exporting point clouds for each individual segmented tree crown basin into separate files. In FUSION2, `treeSeg` produces the crown raster and tabular metrics, but point clipping for individual segments is not yet enabled.
-

C.2.7 Trajectory & Full-Waveform Processing

1. **FWFProcess:**
 - Decomposes digitized full-waveform LiDAR data into discrete return points and Gaussian waveform components.
 2. **SBETPrep & ProcessRange:**
 - Parses Smooth Best Estimate of Trajectory (SBET) airborne trajectory files to compute aircraft positions, scan ranges, and incidence angles for each pulse.
-

C.2.8 Data Management & Workflow Utilities

1. **JoinDB:**
 - Relational table joining utility to combine columns from two data tables (e.g., merging LiDAR metric CSVs with field-measured plot inventory tables).
 2. **SRSPlotSummary, SRSAggregate, SRSClassify, SRSSummarize, SRSAdjustIntensity:**
 - Tools developed for regional forest inventory analysis, automating plot metric aggregation, swath intensity normalization, and metric-based forest type classification.
 3. **Identify:**
 - Queries metadata stored in output files to recover the specific legacy FUSION program version, execution timestamp, and command-line parameters used to generate them.
-

C.3 Option and Feature Gaps Within Ported Tools

Beyond entirely omitted tools, several tools that were ported to FUSION2 omit specific options or capabilities found in their legacy equivalents:

Ported Tool	Missing Legacy Feature or Parameter	Practical Impact
<code>clipdata</code>	Missing <code>/anglemin//anglemax, /timemin//timemax, /biaselev</code> .	Cannot filter by scan angle or GPS timestamp while clipping, or shift output elevations.
<code>cloudmetrics</code>	Missing direct plot-coordinate list processing (<code>/id, radius</code>).	Plot metrics need plot polygons (<code>/shape</code>) rather than a list of plot centers and a radius.
<code>groundfilter</code>	Uses legacy's Kraus & Pfeifer filter and switches (<code>/gparam, /wparam, /aparam, /bparam, /iterations, /tolerance</code>), but its intermediate surfaces are local weighted planes plus a coarse-to-fine stage (<code>/</code>	A fully reclassified point cloud cannot be written; results agree with, but are not identical to, legacy GroundFilter.

Ported Tool	Missing Legacy Feature or Parameter	Practical Impact
	<code>filtercell</code> , <code>/coarsecell</code> , <code>/coarsecut</code>), not legacy's own surface interpolation, so classifications will not match legacy point for point. <code>/output-points</code> writes only the ground points (as class 2), not the whole cloud with new classifications.	
<code>canopymodel</code>	Omission of Delaunay TIN interpolation (<code>/tin</code>), peak preservation (<code>/peaks</code>), and texture metrics (<code>/texture</code>).	CHM creation relies solely on grid binning and raster smoothing rather than facet interpolation.
<code>thindata</code>	Omission of targeted point selection algorithms (lowest, highest, closest to center) and density-based thinning (<code>/density</code>).	Thins strictly to one point per grid cell rather than decimating to a uniform pulse density.
<code>treeseq</code>	Individual tree point cloud clipping is not yet implemented.	Crown segments are output as rasters and summary tables, but individual tree LAS point clips are not written.
<code>returndensity</code>	Grid alignment options (<code>/align</code> , <code>/gridxy</code> , <code>/extent</code>) are simplified.	Mosaicking return density rasters across adjacent custom tiles requires post-processing with GDAL.

C.4 Architectural Summary

FUSION2 adds 4.5 to 6.6 times the throughput of legacy FUSION, along with streaming point cloud I/O, multi-threading, and native GeoTIFF raster support. Its scope is the core automated forest modeling pipeline:

Point Cloud → Ground Filter → CHM → Tree Detection / GridMetrics (3.1)

Polygon clipping (`PolyClipData`), surface geometry statistics (`GridSurfaceStats/SurfaceStats`), and height-slice density (`DensityMetrics`) are now covered by `clipdata /shape`, `gridsurfacestats` with `cloudmetrics /surfstats`, and `densitymetrics`. The broader capabilities of legacy FUSION—specifically the **interactive 2D/3D visualization workstation** (`FUSION.exe` and `LDV.exe`), **Delaunay TIN surface generation** (`TINSurfaceCreate`), **ortho-intensity raster imaging** (`IntensityImage`), and **first/last return pulse splitting** (`FirstLastReturn`)—remain exclusive to the legacy codebase.

D. Changelog

The changes in each FUSION2 release, most recent first. This is the repository's `CHANGELOG.md`, which is also bundled with each release.

Notable changes to the FUSION2 tool suite, one section per published version. Format loosely follows [Keep a Changelog](#). Versions match the `VERSION` set in `CMakeLists.txt`'s `project()` call, which `build.ps1` reads to build each release's tag and title – see the README's "Distributing built tools" section.

D.1 [Unreleased]

- `catalog /density` now writes its point-count GeoTIFF on the input's real coordinates, with the input's coordinate system attached. It previously read both from the point reader after closing it, which reset them, so the raster landed at (0, 0) with no coordinate system. Points lying exactly on the input's maximum-X or minimum-Y edge are now counted in the last column or row instead of being dropped. The raster code moved to `fusion::lidar::WritePointCountRaster`, with tests.
- Options written as `--name=value` now work. The argument parser kept one of the two dashes in the option name, so every tool ignored `--output=...`, `--cellsize=...`, and the like, used its defaults, and still reported success. `/name:value` and `-name value` were unaffected.
- Help text and console messages no longer label heights, cell sizes, and voxel sizes as meters; the tools use the input's own units. `pipeline`'s description names the FUSION2 tools.
- The PDF manual is now the FUSION2 User Manual ([FUSION2_Manual.pdf](#)), a Quarto book covering installation, a quick start, the conventions every tool shares, a worked example for each of the 15 tools (commands, options, and figures from one example tile), the option reference with each tool's built-in help, metric definitions, the unit tests, and the automated checks on the example tile. It replaces [FUSION_Documentation.pdf](#), which held only the overview page.
- `build.ps1` stops if compilation or any unit test fails, and saves the unit-test results and each tool's help text for the manual.

D.2 [0.0.2] - 2026-09-26

- Ground-height lookups (`GDALRaster::GetElevation`, used by every tool that reads a ground DEM) now give a point up to one cell past the right or bottom edge of the DEM the edge value, as they already did past the left and top edges. Such points previously got no ground height.
- Added tests for ground-height lookups near raster edges and nodata cells, surface-statistics volume units and patchy grids, and the `pipeline` stage list's `densitymetrics` entry.
- `groundfilter` now classifies ground with the Kraus & Pfeifer iterative robust filter (legacy FUSION's `/gparam, /wparam, /aparam, /bparam, /iterations, /tolerance`, plus `/filtercell, /coarsecell, /coarsecut`) and fills DEM cells that hold no ground point. It previously took the lowest return in each cell, which put canopy into the DEM wherever a cell had no ground return and left empty cells as nodata. `/output-points` now writes the points the filter classified as ground (as class 2) rather than points already classed 2 in the input.
- LAZ output in LAS 1.4 point formats 6-10 was corrupt (readers decoded only the first few dozen points); uncompressed output and LAS 1.2 formats were unaffected.
- Renamed the project from "FUSION Update" (`fusion_update`) to FUSION2, matching the GitHub repository name: CMake project name, `vcpkg.json` name, LAS header generating-software field, docs, and source headers. `build.ps1` now builds under `%LOCALAPPDATA%\FUSION2_build` and names bundles `FUSION2_tools_v<version>.zip`. `speed_benchmark.qmd` records the new tools as `fusion2`; `fusion_evaluation_report.qmd` still reads earlier results labeled `fusion_update`.
- Point cloud outputs (`filterdata, thindata, clipdata, groundfilter /output-points, pipeline` tile clips) now keep the input's header global encoding bits (GPS time type; WKT flag) and its GeoTIFF-

key coordinate system VLRs. Without the WKT flag, lidR/rLas treated outputs as having no coordinate system; files that stored their coordinate system as GeoTIFF keys lost it entirely.

- `groundfilter /output-points` now carries the input's coordinate system (its header was previously built field by field and omitted it).
- Build links under Rtools 4.5: `nghttp2`, `psl`, `brotli`, and `deflate` are added to the static link group when the toolchain provides them.
- `build_gdal_minimal.psl` replaces a vendored `proj.db` that no longer matches the newest Rtools' PROJ, instead of keeping a stale copy.
- Added a `gridmetrics /intstrata` output (CSV columns and raster bands), opt-in `/output-table` (CSV or SQLite) and `/noraster` switches across the raster tools, and `clipdata /shape,/multifile,/outside`.

D.3 [0.0.1] - 2026-09-09

First draft release of the tool bundle as a GitHub Release asset (`tools-v0.0.1-20260909-124109`).

- Added a shared `PointFilter` module (elevation, return-number, classification, and scan-angle filtering) used across the point cloud tools.
- Added a lightweight, assert-based test harness under `tests/cpp/`, registered with CTest – no GoogleTest/Catch2 dependency.
- Corrected the project version (previously `5.0.0`, a placeholder) so release tags and titles reflect this being the first draft release.
- Linked the `FUSION2-examples` companion repository from the README.